

Network Computing

(058172)



POLITECNICO
MILANO 1863

Datacenter Layer 4 Load Balancing

Some parts of the slides are inspired from the NSDI 2018 talk from Joao Araujo and Marco Chiesa lectures on datacenter load balancing

Content

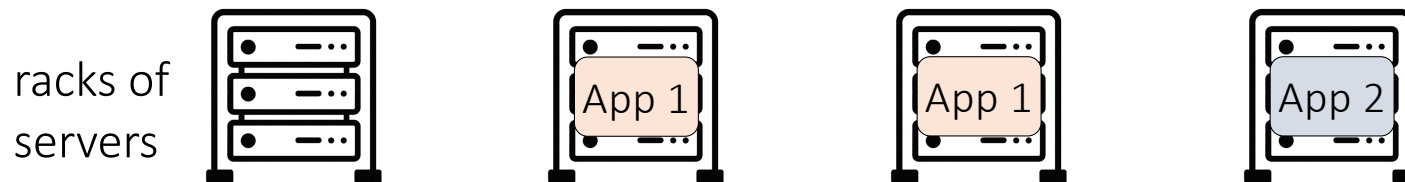
- What is Layer 4 Load Balancing and why we need it
- We will look at three specific solutions:
 - Design space for L4 load balancers
 - Cheetah: a new load balancer proposed by the research community
 - Faild: a production level load balancer for the edge from Fastly

Content

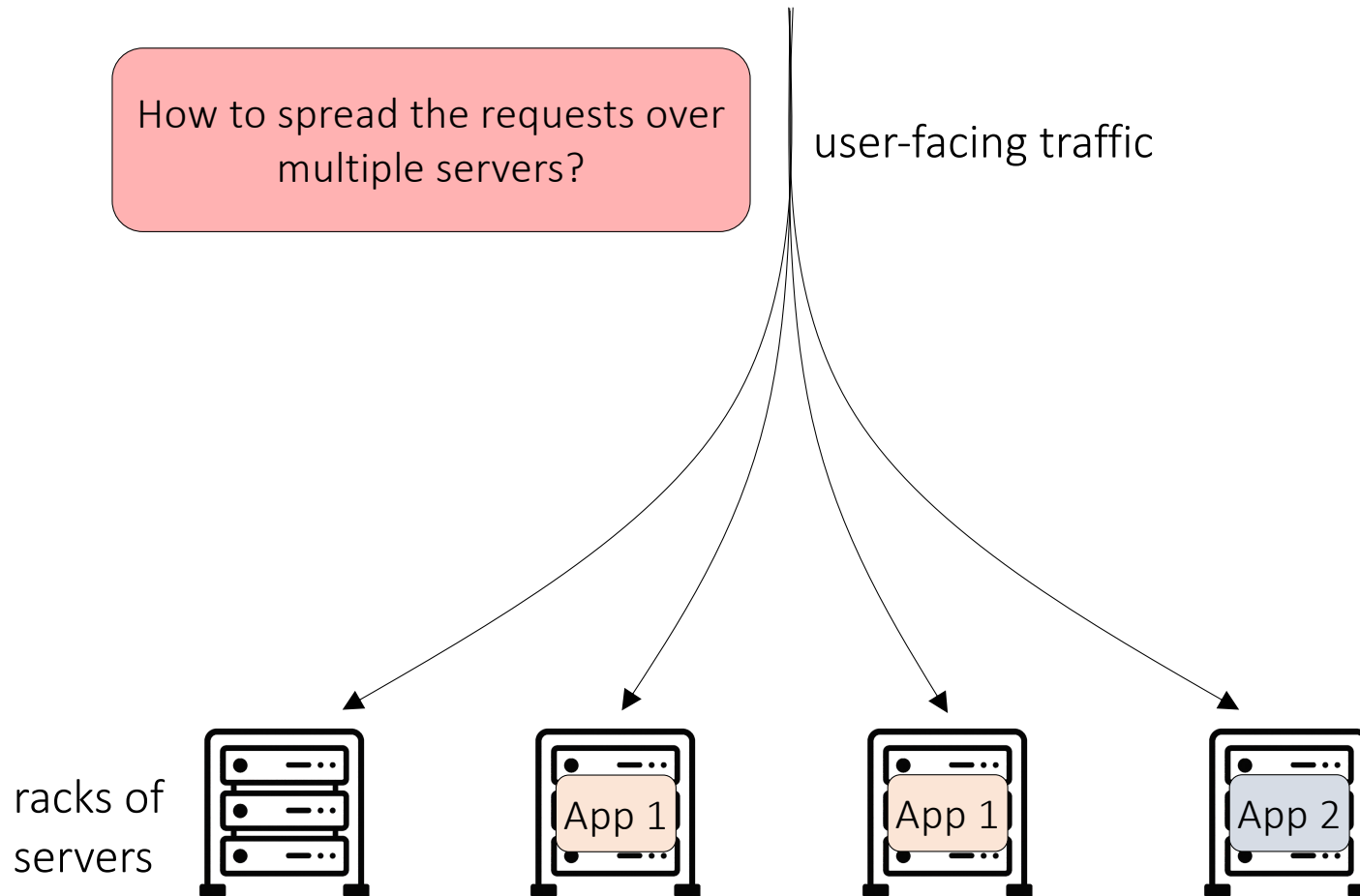
- What is Layer 4 Load Balancing and why we need it
- We will look at three specific solutions:
 - Design space for L4 load balancers
 - Cheetah: a new load balancer proposed by the research community
 - Faild: a production level load balancer for the edge from Fastly

Today we will focus on how to handle user-facing traffic

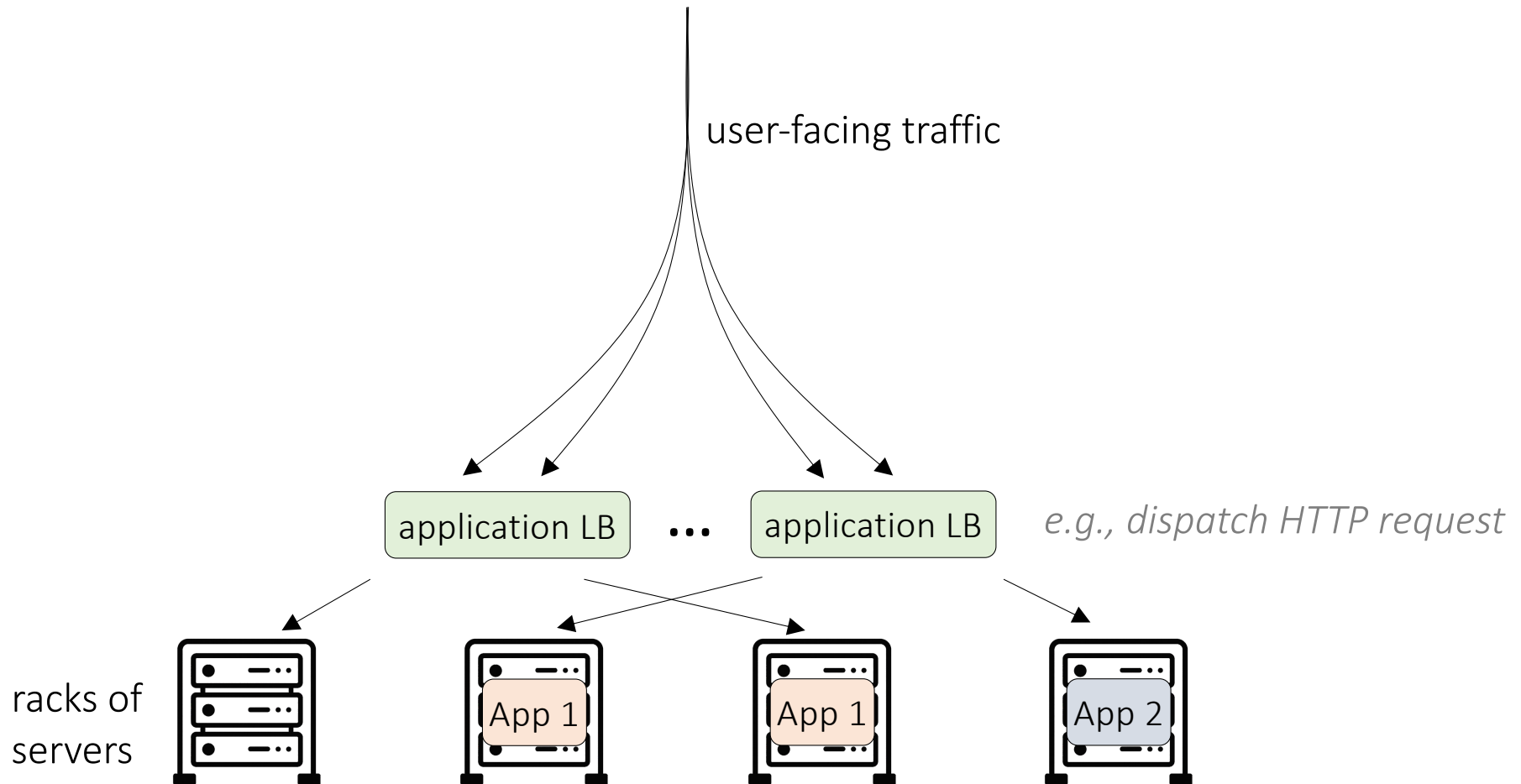
Traditional cloud load balancing architecture



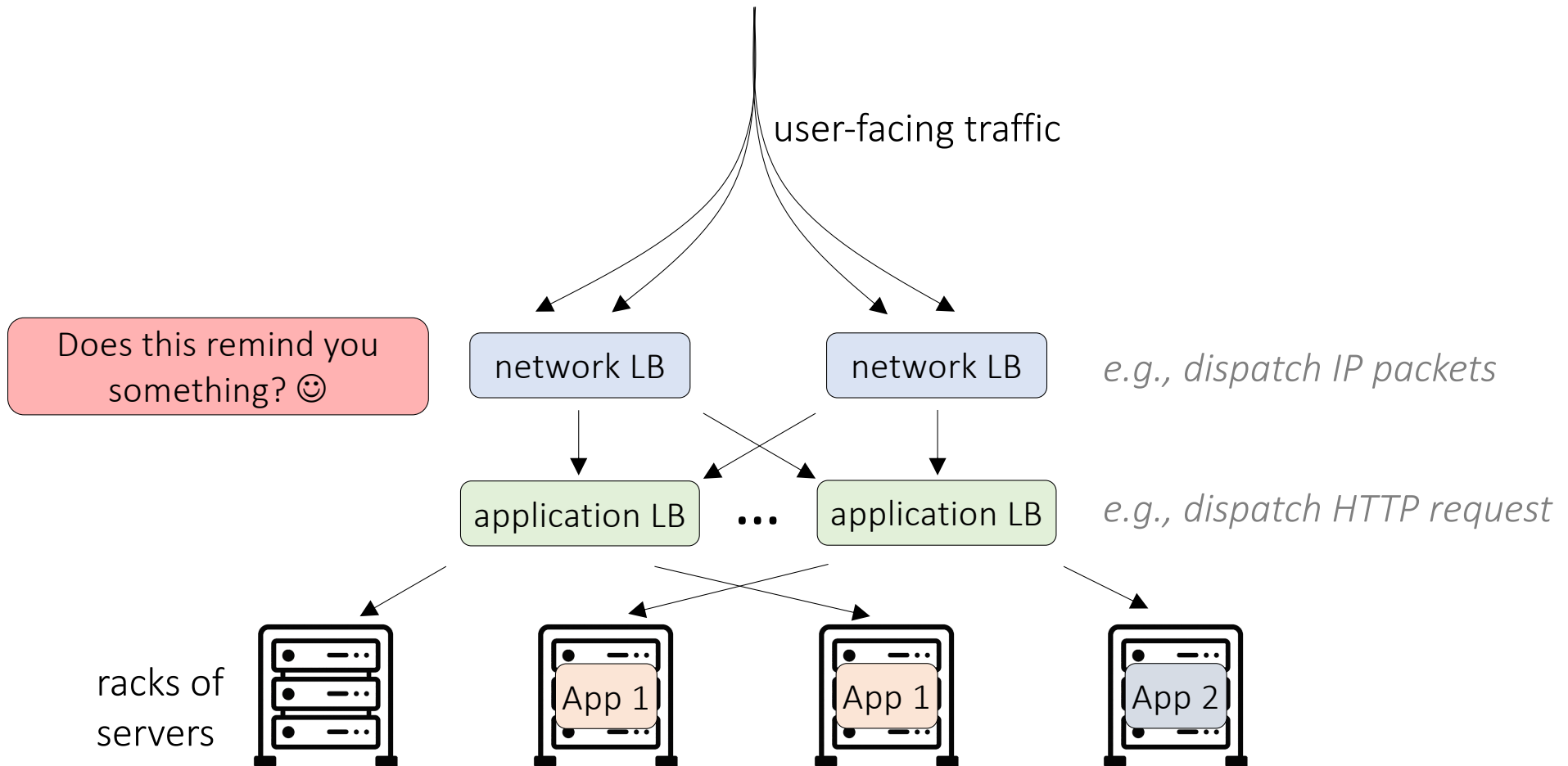
Traditional cloud load balancing architecture



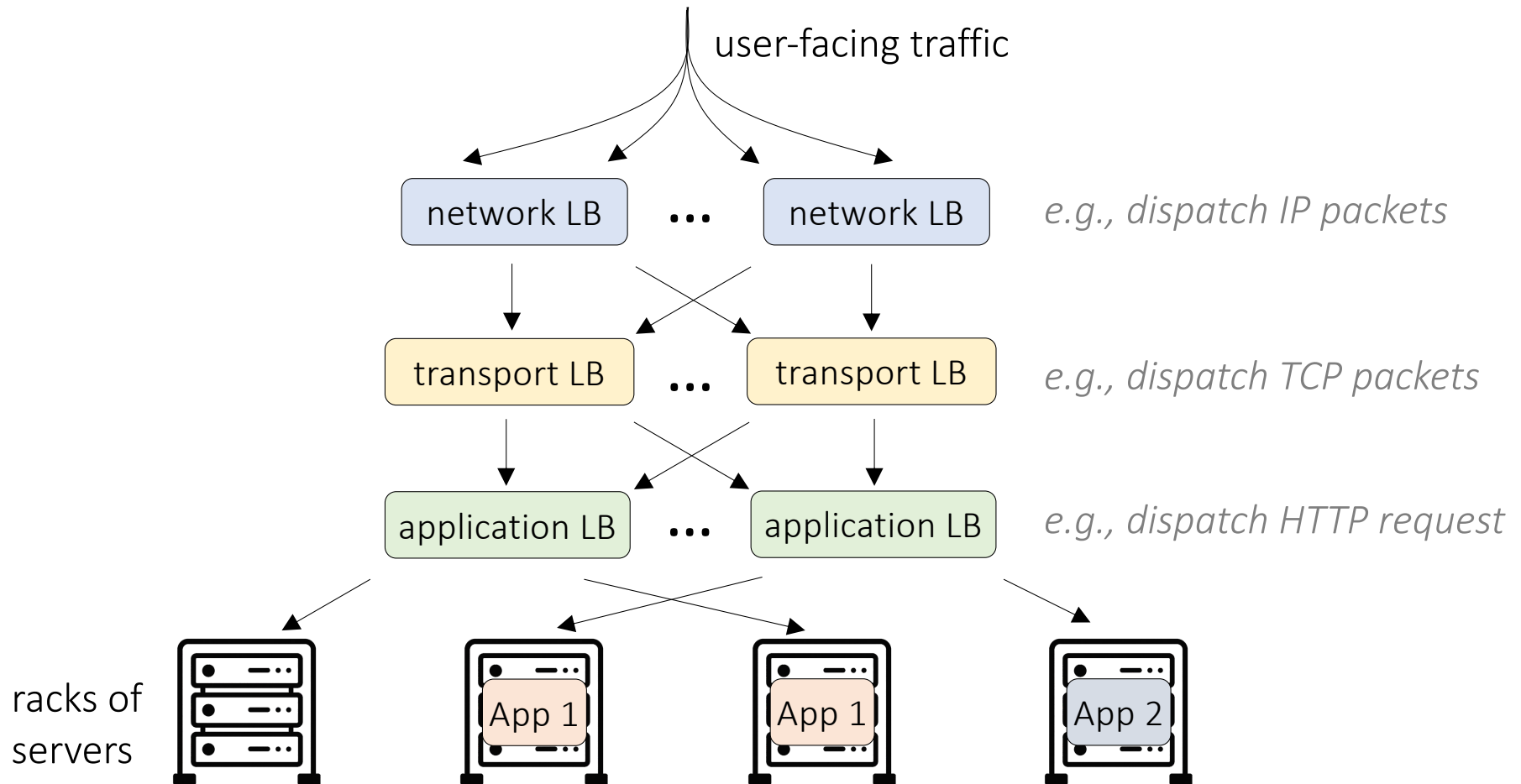
Traditional cloud load balancing architecture



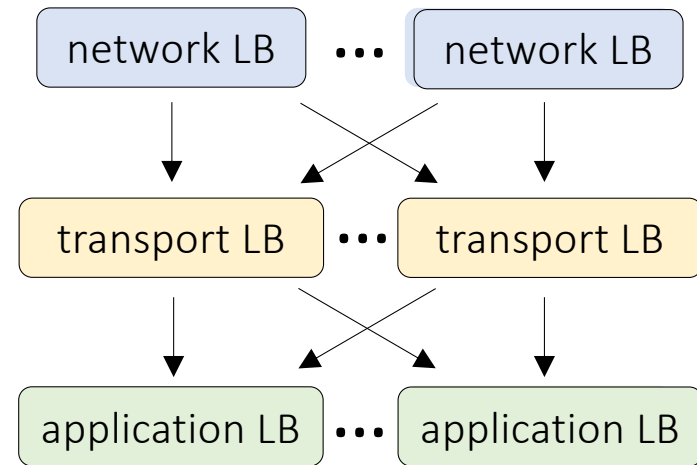
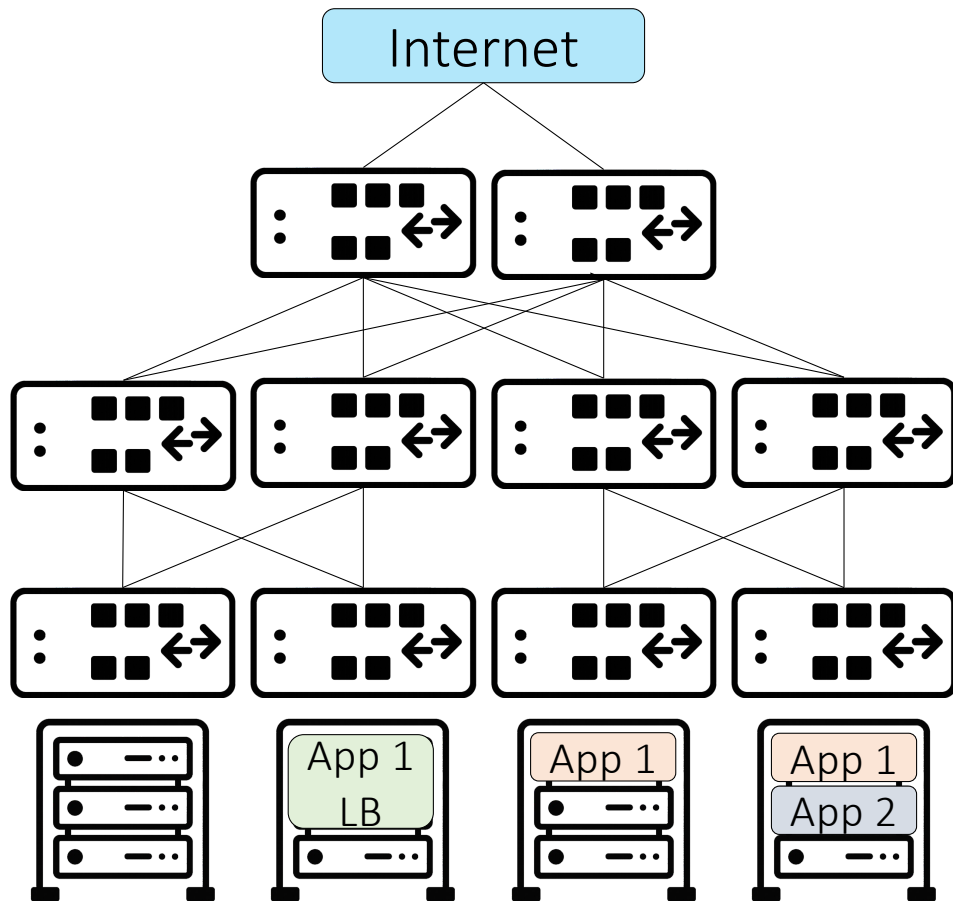
Traditional cloud load balancing architecture



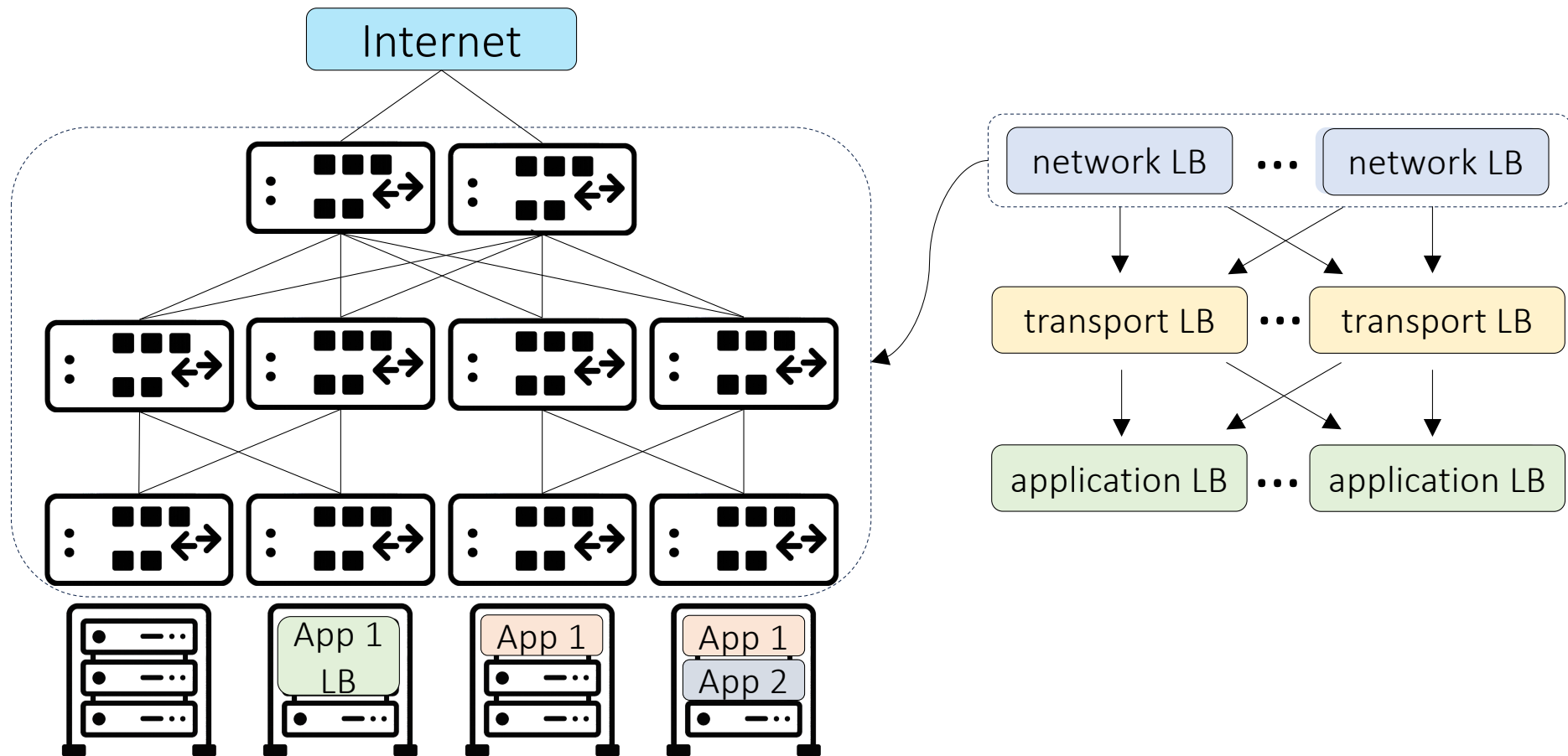
Traditional cloud load balancing architecture



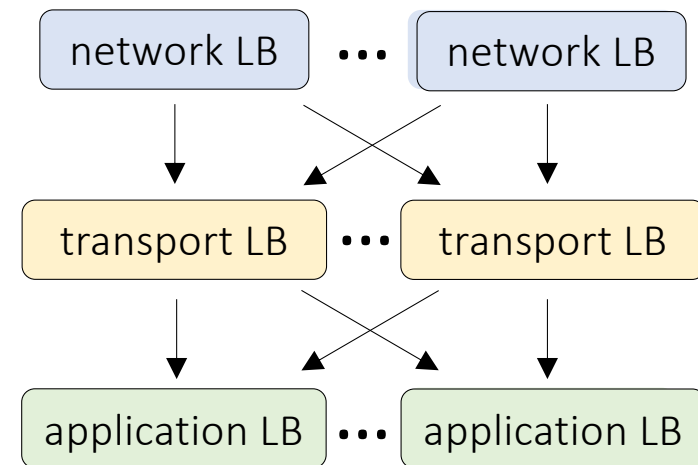
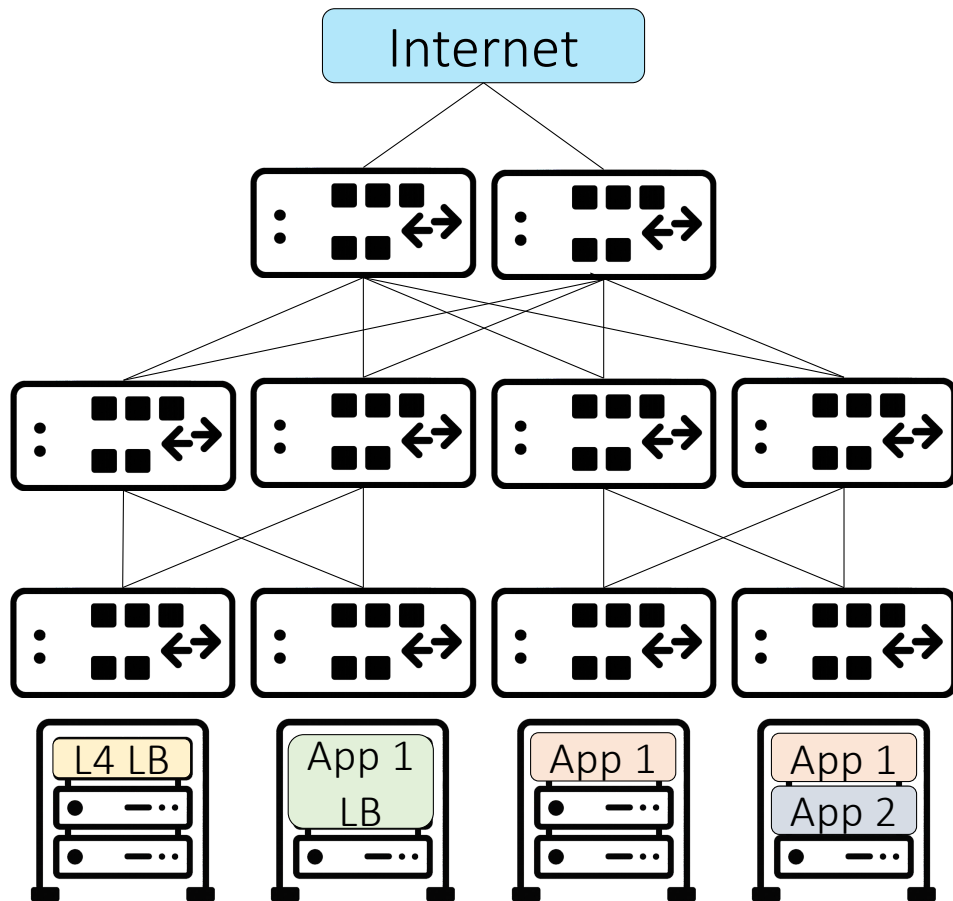
Traditional cloud load balancing architecture



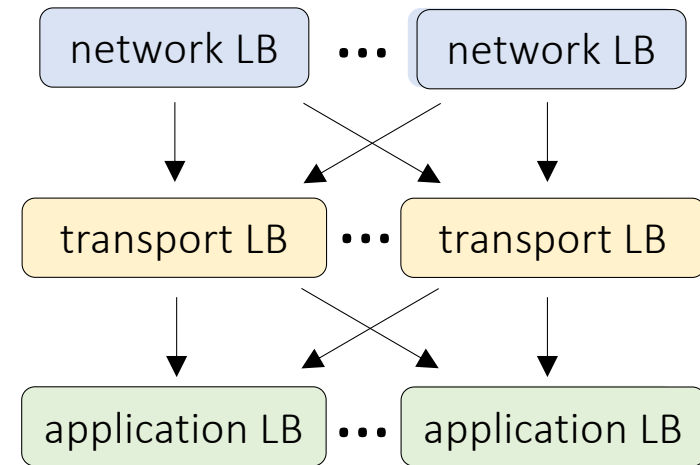
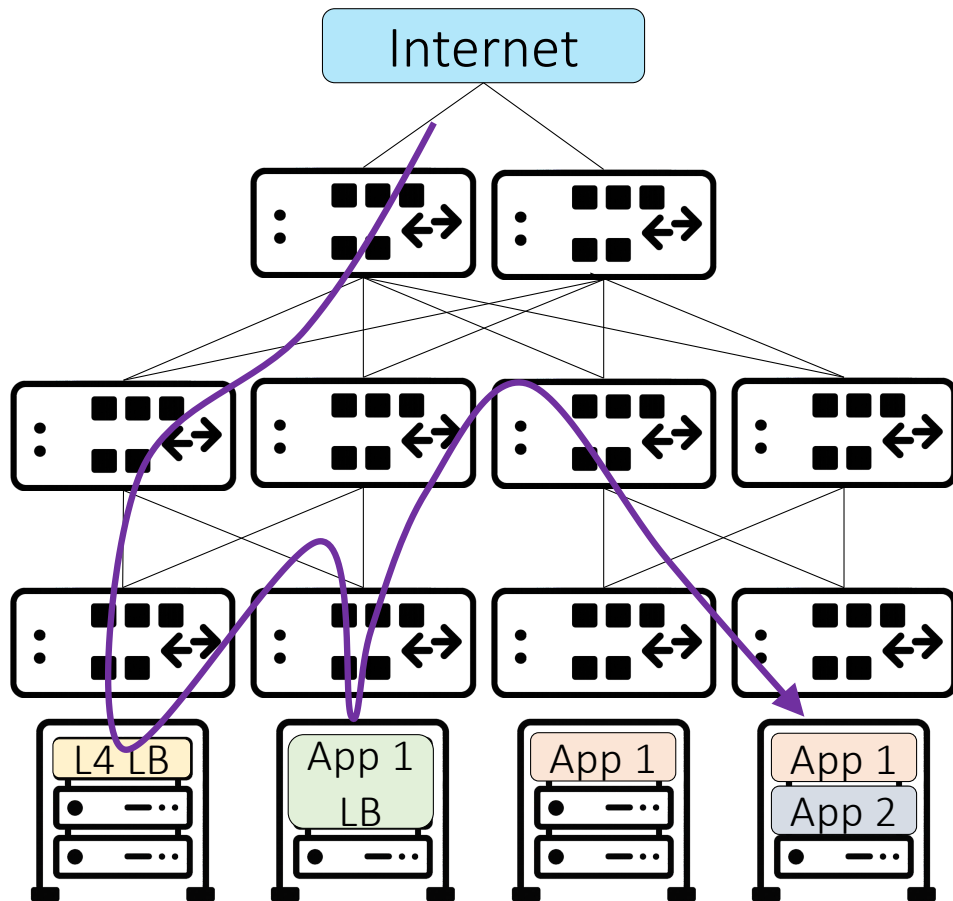
Traditional cloud load balancing architecture



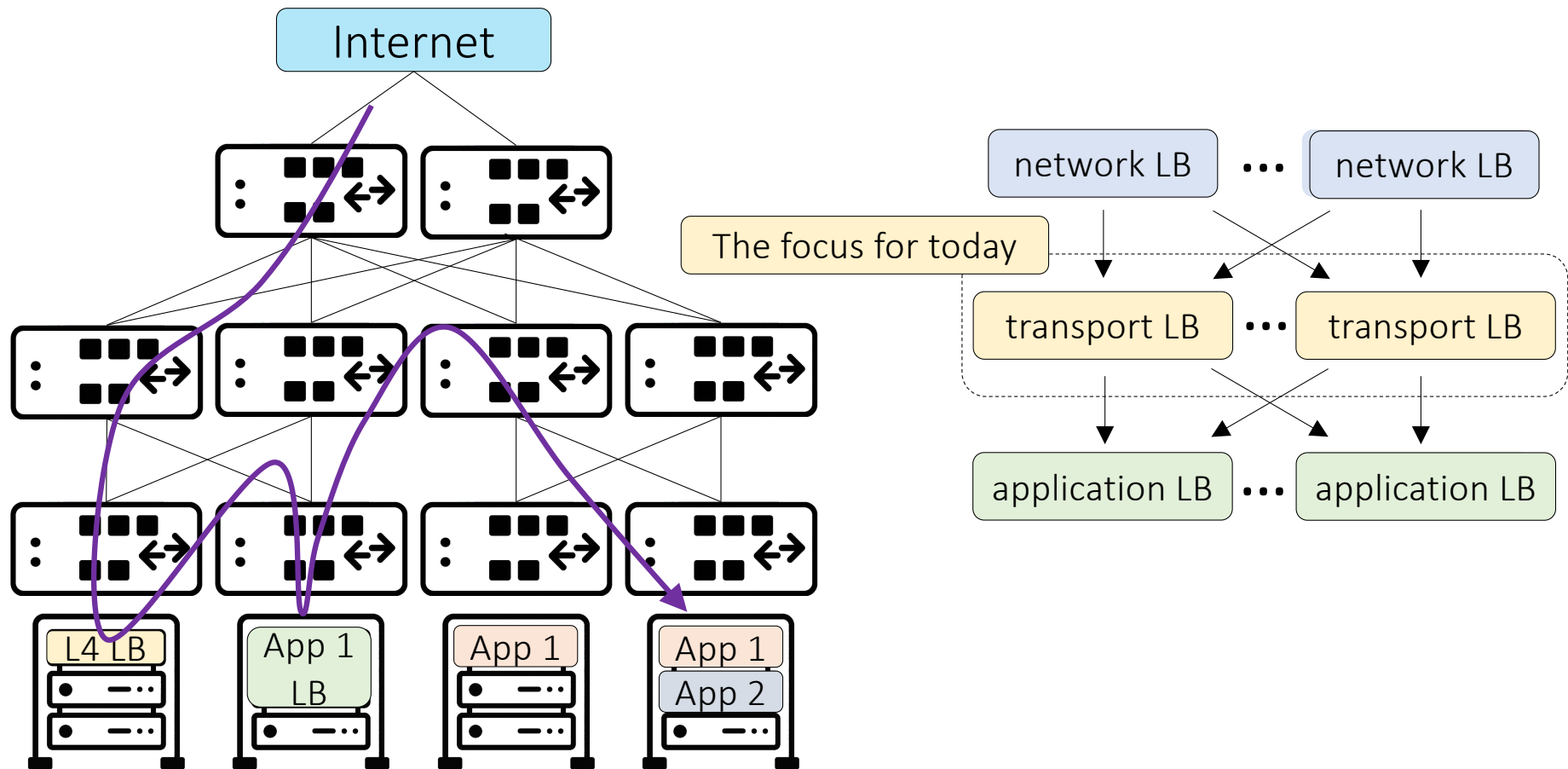
Traditional cloud load balancing architecture



Traditional cloud load balancing architecture



Traditional cloud load balancing architecture



Do you remember this?

Packet-Level Telemetry in Large Datacenter Networks

Yibo Zhu^{1,2} Nanxi Kang^{1,3} Jiaxin Cao¹ Albert Greenberg¹ Guohan Lu¹ Ratul Mahajan¹
Dave Maltz¹ Lihua Yuan¹ Ming Zhang¹ Ben Y. Zhao² Haitao Zheng²

¹Microsoft ²UC Santa Barbara ³Princeton University

ABSTRACT

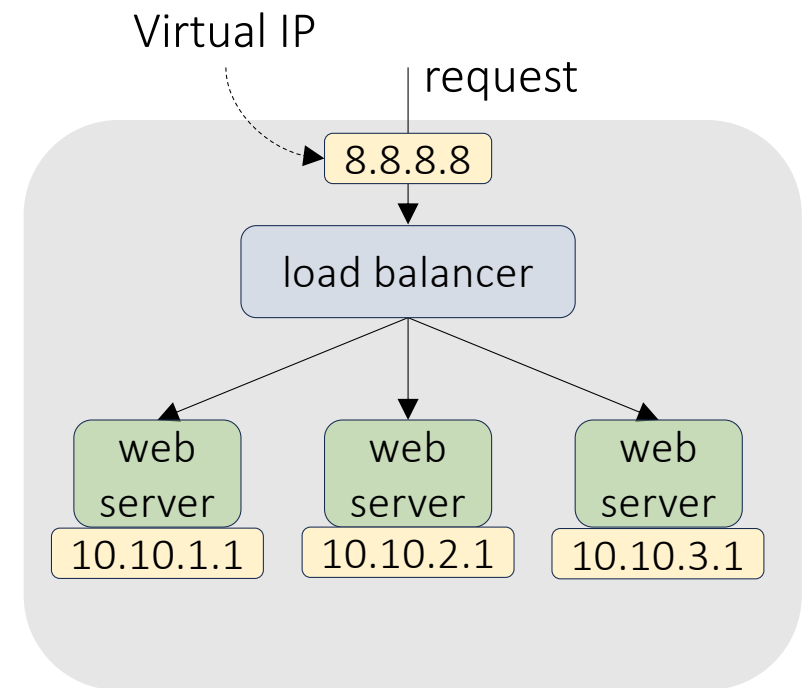
Debugging faults in complex networks often requires capturing and analyzing traffic at the packet level. In this task, datacenter networks (DCNs) present unique challenges with their scale, traffic volume, and diversity of faults. To troubleshoot faults in a timely manner, DCN administrators must *a)* identify affected packets inside large volume of traffic; *b)* track them across multiple network components; *c)* analyze traffic traces for fault patterns; and *d)* test or confirm potential causes. To our knowledge, no tool today can achieve both the specificity and scale required for this task.

We present Everflow, a packet-level network telemetry system for large DCNs. Everflow traces specific packets by implementing a powerful packet filter on top of “match and mirror” functionality of commodity switches. It shuffles captured packets to multiple analysis servers using load balancers built on switch ASICs, and it sends “guided probes” to test or confirm potential faults. We present experiments

proactive model of DCN management, where infrastructure observes, analyzes, and corrects faults in near-real time.

Understanding and debugging faults in DCNs is challenging because faults come in all shapes and sizes. For example:

1. Some packets may experience abnormally high delay between servers *A* and *B*, but it may not be clear which of the many links is responsible.
2. Packets destined for a specific set of servers may be dropped, even when the packet-drop counters at switches exhibit no abnormality.
3. TCP connections to a VIP (Virtual IP) may encounter intermittent timeouts, and traceroute probes to debug the issue may be blocked by the load balancers.
4. Load may not be balanced among a group of ECMP (Equal Cost Multi Path) links; the network administrators do not know if this issue is due to flow size differences or a deeper problem (e.g., a poor hash function).



L4 Load Balancers in the wild

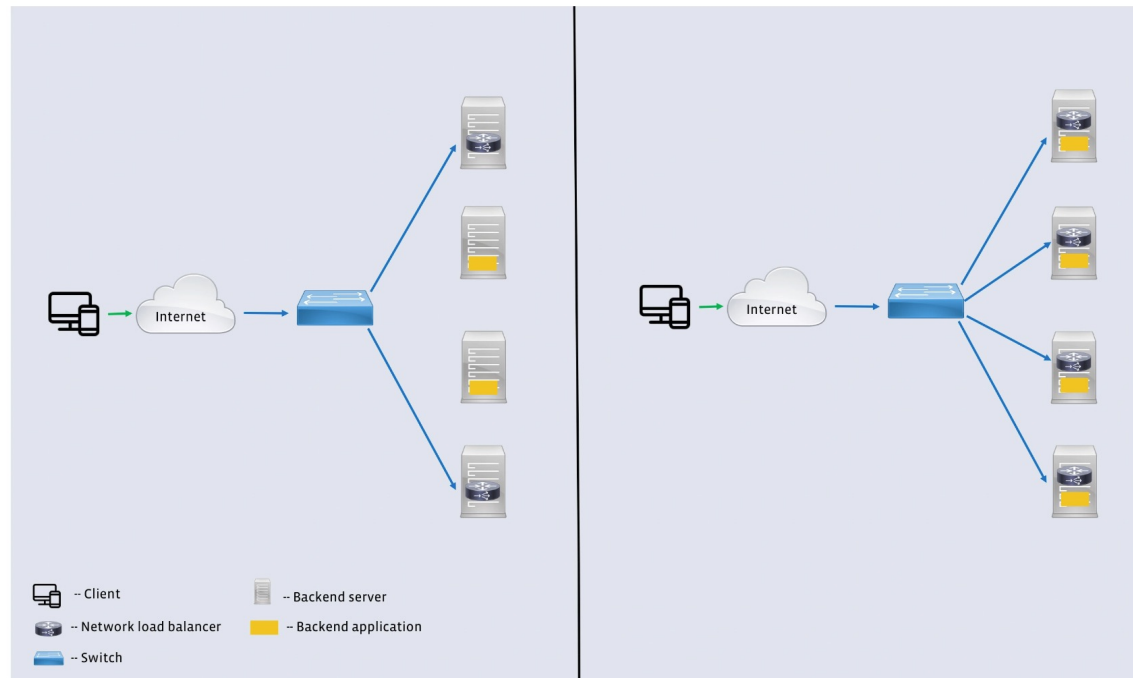
Engineering at Meta

Open Source ▾ Platforms ▾ Infrastructure Systems ▾ Physical Infrastructure ▾ Video Engineering & AR/VR ▾

Meta

POSTED ON MAY 22, 2018 TO [OPEN SOURCE](#)

Open-sourcing Katran, a scalable network load balancer



L4 Load Balancers in the wild

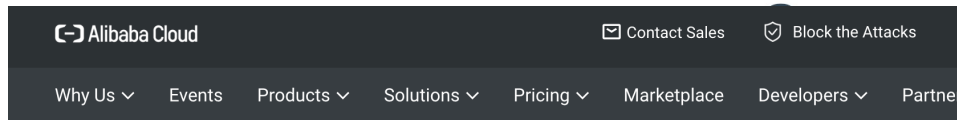
Engineering at Meta

Open Source ▾ Platforms ▾ Infrastructure Systems ▾ Physical Infrastructure ▾ Video Engineering & AR/VR ▾

Meta

POSTED ON MAY 22, 2018 TO [OPEN SOURCE](#)

Sourcing Katran, a scalable network load balancer



Alibaba Cloud > Products > Networking & CDN > Server Load Balancer

Server Load Balancer [Play Video](#)

Server Load Balancer (SLB) distributes network traffic across groups of backend servers to improve the service capability and application availability. It Includes Layer 4 Network Load Balancer (NLB), Layer 7 Application Load Balancer (ALB), and Classic Load Balancer (CLB). It Is the Official Cloud-Native Gateway of Alibaba Cloud.

Create an ALB

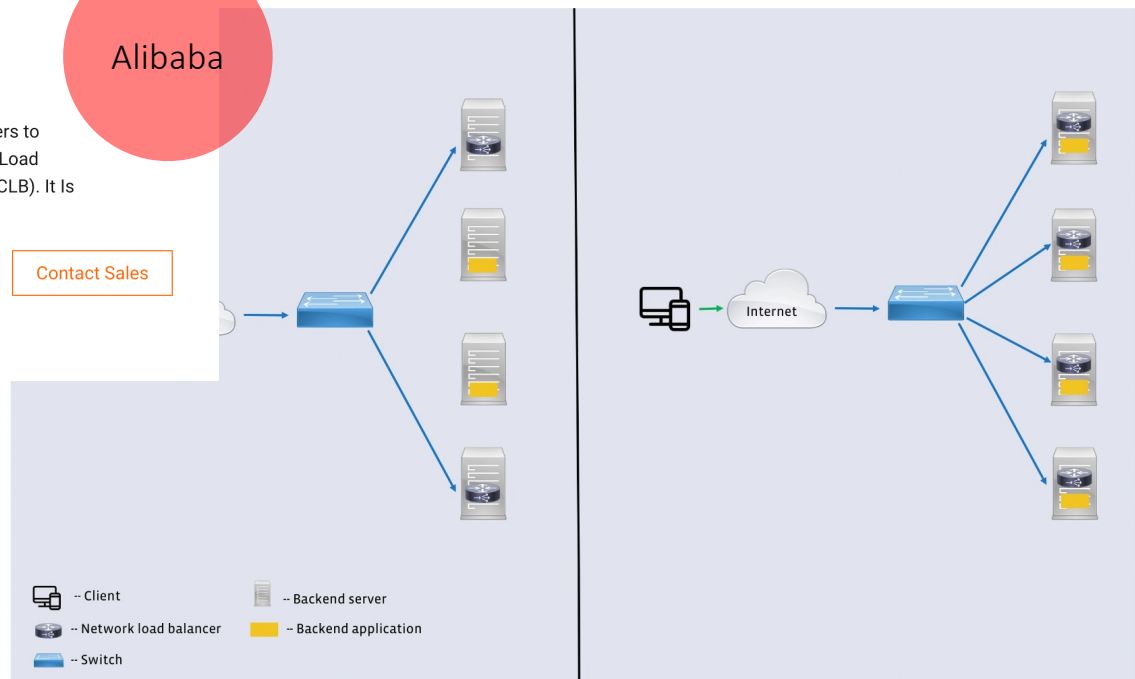
Create a NLB

Create a CLB

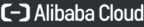
ALB Service Plan

Contact Sales

Alibaba



L4 Load Balancers in the wild



Why Us ▾ Events Products ▾ Solutions ▾ Pricing ▾

Alibaba Cloud > Products > Networking & CDN > Server Load Balancer

Server Load Balancer

Server Load Balancer (SLB) distributes network traffic across groups of servers to improve the service capability and application availability. It includes Classic Load Balancer (CLB), Network Load Balancer (NLB), Layer 7 Application Load Balancer (ALB), and Cloud Native Gateway (CNGW), the Official Cloud-Native Gateway of Alibaba Cloud.

[Create an ALB](#) [Create a NLB](#) [Create a CLB](#)



Learn Documentation Training Certifications Q&A Code Samples Assessments Shows Events

Azure Product documentation ▾ Architecture ▾ Learn Azure ▾ Develop ▾ Resources ▾

Filter by title

Load Balancer documentation

Overview

What is Azure Load Balancer?

> Quickstarts

> Tutorials

> Samples

> Concepts

> How-to

> Reference

> Resources

Learn / Azure / Networking / Load Balancer /

What is Azure Load Balancer?

Article • 01/06/2023 • 3 minutes to read • 32 contributors

 Feedback

In this article

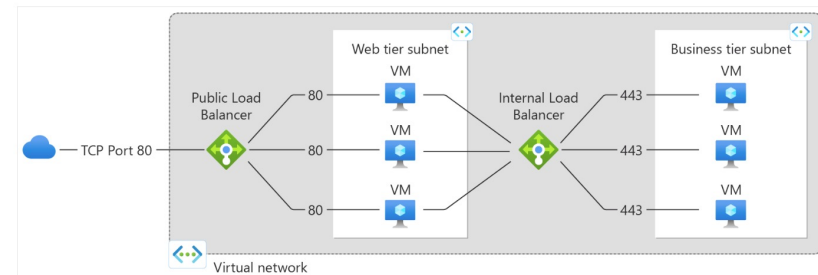
[Why use Azure Load Balancer?](#)[Pricing and SLA](#)[What's new?](#)[Next steps](#)

Load balancing refers to evenly distributing load (incoming network traffic) across a group of backend resources or servers.

Azure Load Balancer operates at layer 4 of the Open Systems Interconnection (OSI) model. It's the single point of contact for clients. Load balancer distributes inbound flows that arrive at the load balancer's front end to backend pool instances. These flows are according to configured load-balancing rules and health probes. The backend pool instances can be Azure Virtual Machines or instances in a Virtual Machine Scale Set.

A **public load balancer** can provide outbound connections for virtual machines (VMs) inside your virtual network. These connections are accomplished by translating their private IP addresses to public IP addresses. Public Load Balancers are used to load balance internet traffic to your VMs.

An **internal (or private) load balancer** is used where private IPs are needed at the frontend only. Internal load balancers are used to load balance traffic inside a virtual network. A load balancer frontend can be accessed from an on-premises network in a hybrid scenario.

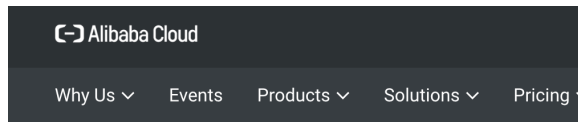


& AR/VR ▾

Meta

Microsoft

L4 Load Balancers in the wild



Alibaba Cloud > Products > Networking & CDN > Server Load Balance

Server Load Balancer

Server Load Balancer improves the availability and performance of your applications by distributing incoming traffic across multiple servers. The Official Google Research Publications

Create

PUBLICATIONS

Maglev: A Fast and Reliable Software Network Load Balancer

Daniel E. Eisenbud, [Cheng Yi](#), Carlo Contavalli, Cody Smith, Roman Kononov, Eric Mann-Hielscher, Ardas Ciltingiroglu, Bin Cheyney, Wentao Shang, Jinnah Dylan Hosein
13th USENIX Symposium on Networked Systems Design and Implementation (NSDI 16), USENIX Association, Santa Clara, CA (2016), pp. 523-535

Download

Google Scholar

Copy Bibtext

Abstract

Maglev is Google's network load balancer. It is a large distributed software system that runs on commodity Linux servers. Unlike traditional hardware network load balancers, it does not require a specialized physical rack deployment, and its capacity can be easily adjusted by adding or removing servers. Network routers distribute packets evenly to the Maglev machines via Equal Cost Multipath (ECMP); each Maglev machine then matches the packets to their corresponding services and spreads them evenly to the service endpoints. To accommodate high and ever-increasing traffic, Maglev is specifically optimized for packet processing performance. A single Maglev machine is able to saturate a 10Gbps link with small packets. Maglev is also equipped with consistent hashing and connection tracking features, to minimize the negative impact of unexpected faults and failures on connection-oriented protocols. Maglev has been serving Google's traffic since 2008. It has sustained the rapid global growth of Google services, and it also provides network load balancing for Google Cloud Platform.

What is Azure Load Balancer?

Article • 01/06/2023 • 3 minutes to read • 32 contributors

Feedback

In this article

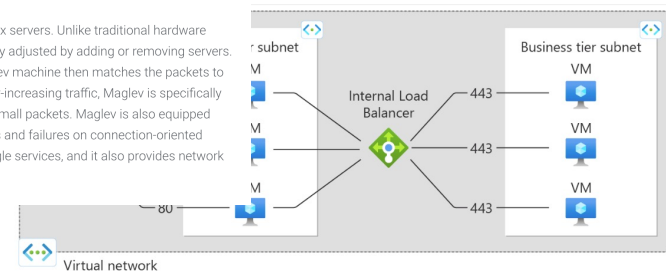
[Why use Azure Load Balancer?](#)
[Pricing and SLA](#)
[What's new?](#)
[Next steps](#)

Load balancing refers to evenly distributing load (incoming network traffic) across a group of backend resources or servers.

ms Interconnection (OSI) model. It's the single point of contact drive at the load balancer's front end to backend pool instances. les and health probes. The backend pool instances can be cale Set.

s for virtual machines (VMs) inside your virtual network. These IP addresses to public IP addresses. Public Load Balancers are

te IPs are needed at the frontend only. Internal load balancers load balancer frontend can be accessed from an on-premises

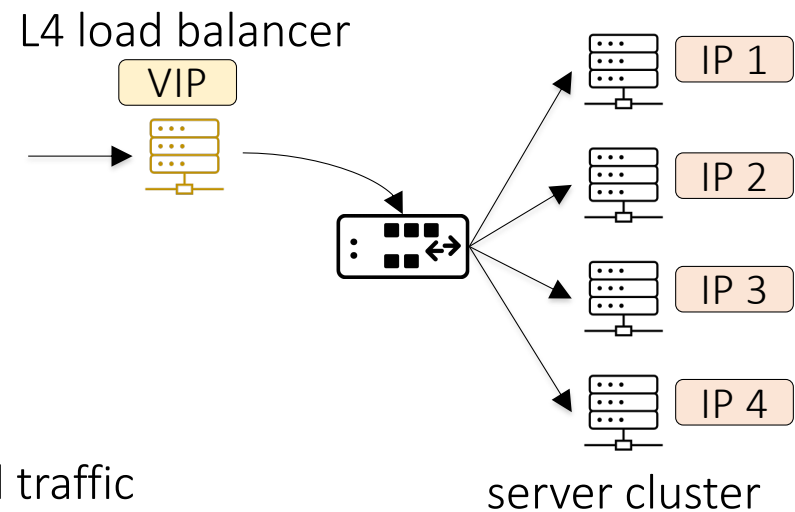


AR/VR
Meta

Microsoft

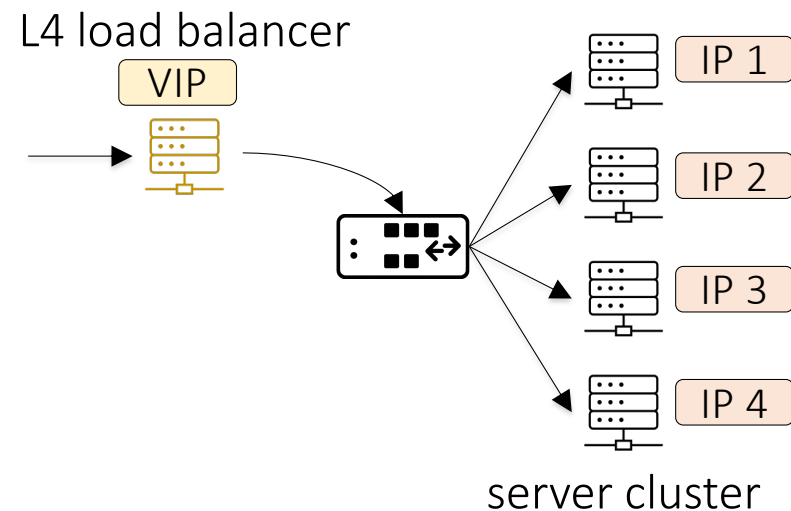
L4 load balancing

- Goal: split requests over available servers
- Three possible approaches to interact with external traffic
 - Load balancer terminates TCP and opens own connection to servers
 - NAT approach: load balancer replace service VIP with server's actual IP
 - DSR (Direct Server Return)

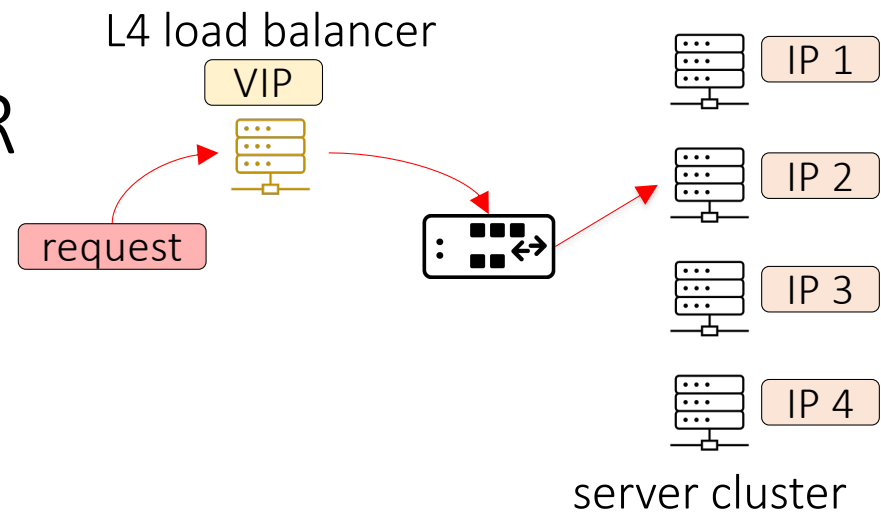


L4 load balancing with DSR

- Servers bind to both VIP and dedicated IP
- Load balancer replaces destination MAC
- Server sees client IP and respond directly
 - Packets in reverse direction do not pass through the load balancer
 - Greater scalability, particularly for traffic with asymmetric bandwidth needs (e.g., HTTP get)

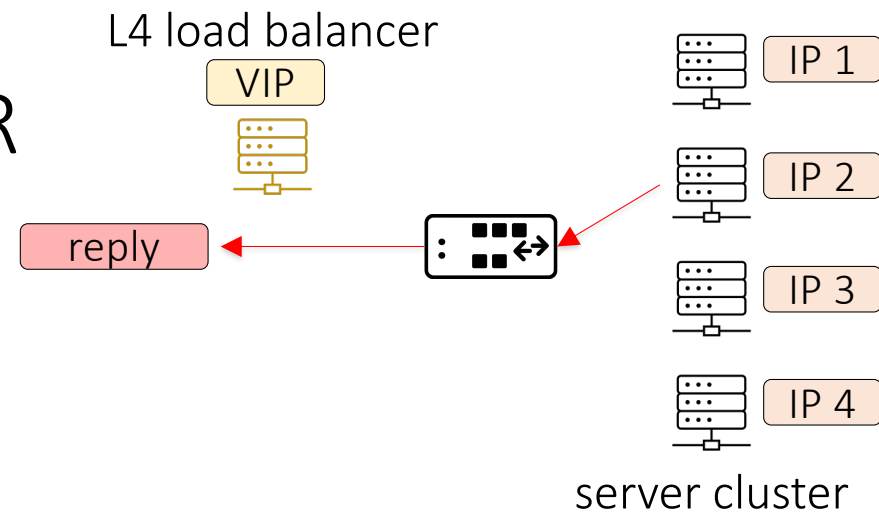


L4 load balancing with DSR



- Servers bind to both VIP and dedicated IP
- Load balancer replaces destination MAC
- Server sees client IP and respond directly
 - Packets in reverse direction do not pass through the load balancer
 - Greater scalability, particularly for traffic with asymmetric bandwidth needs (e.g., HTTP get)

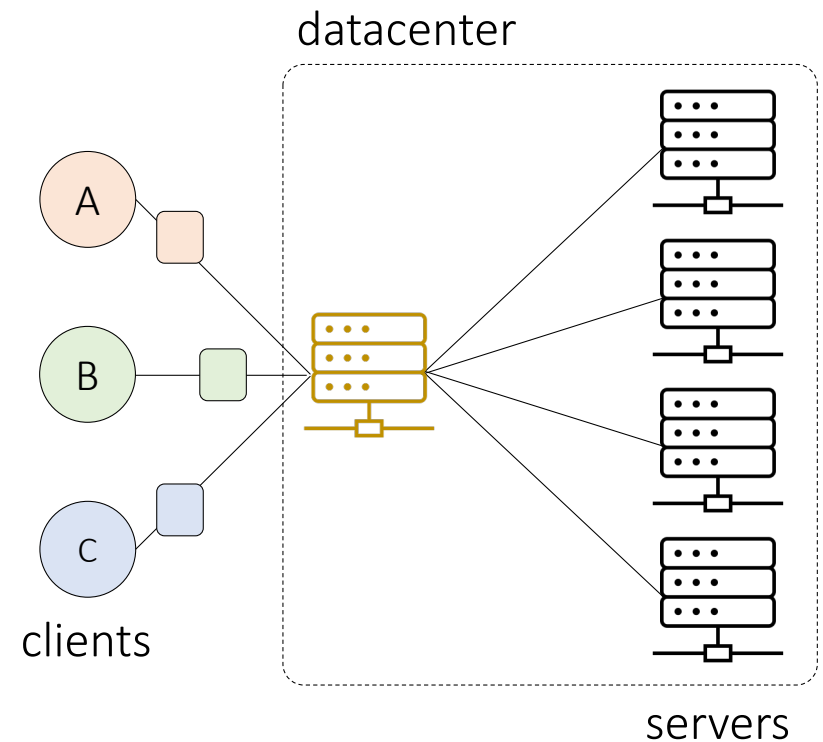
L4 load balancing with DSR



- Servers bind to both VIP and dedicated IP
- Load balancer replaces destination MAC
- Server sees client IP and respond directly
 - Packets in reverse direction do not pass through the load balancer
 - Greater scalability, particularly for traffic with asymmetric bandwidth needs (e.g., HTTP get)

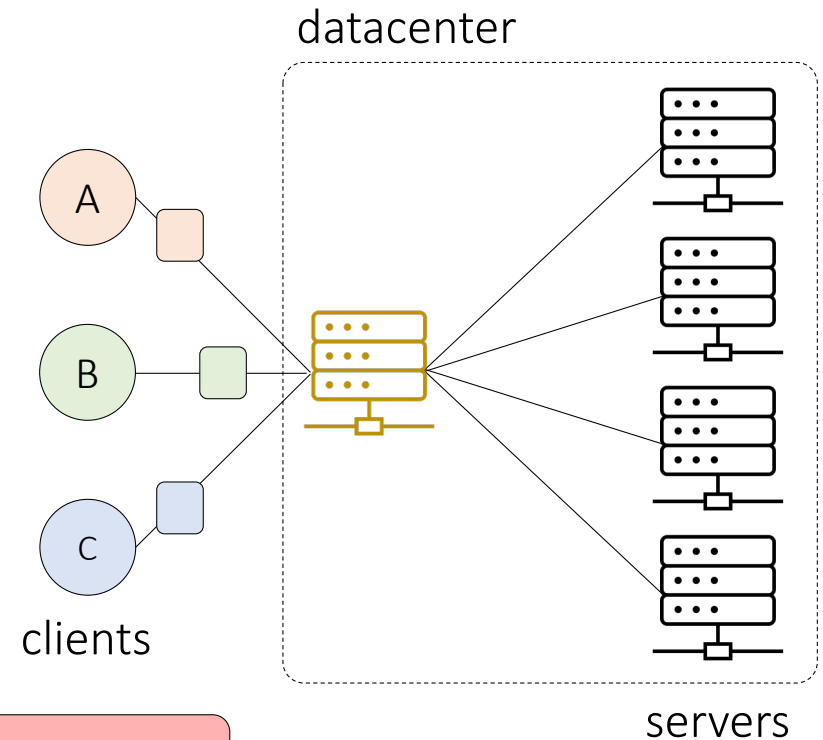
What a L4 load balancer does

- A device that *assigns* client requests to server machines
- Goal: to make *efficient* utilization of resources (compute, network, storage)



What a L4 load balancer does

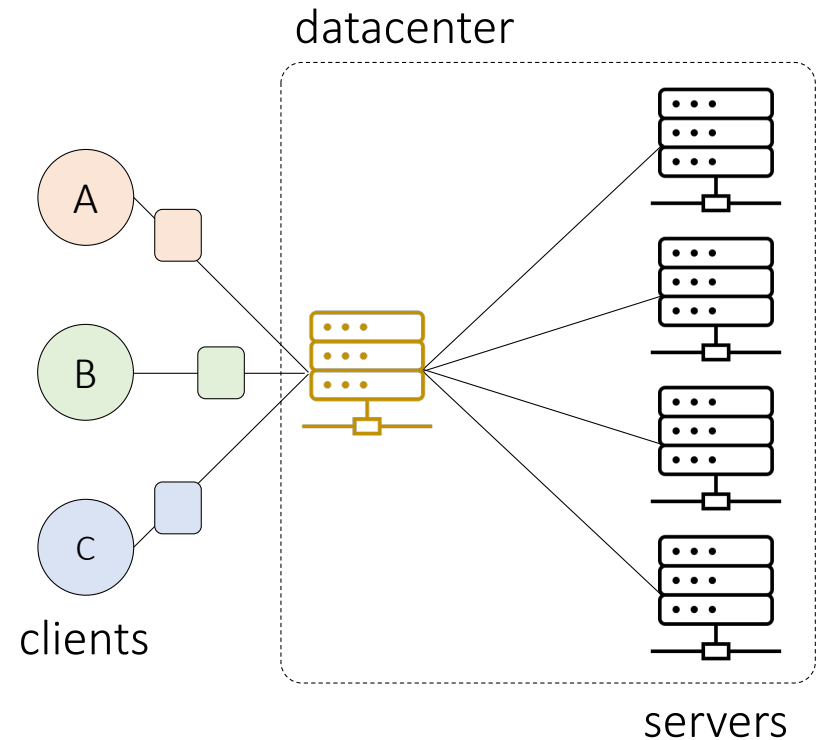
- A device that *assigns* client requests to server machines
- Goal: to make *efficient* utilization of resources (compute, network, storage)



What policy would you use to equally share resources at the servers?

What a L4 load balancer does

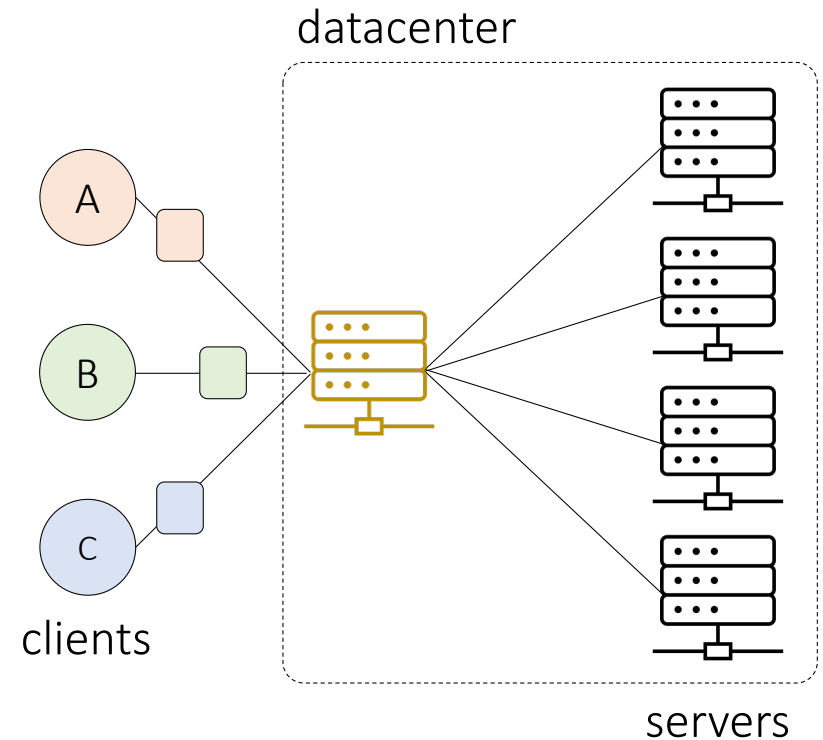
- State at the LB: a counter with size = # servers
- When a new packet arrives, assign it to the server indexed by the counter and increase the counter



What a L4 load balancer does

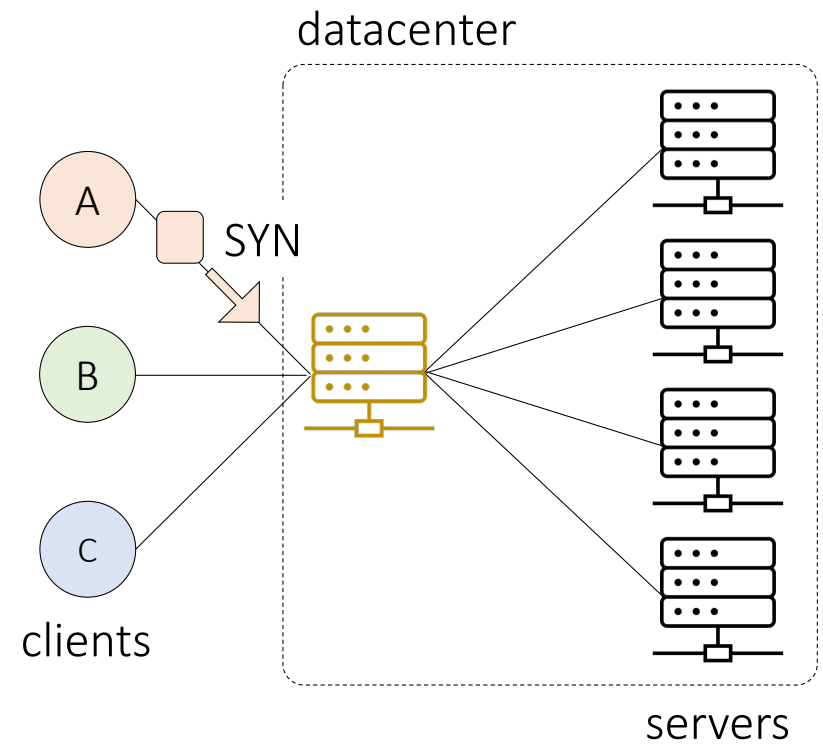
- State at the LB: a counter with size = # servers
- When a new packet arrives, assign it to the server indexed by the counter and increase the counter

Can you see a problem here?



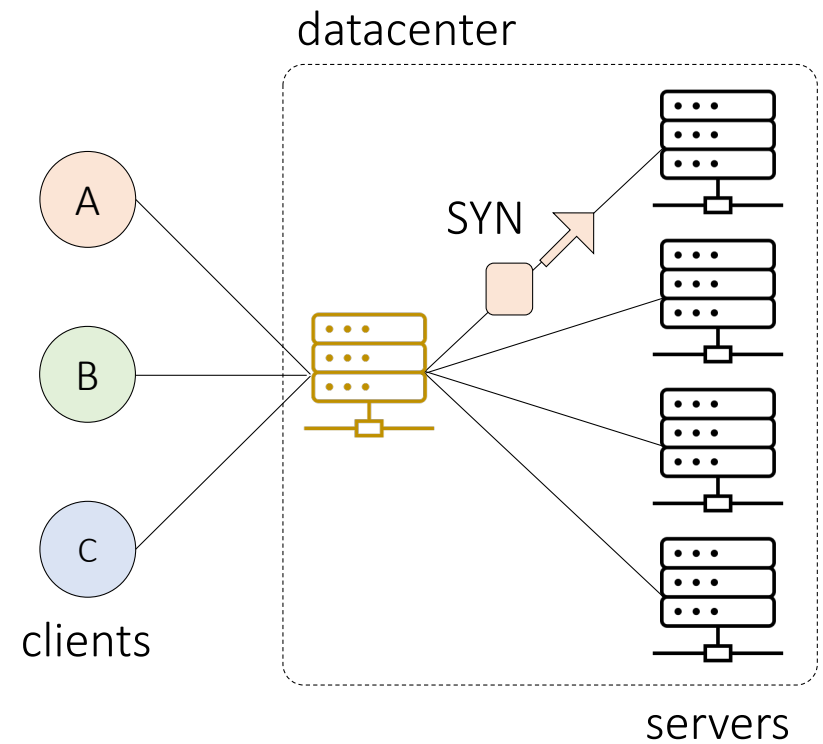
What a L4 load balancer does

- A thought example with TCP



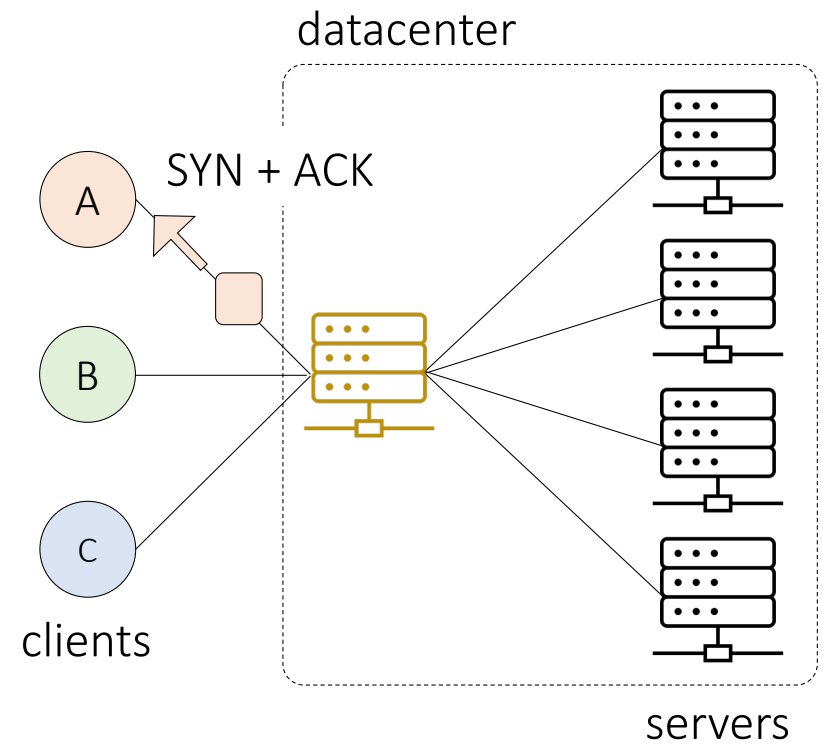
What a L4 load balancer does

- A thought example with TCP



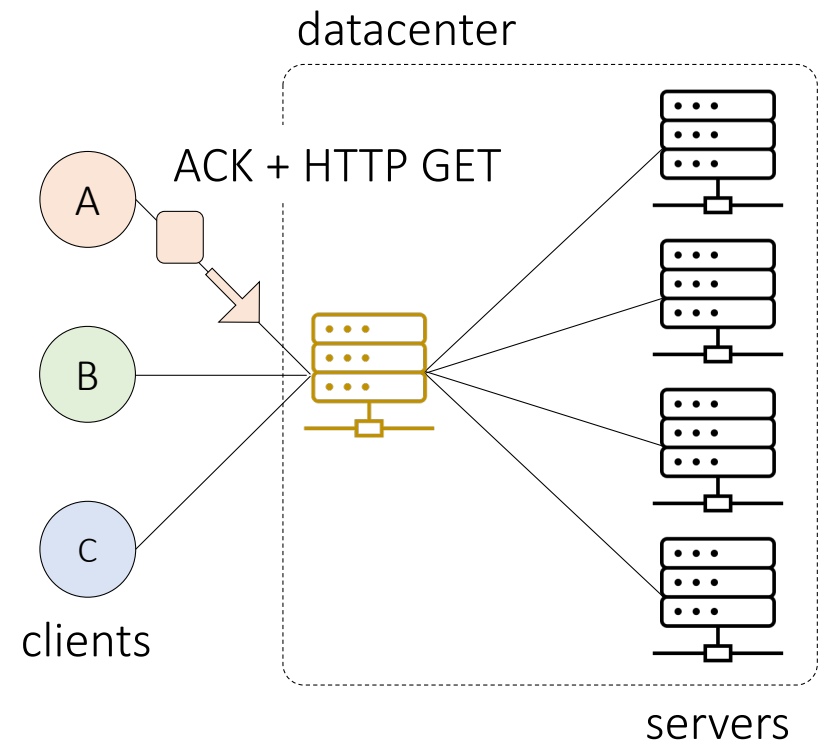
What a L4 load balancer does

- A thought example with TCP



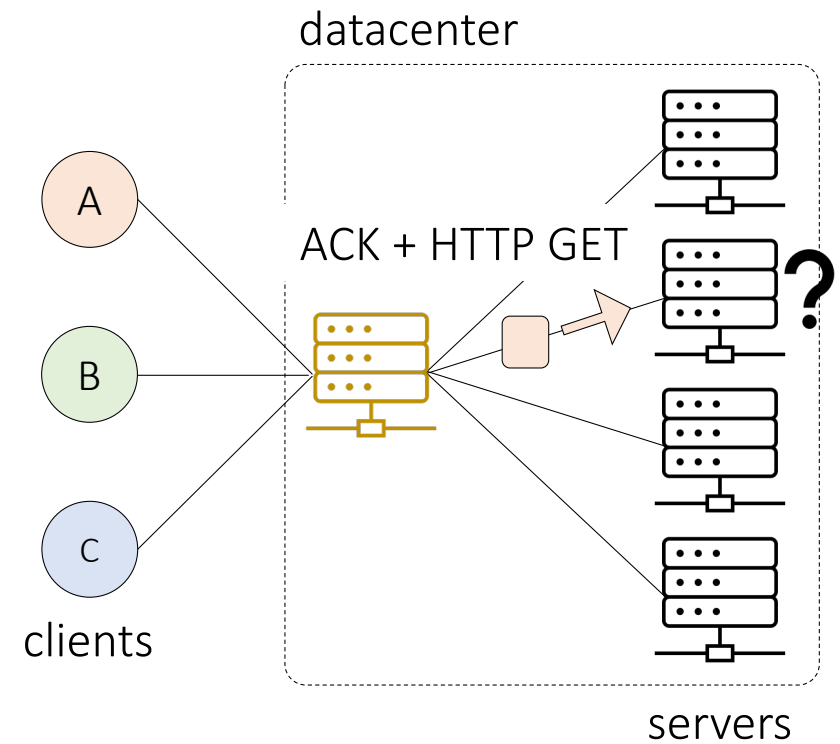
What a L4 load balancer does

- A thought example with TCP



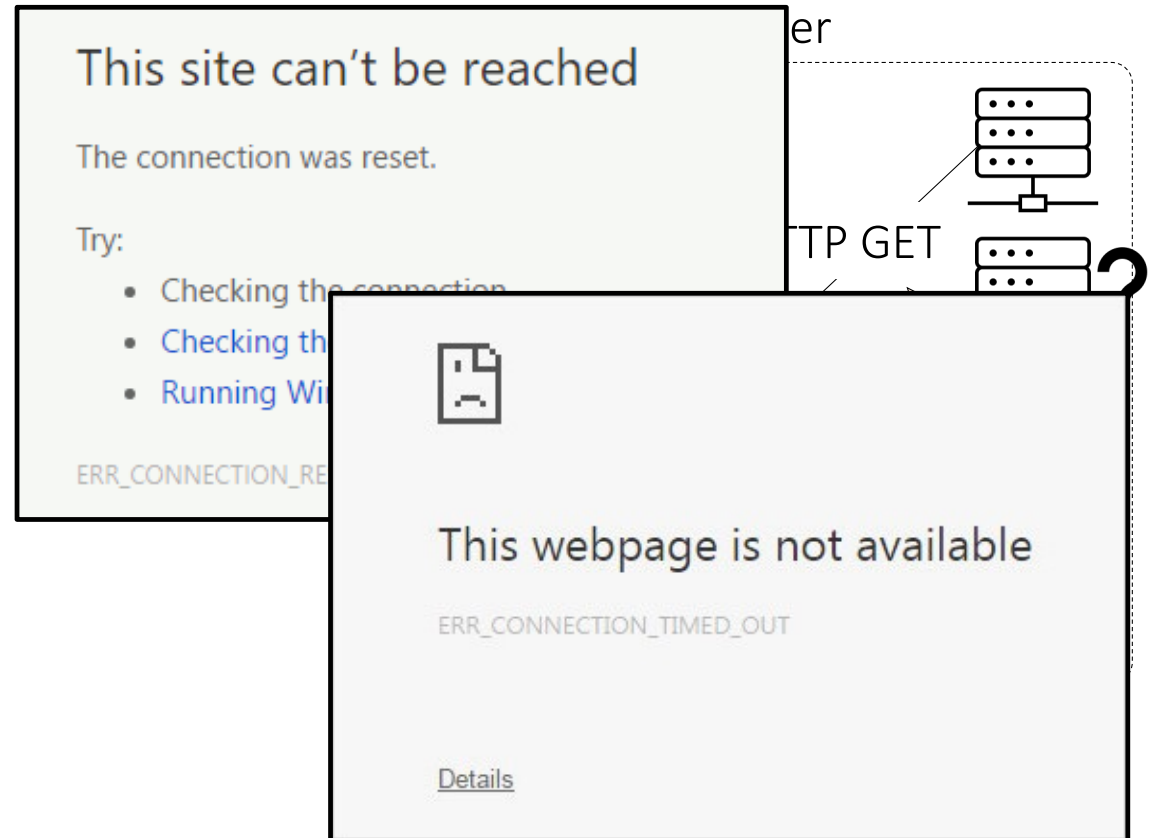
What a L4 load balancer does

- A thought example with TCP



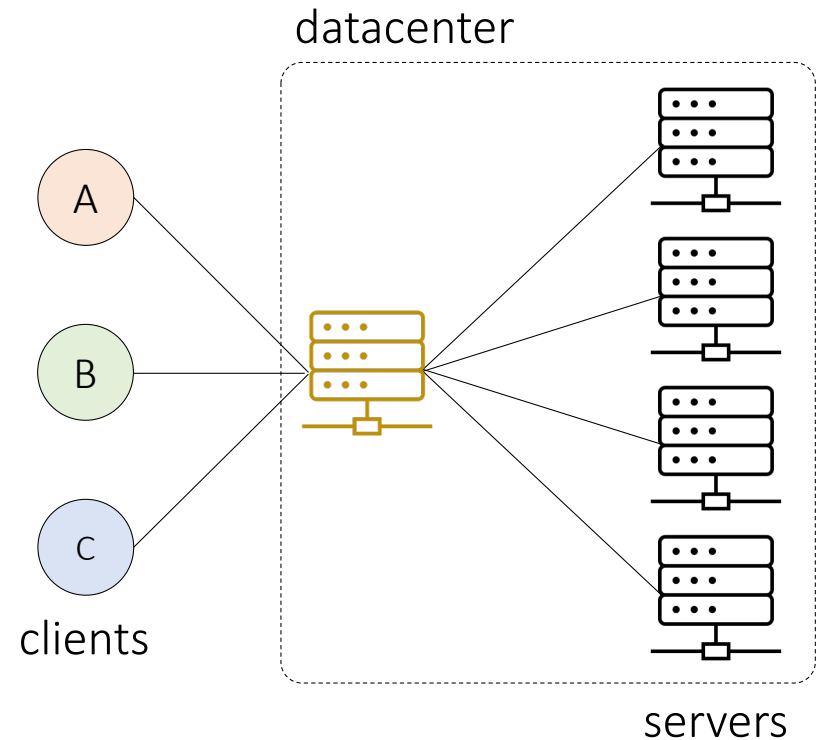
What a L4 load balancer does

- A thought example with TCP



What a L4 load balancer does

- Packets belonging to the same request must hit the same server instance
- Processing request time may vary
 - Some server may be slower
 - Some requests are more complex

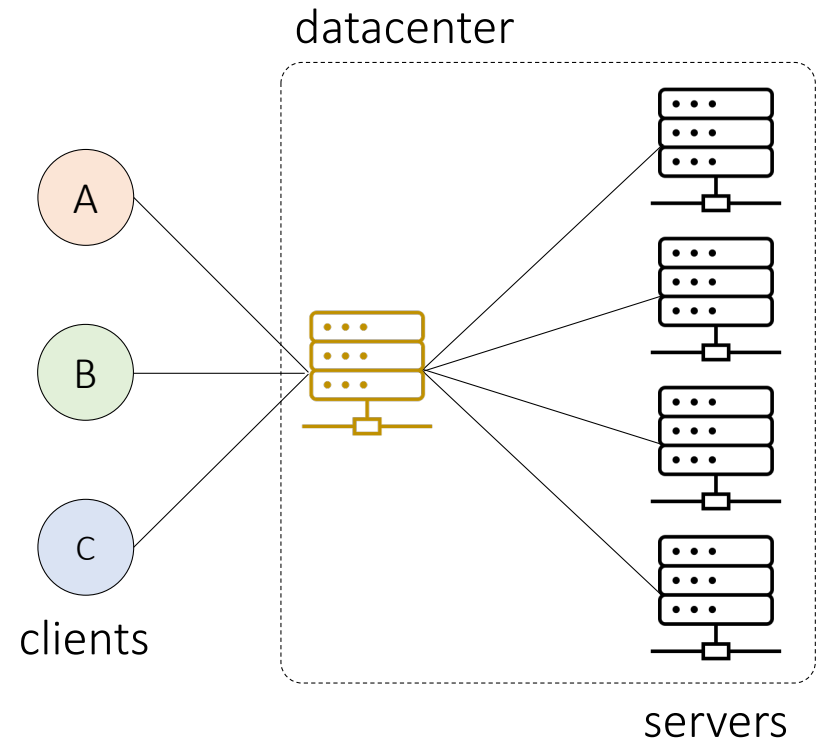


What a L4 load balancer does

- Packets belonging to the same request must hit the same server instance

our focus

- Processing request time may vary
 - Some server may be slower
 - Some requests are more complex



A notion of persistency for load balancers

- Packets belonging to the same flow must be forwarded to the same server
- In the Internet, TCP connections are used for sending requests
 - We must guarantee packets belonging to the same TCP connection hit the same server
 - ...even in the face of topological changes

We need a uniform deterministic LB function

- Uniform: we need to spread loads across servers
- Deterministic: we need to guarantee connection affinity

Ideas? 😊

We need a uniform deterministic LB function

Option 1

- Keep per-flow state at the load balancer
- TCP/IP tuple identifies a connection

TCP/IP header	Server id
ip_src=192.4.1.8, ip_dst=192.4.1.8, tcp_src=32487, tcp_dst=80	2
ip_src=212.3.5.4, ip_dst=192.4.1.8, tcp_src=758, tcp_dst=80	4

We need a uniform deterministic LB function

Option 1

- Keep per-flow state at the load balancer
- TCP/IP tuple identifies a connection

TCP/IP header	Server id
ip_src=192.4.1.8, ip_dst=192.4.1.8, tcp_src=32487, tcp_dst=80	2
ip_src=212.3.5.4, ip_dst=192.4.1.8, tcp_src=758, tcp_dst=80	4

- This is a stateful solution which might require too much memory

We need a uniform deterministic LB function

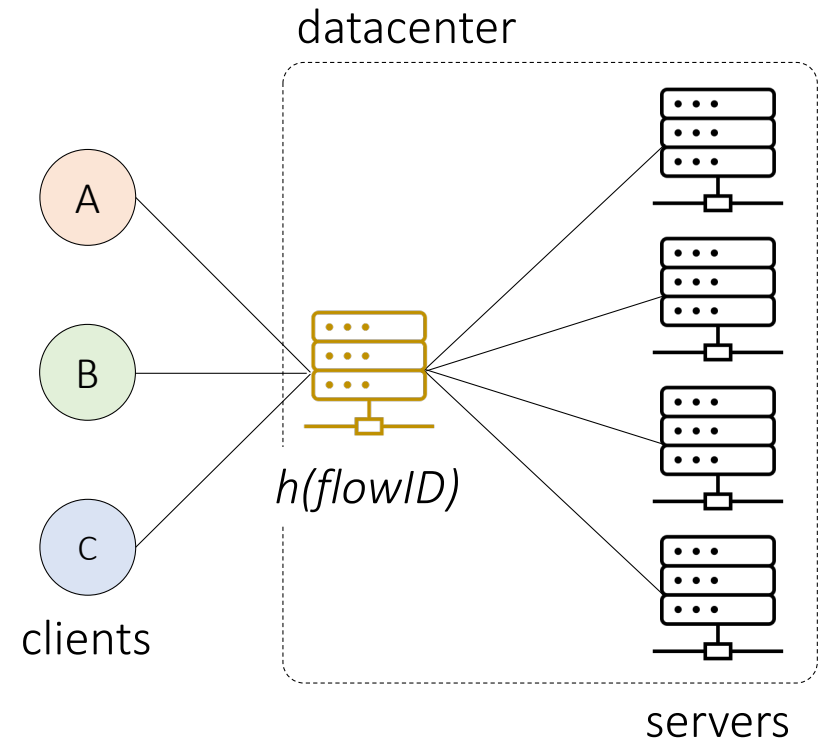
Option 2

- Apply a hash function h on the packet header:
 - Example: $h(ip_src/ip_dst) \bmod \#servers$
- This will allow to map data of arbitrary size (input flows) into data of fixed size (the set of servers)
- This requires zero state at the load balancer 😊
- Packets belonging to the same flow will hit the same server 😊
 - Note: requests from the same user might hit different servers!

A hash-based load balancer example

Packets belonging to the same request must hit the same server instance

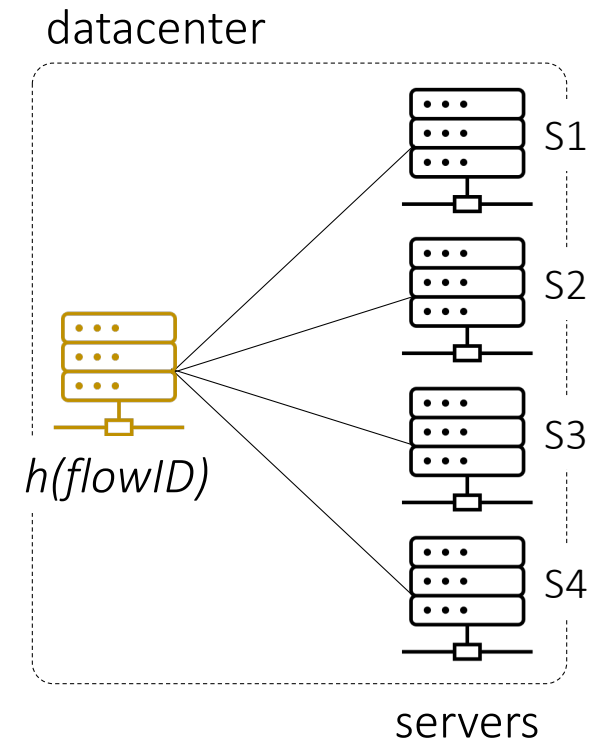
Job done?



A hash-based load balancer example

Hash of the header	Hash modulo 4
0	0
1	1
2	2
3	3
4	0
5	1
6	2
7	3
8	0
...	...

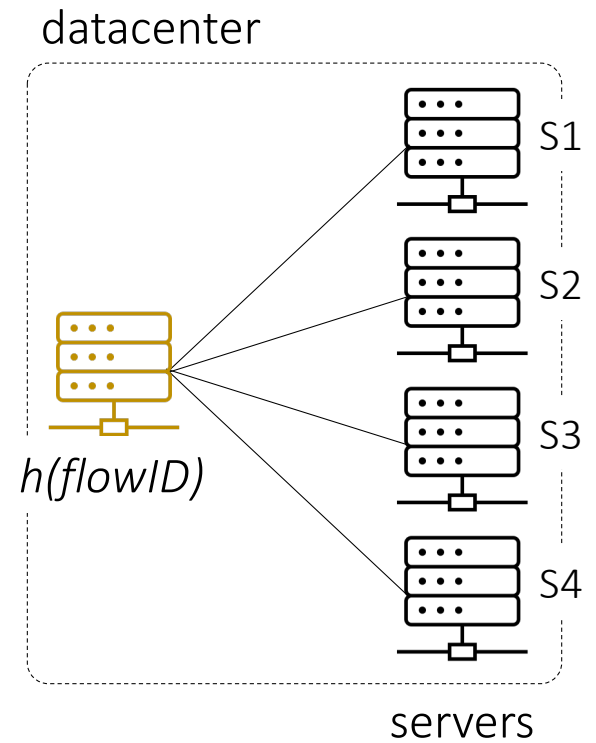
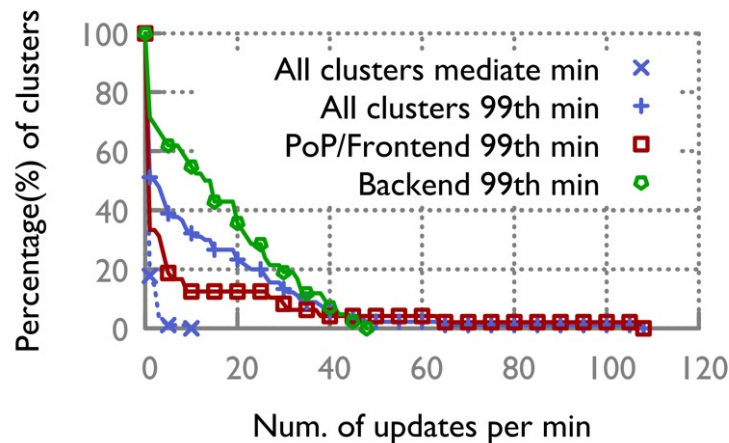
Modulo output	Server id
0	S1
1	S2
2	S3
3	S4



A hash-based load balancer example

What if there is an update in the server pool?

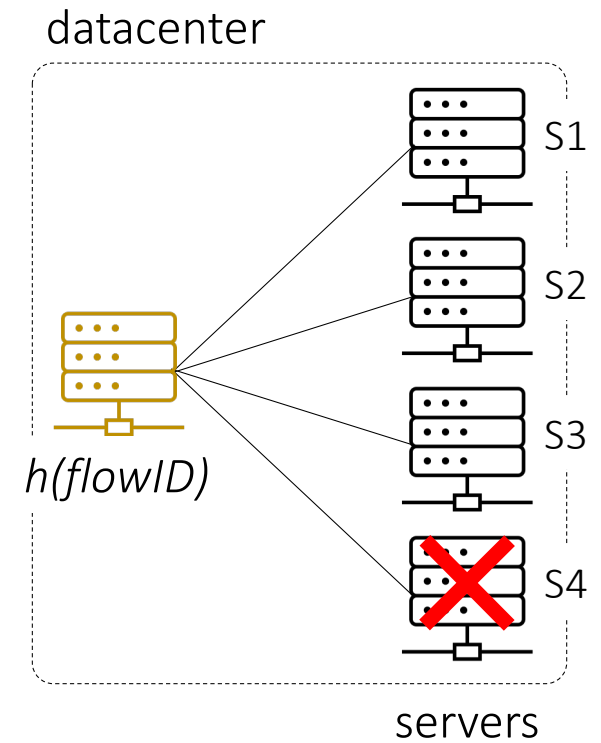
- This can be caused by a failure, service expansion or a server upgrade
- Up to 100 updates per minute in a Facebook/Meta cluster!!!



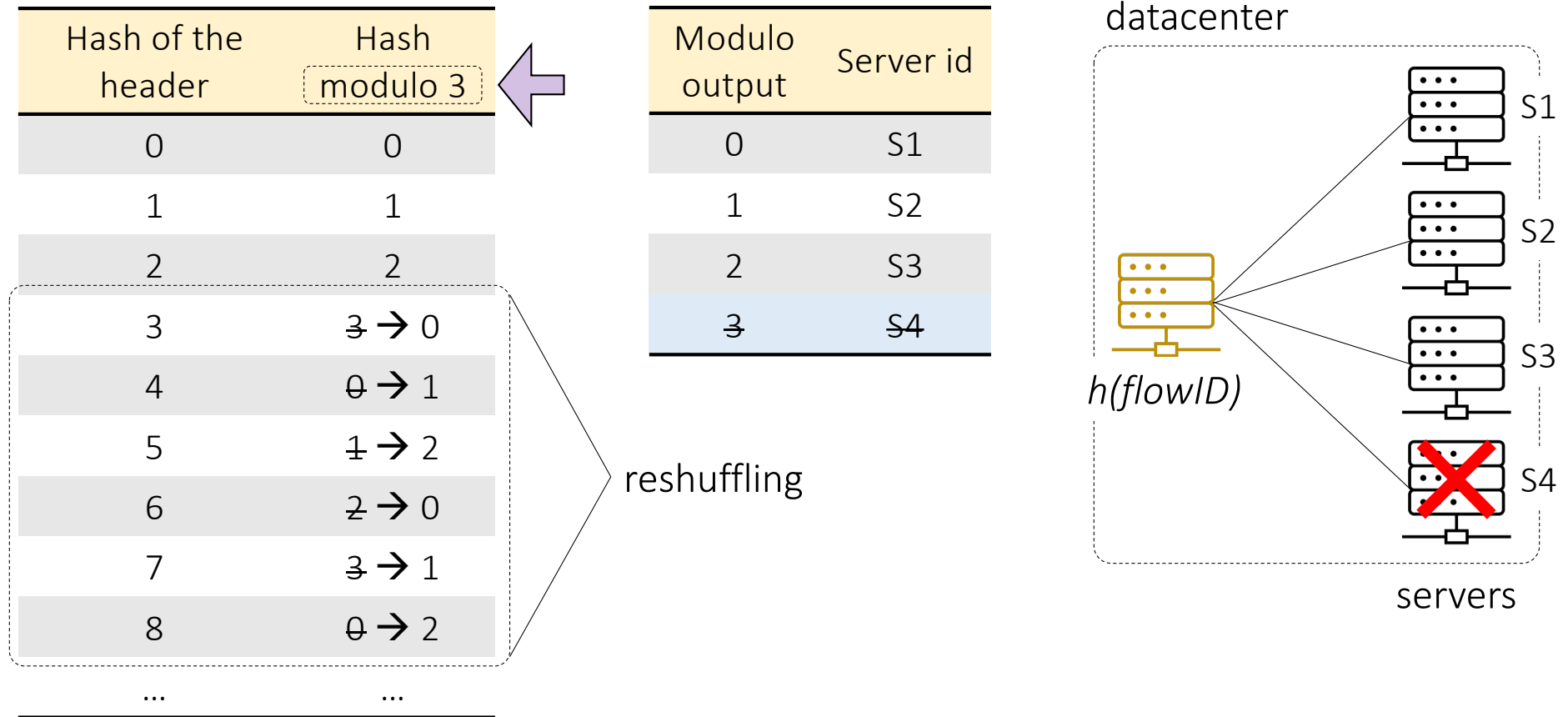
What happens if a server fails?

Hash of the header	Hash modulo 4
0	0
1	1
2	2
3	3
4	0
5	1
6	2
7	3
8	0
...	...

Modulo output	Server id
0	S1
1	S2
2	S3
3	S4



What happens if a server fails?

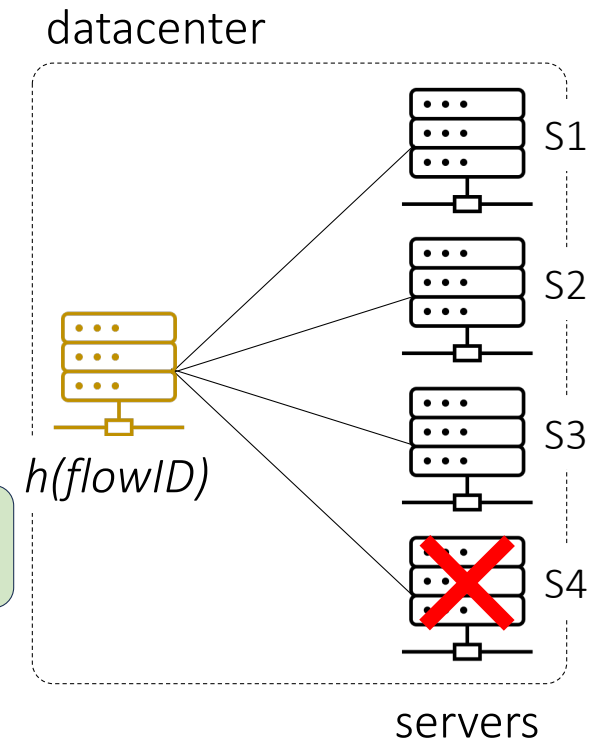


What happens if a server fails?

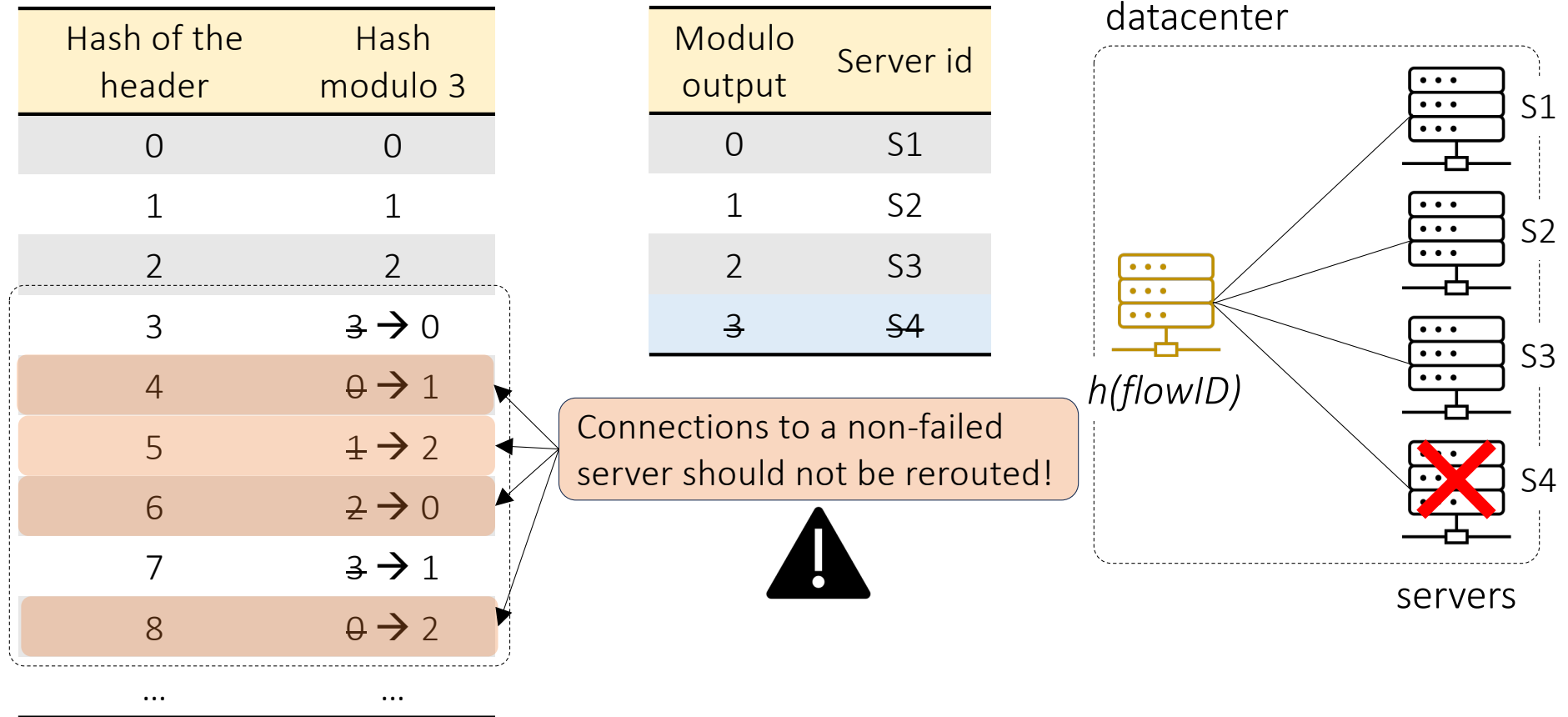
Hash of the header	Hash modulo 3
0	0
1	1
2	2
3	$3 \rightarrow 0$
4	$0 \rightarrow 1$
5	$1 \rightarrow 2$
6	$2 \rightarrow 0$
7	$3 \rightarrow 1$
8	$0 \rightarrow 2$
...	...

Modulo output	Server id
0	S1
1	S2
2	S3
3	S4

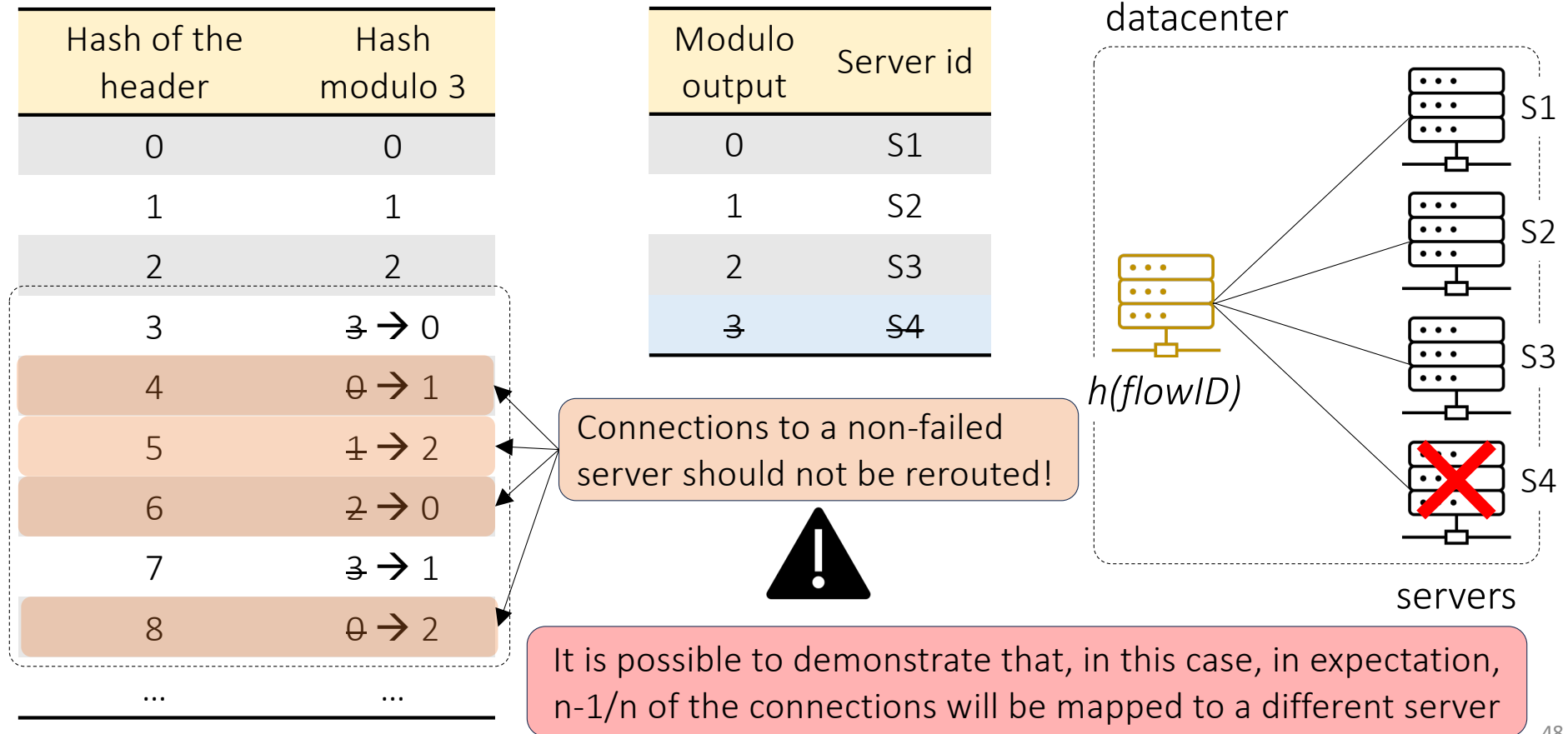
Connections to a failed server must be rerouted in any case



What happens if a server fails?



What happens if a server fails?



What happens if a server fails?

This is the case of having 5 backend servers, originally using hash modulo 5 and then resort to modulo 4 after the failure of server 5

```

111111111122222222223333333333
0123456789012345678901234567890123456789
+-----+
|1234512345123451234512345123451234512345 <- hash mod 5
+-----+
|1234123412341234123412341234123412341234 <- hash mod 4
+-----+

```

What happens if a server fails?

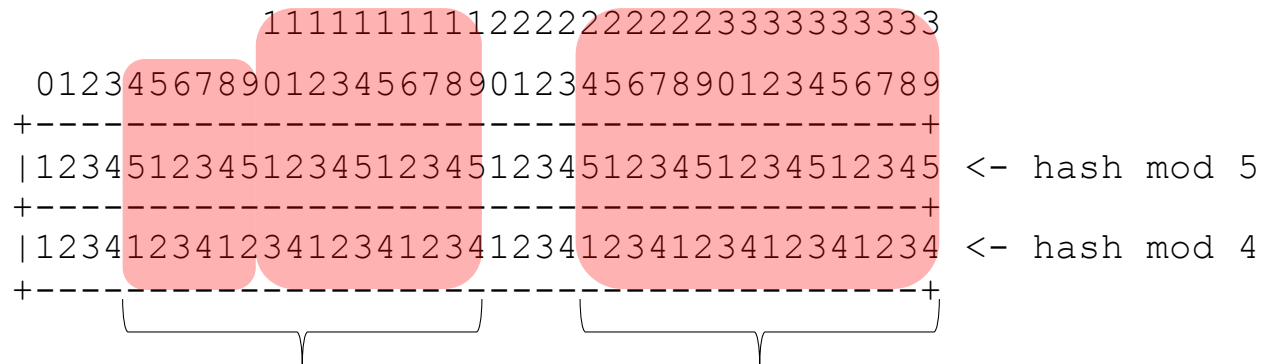
This is the case of having 5 backend servers, originally using hash modulo 5 and then resort to modulo 4 after the failure of server 5

1111111111										2222222222										3333333333																			
0123456789	0123456789	0123456789	0123456789	0123456789	0123456789	0123456789	0123456789	0123456789	0123456789																														
+-----+																																							
12345	12345	12345	12345	12345	12345	12345	12345	12345	12345	← hash mod 5																													
+-----+																																							
12341	23412	34123	41234	12341	23412	34123	41234	12341	23412	← hash mod 4																													
+-----+																																							
└──────────────────┘																				└──────────────────┘																			

ideally, the changes (in red) shall affect only the failed server (number 5)

What happens if a server fails?

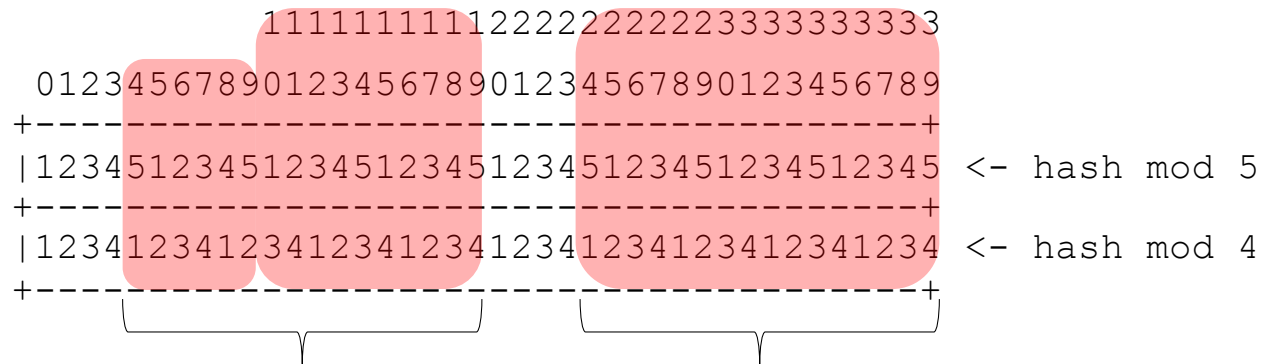
This is the case of having 5 backend servers, originally using hash modulo 5 and then resort to modulo 4 after the failure of server 5



Using hash mod 4, however, forces us to change many more backend servers than needed!

What happens if a server fails?

This is the case of having 5 backend servers, originally using hash modulo 5 and then resort to modulo 4 after the failure of server 5



The challenge: how to reduce the “red spaces” as much as possible!

Can we do better?

Consistent hashing

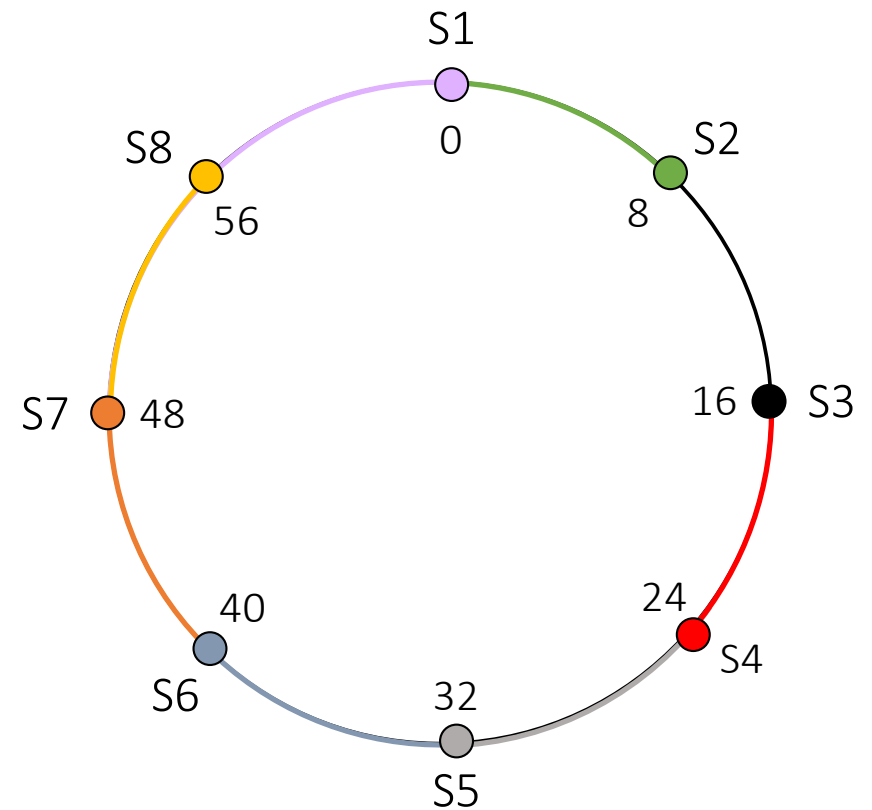
is a special kind of hashing technique designed so that changes in the hash table size cause only a small fraction of keys to be remapped ($\approx 1/m$ per slot change).

m is the number of buckets (servers / machines participating in the hash ring)

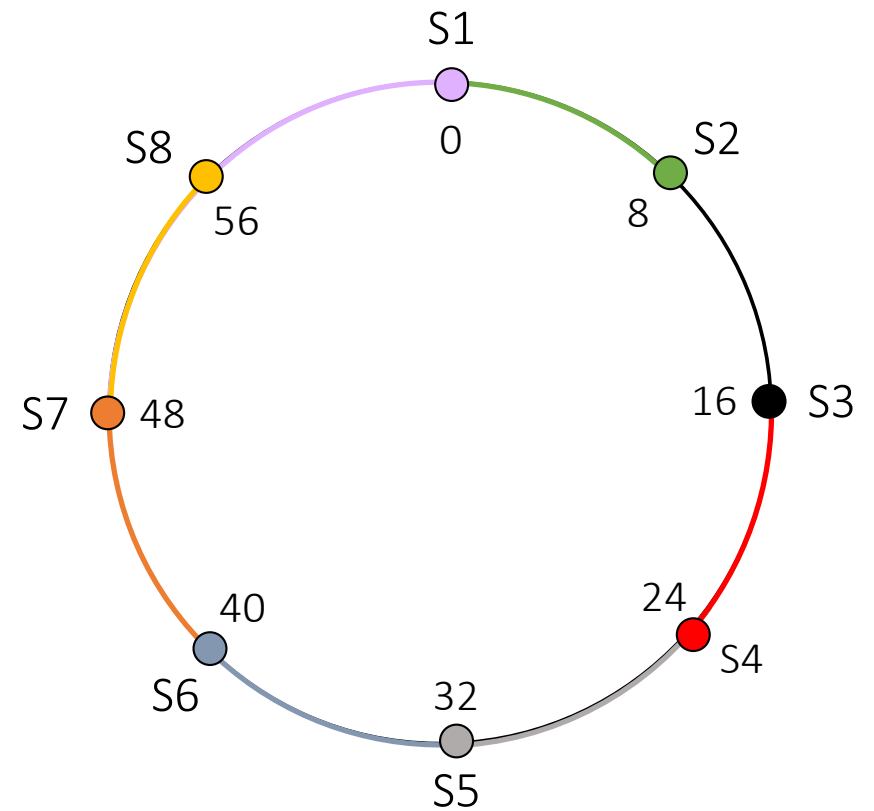
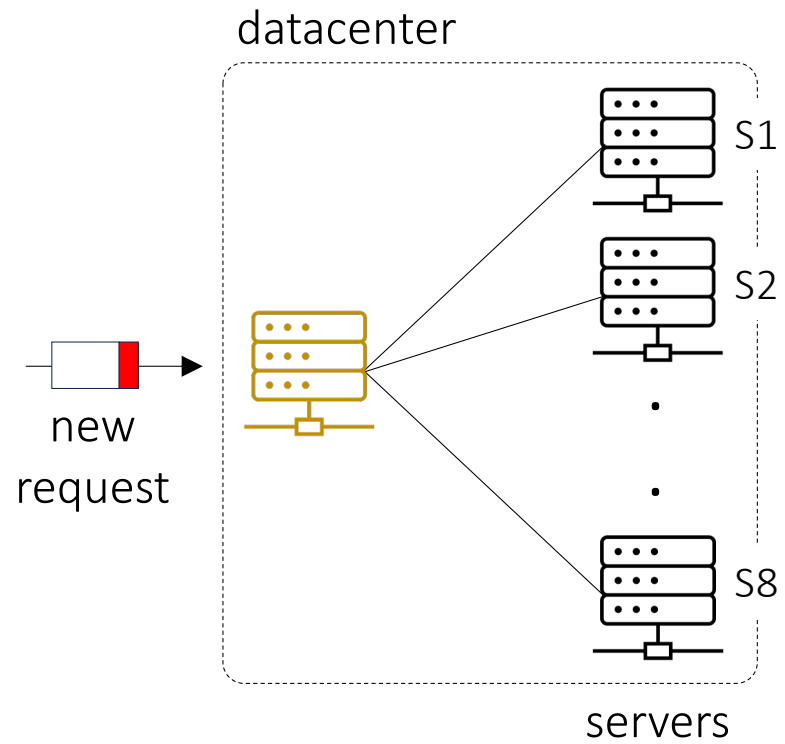
Can we do better?

Consistent hashing

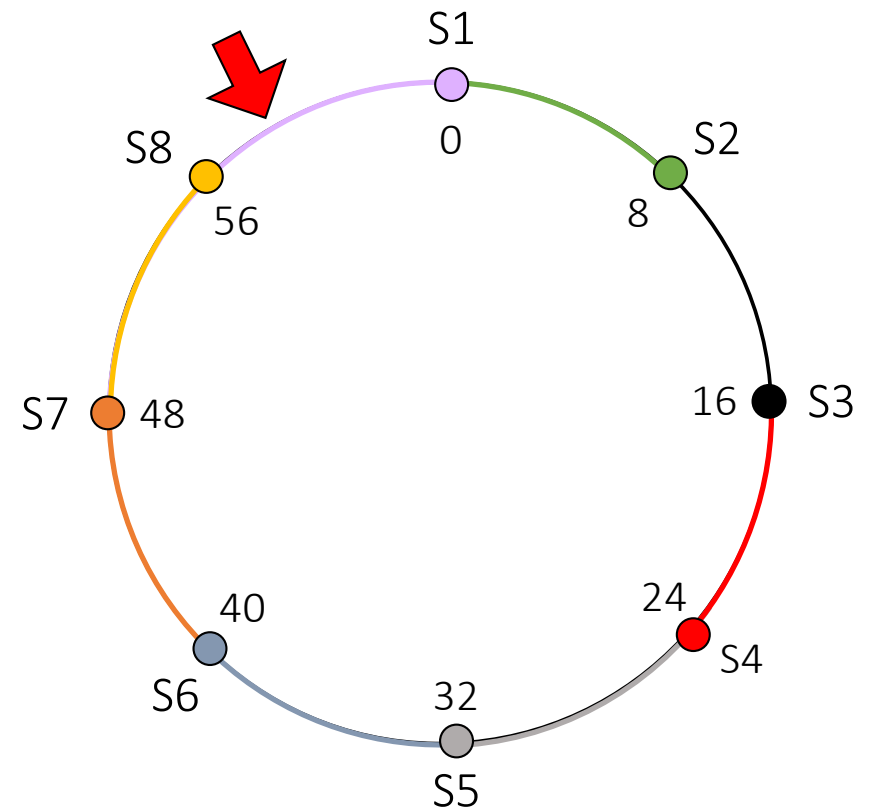
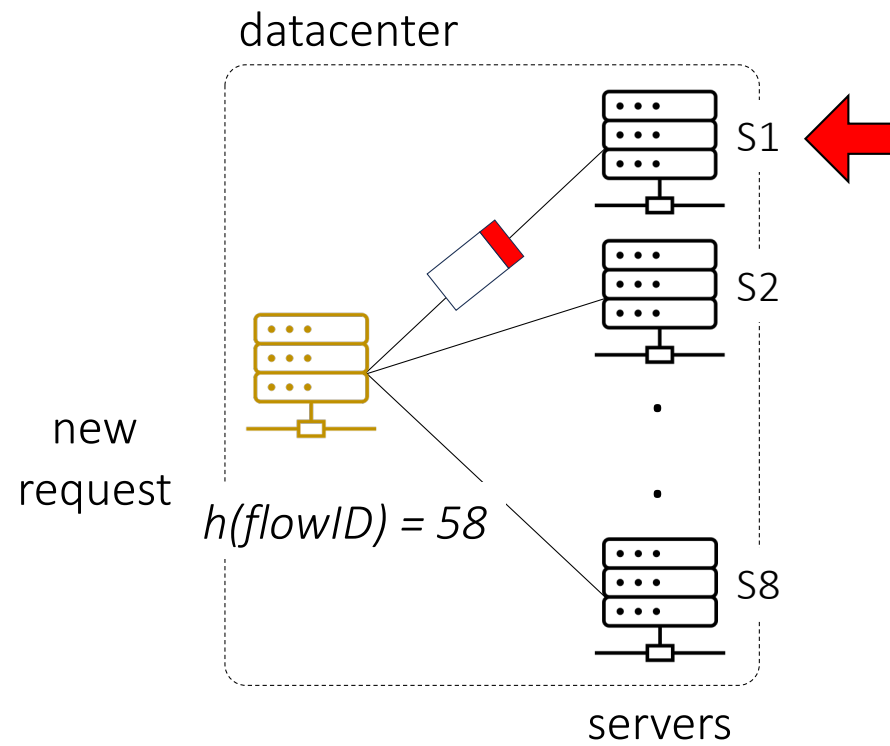
- Every server is mapped to a point on a circle
- Every server is responsible for handling the hashing space until its predecessor



Can we do better?



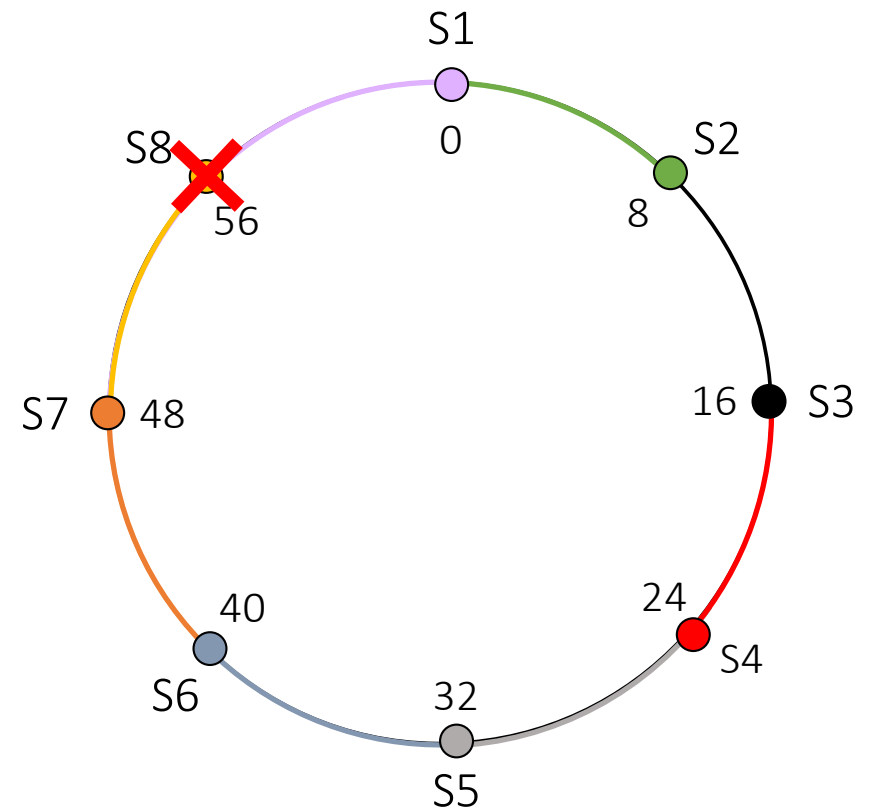
Can we do better?



Can we do better?

Consistent hashing

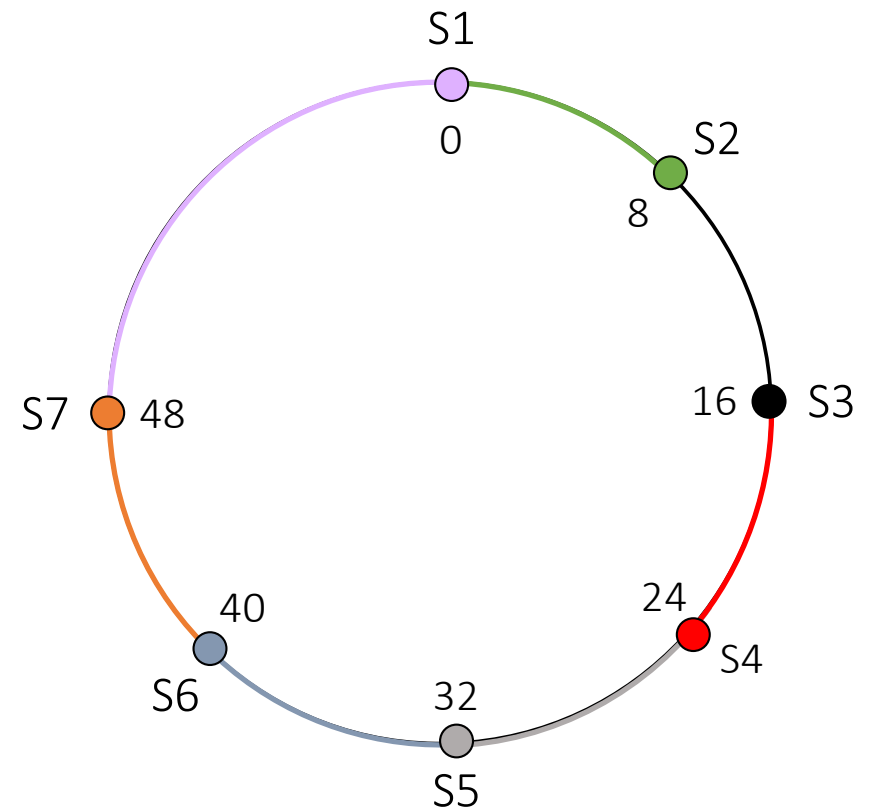
- When a server fails...



Can we do better?

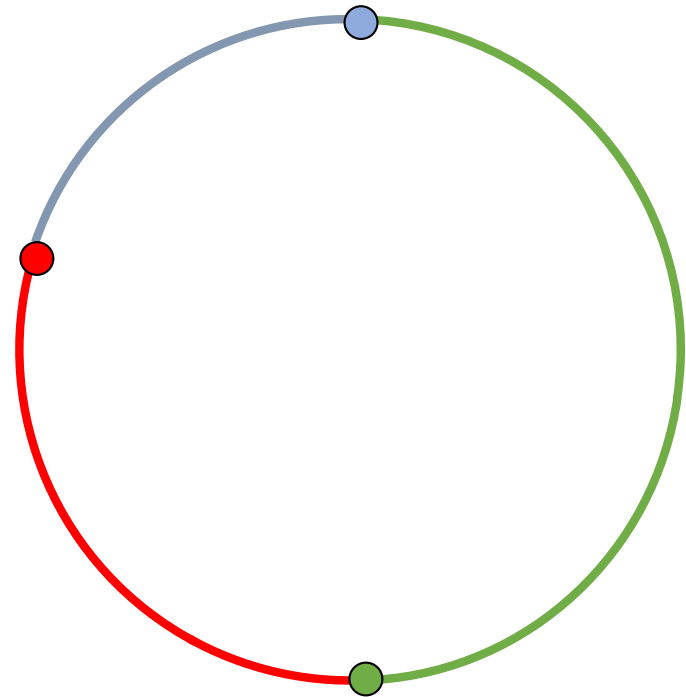
Consistent hashing

- When a server fails, its space is taken by its successor
- This minimize disruption in case of server fail



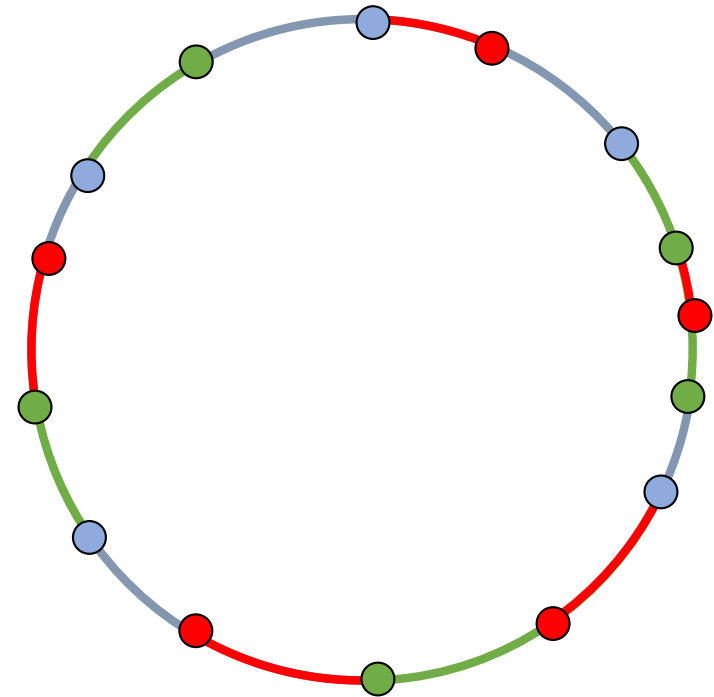
Can we do better?

- Mapping a server to a single point might lead to imbalances
- Ideas for a solution?



Can we do better?

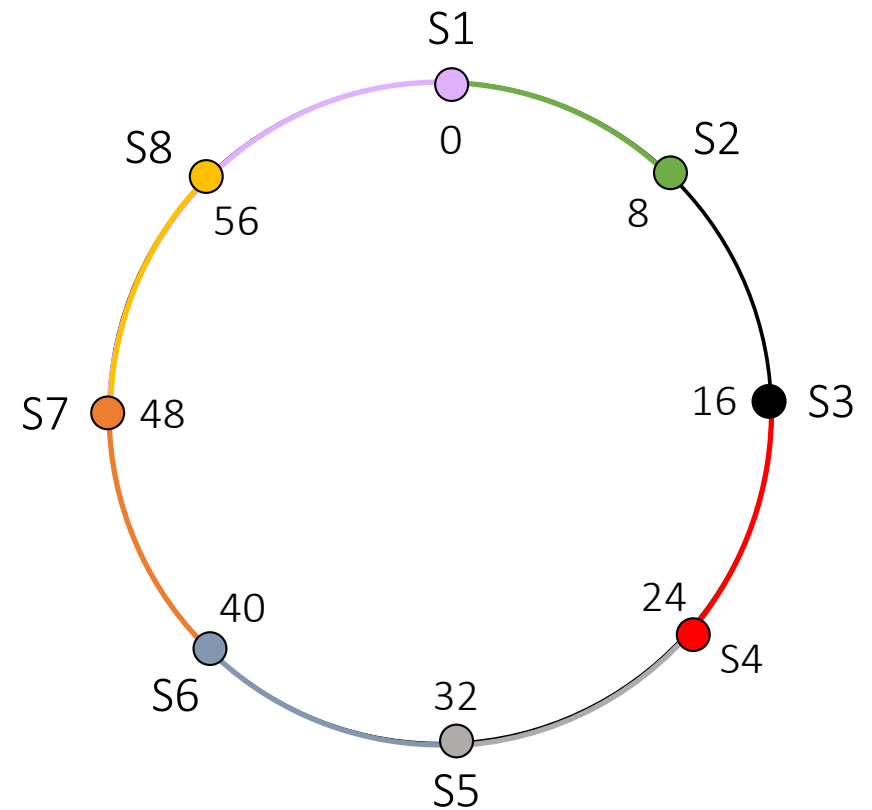
- Mapping a server to a single point might lead to imbalances
- We can map each server to *multiple* points on the circle
- When adding a new node, on expectation the load will be balanced



Consistent hashing

What happens when we add a node?

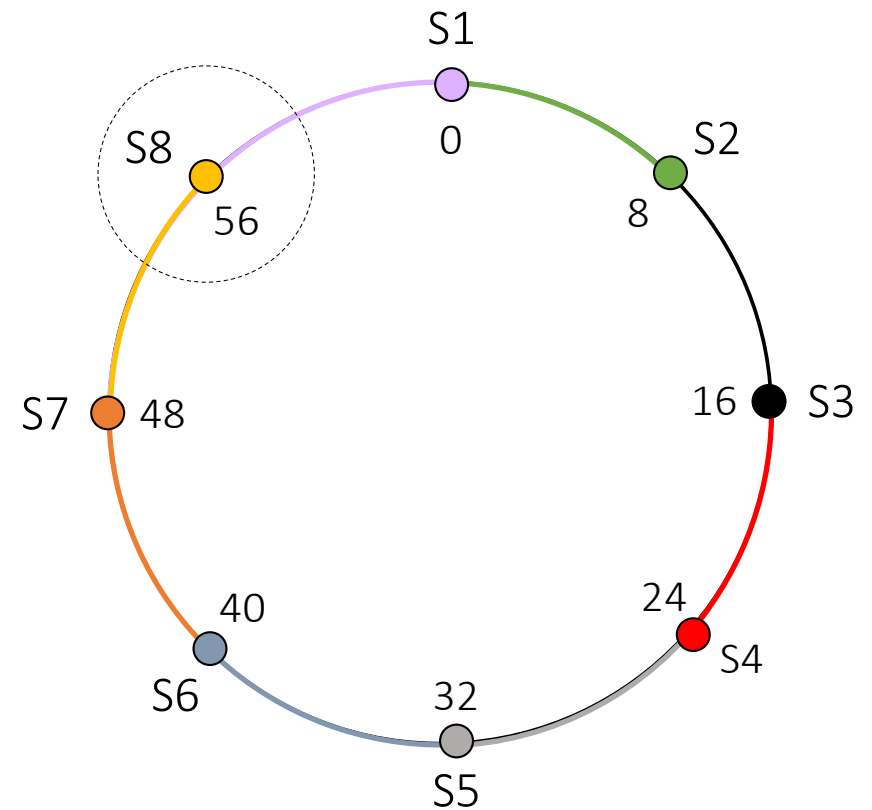
- Some headers are assigned to the new server
- Previous connections shall be routed to the old server



Consistent hashing

What happens when we add a node?

- Some headers are assigned to the new server
 - Previous connections shall be routed to the old server
- Assuming we just added S8
 - How do we know some of the connections routed to S8 were pre-existing and shall be forwarded to S1, instead?



Going back to stateful load balancers?

Forwarding

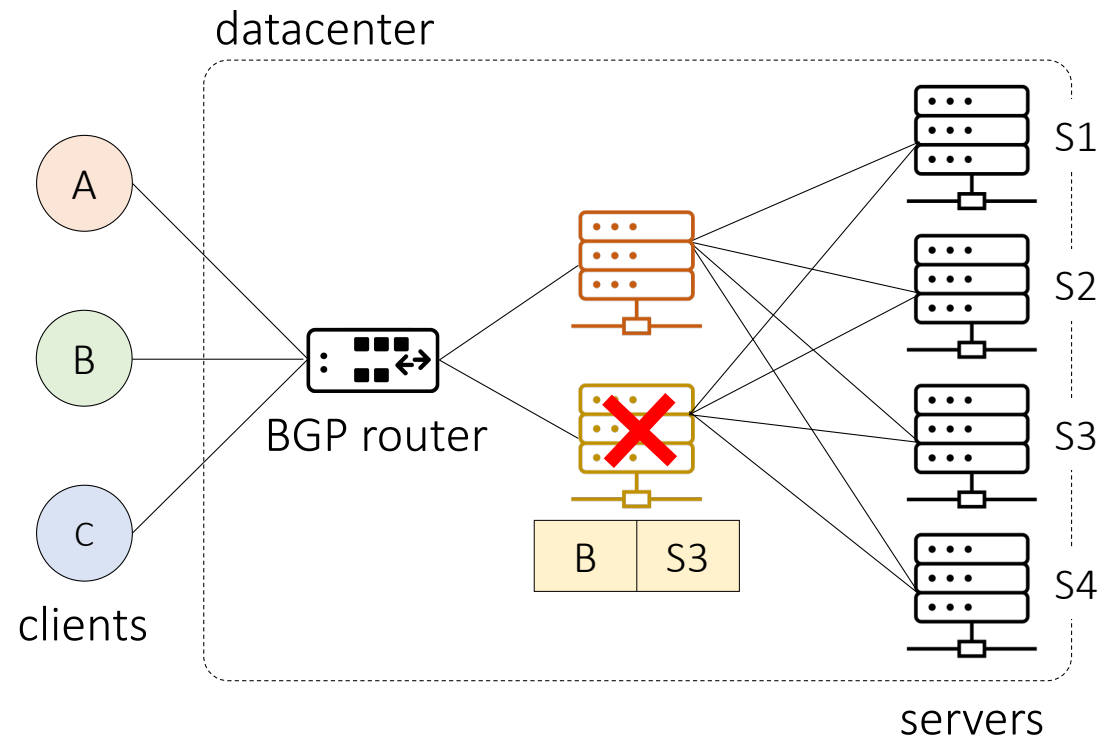
- For new connections, use consistent hashing and store mapping header -> server
- For existing connections, use stored mapping

FlowID	Server id
ip_src=192.4.1.8, ip_dst=192.4.1.8, tcp_src=32487, tcp_dst=80	2
ip_src=212.3.5.4, ip_dst=192.4.1.8, tcp_src=758, tcp_dst=80	4

Going back to stateful load balancers?

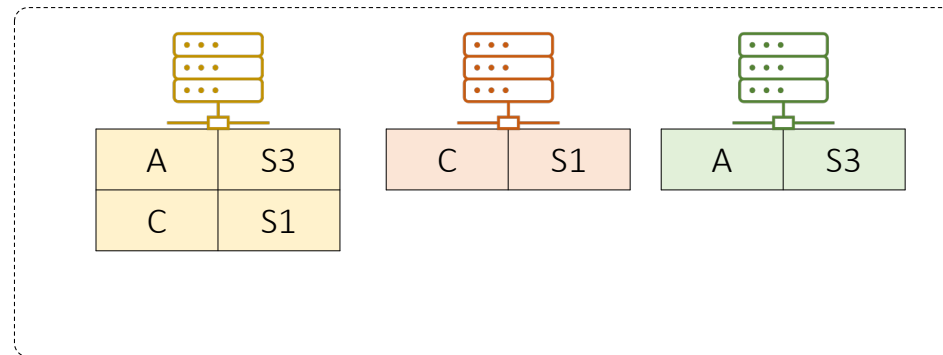
Unfortunately, this might not be enough ☹️

- What if the per-flow state is lost? (a load balancer goes down)



Going back to stateful load balancers?

- You can replicate state at load balancers

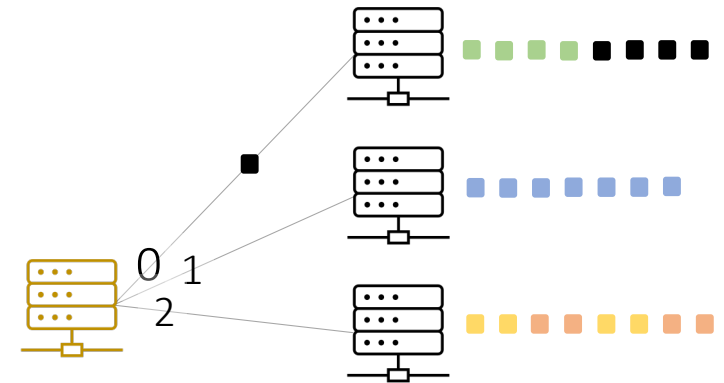


- Increased latency for retrieving state of existing connections

Recap: a load balancer should..

- Preserve per-connection-consistency (PCC)

PCC

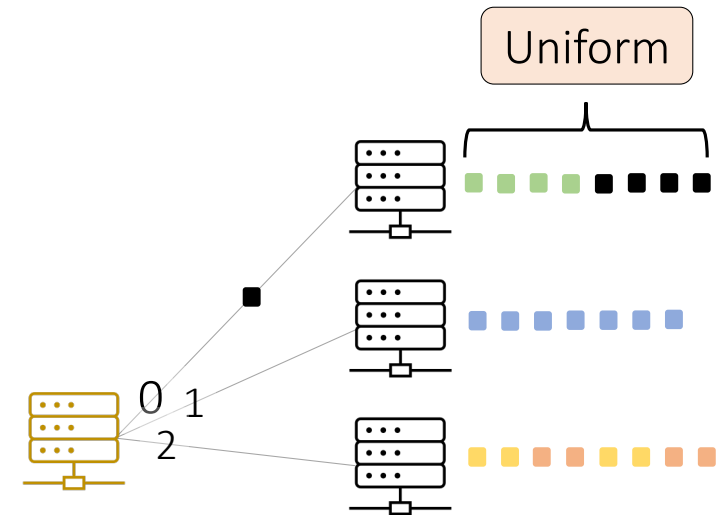


Connection Table

conn-id	nexthop
	1
 	0
 	2

Recap: a load balancer should..

- Preserve per-connection-consistency (PCC) PCC
- Spread the requests uniformly between backend servers

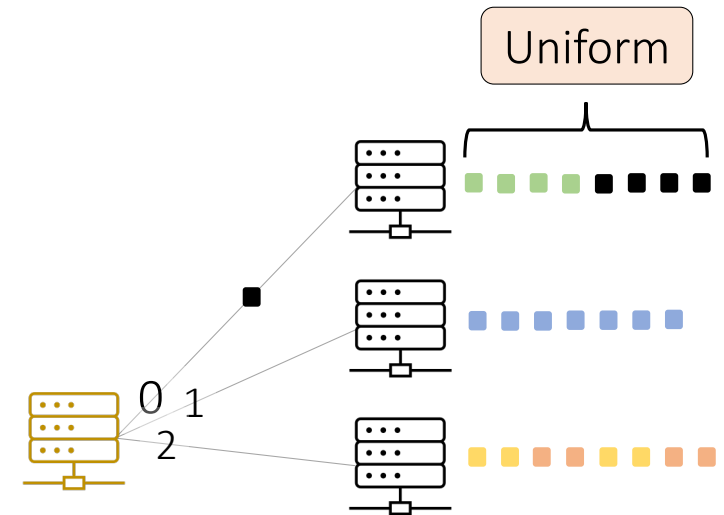


Connection Table

conn-id	nexthop
	1
 	0
 	2

Recap: a load balancer should..

- Preserve per-connection-consistency (PCC) PCC
- Spread the requests uniformly between backend servers
- Efficiently handle many requests (fast decisions) Efficient

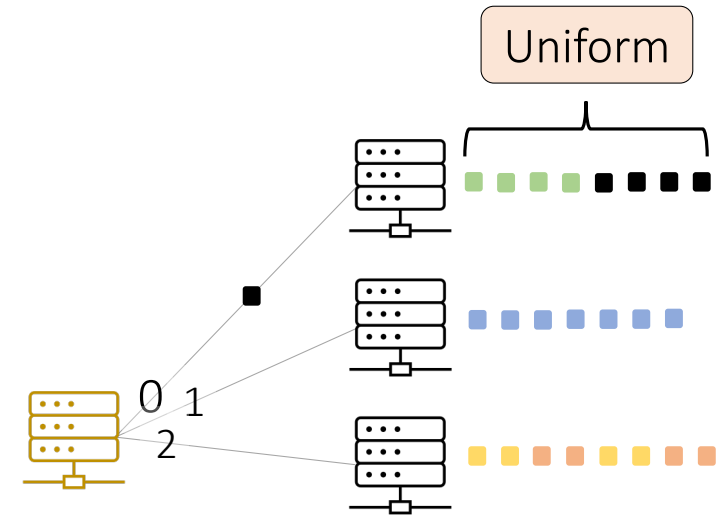


Connection Table

conn-id	nexthop
	1
 	0
 	2

Recap: a load balancer should..

- Preserve per-connection-consistency (PCC) PCC
- Spread the requests uniformly between backend servers
- Efficiently handle many requests (fast decisions) Efficient
- Able to dynamically add new servers without causing disruptions Dynamic



Connection Table

conn-id	nexthop
	1
 	0
 	2

Cheetah: A High-Speed Load-Balancer Design with Guaranteed Per-Connection-Consistency

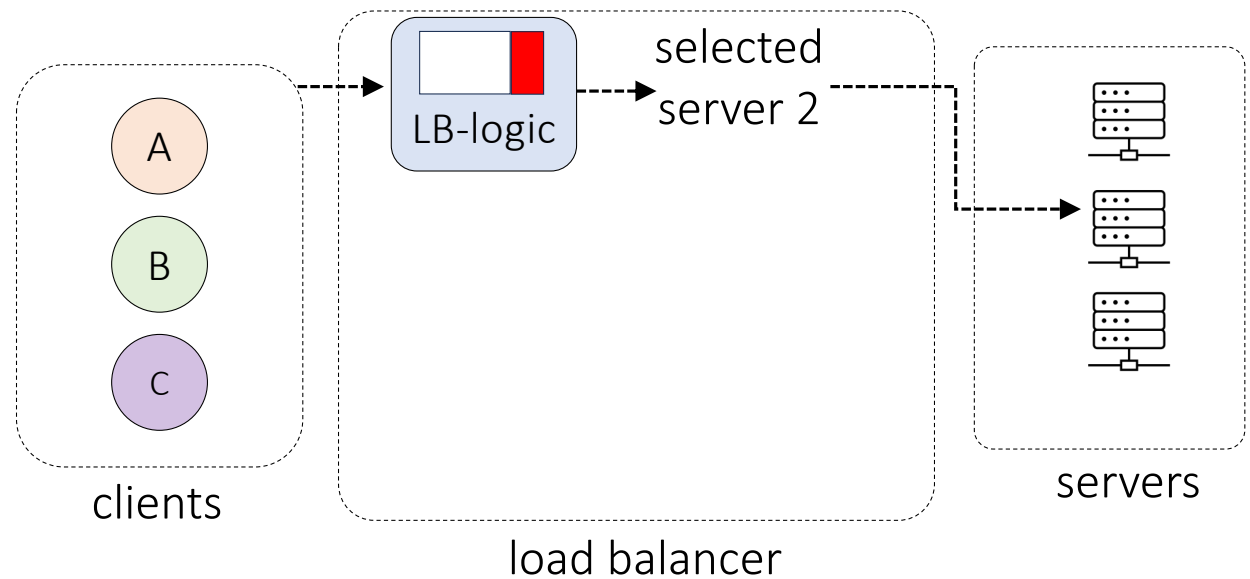
A High-Speed Load-Balancer Design with Guaranteed Per-Connection-Consistency

Tom Barbette Chen Tang Haoran Yao Dejan Kostić
Gerald Q. Maguire Jr. Panagiotis Papadimitratos Marco Chiesa
KTH Royal Institute of Technology

- A solution proposed to meet all the previous requirements
- Work presented at USENIX NSDI 2020 (Santa Clara)

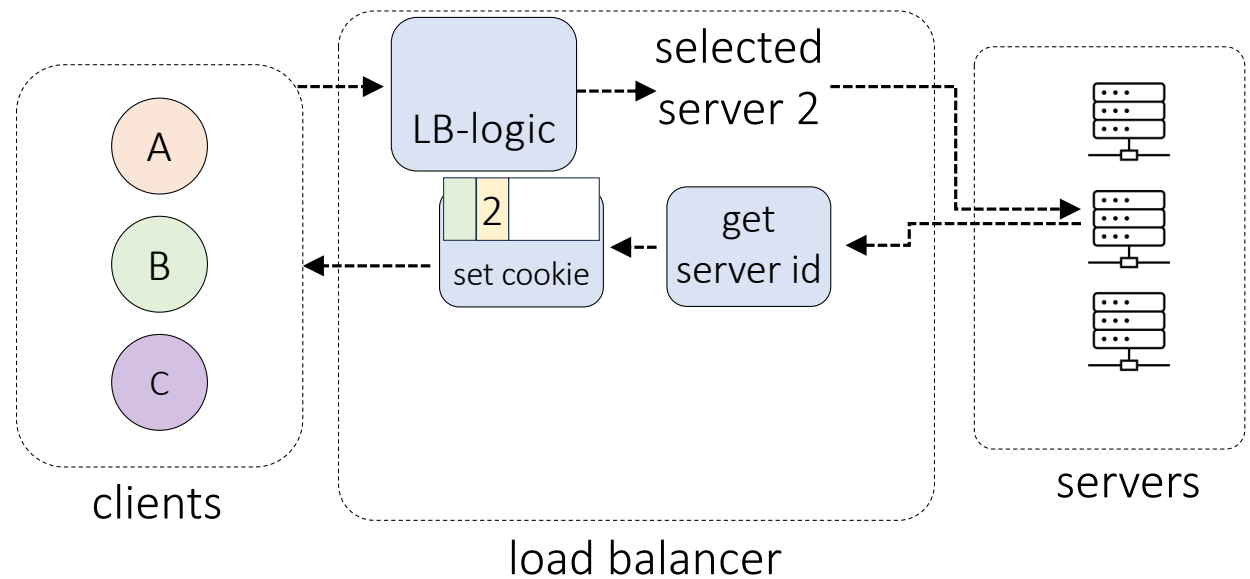
Cheetah overview

- A new request from a user hit the load balancer and a server is selected



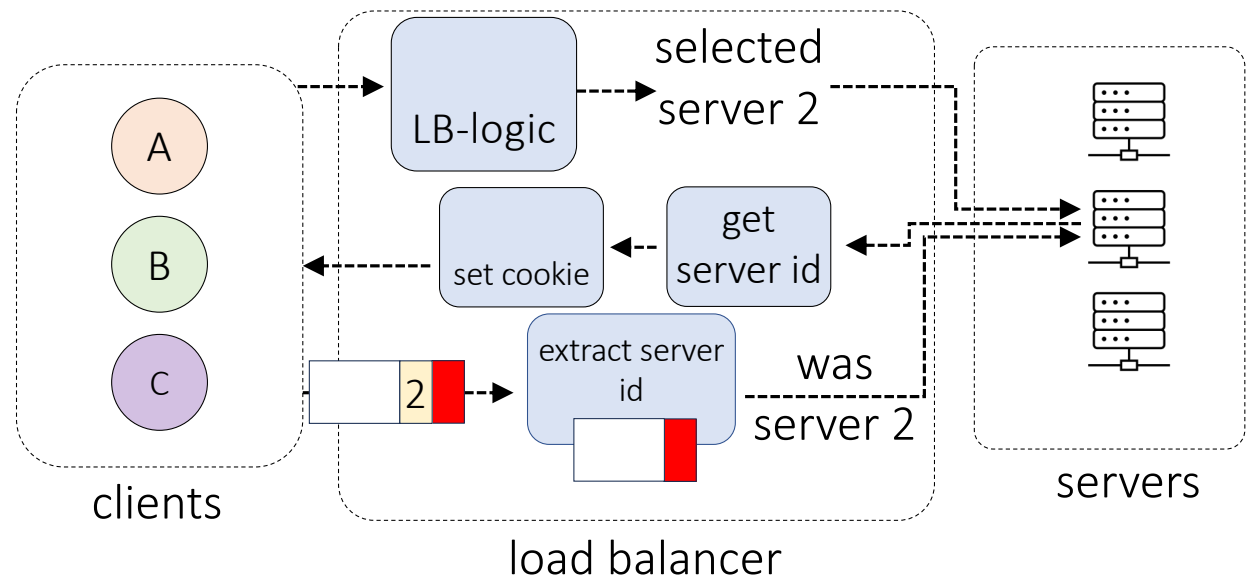
Cheetah overview

- A new request from a user hit the load balancer and a server is selected
- On the way back a cookie is inserted in the reply



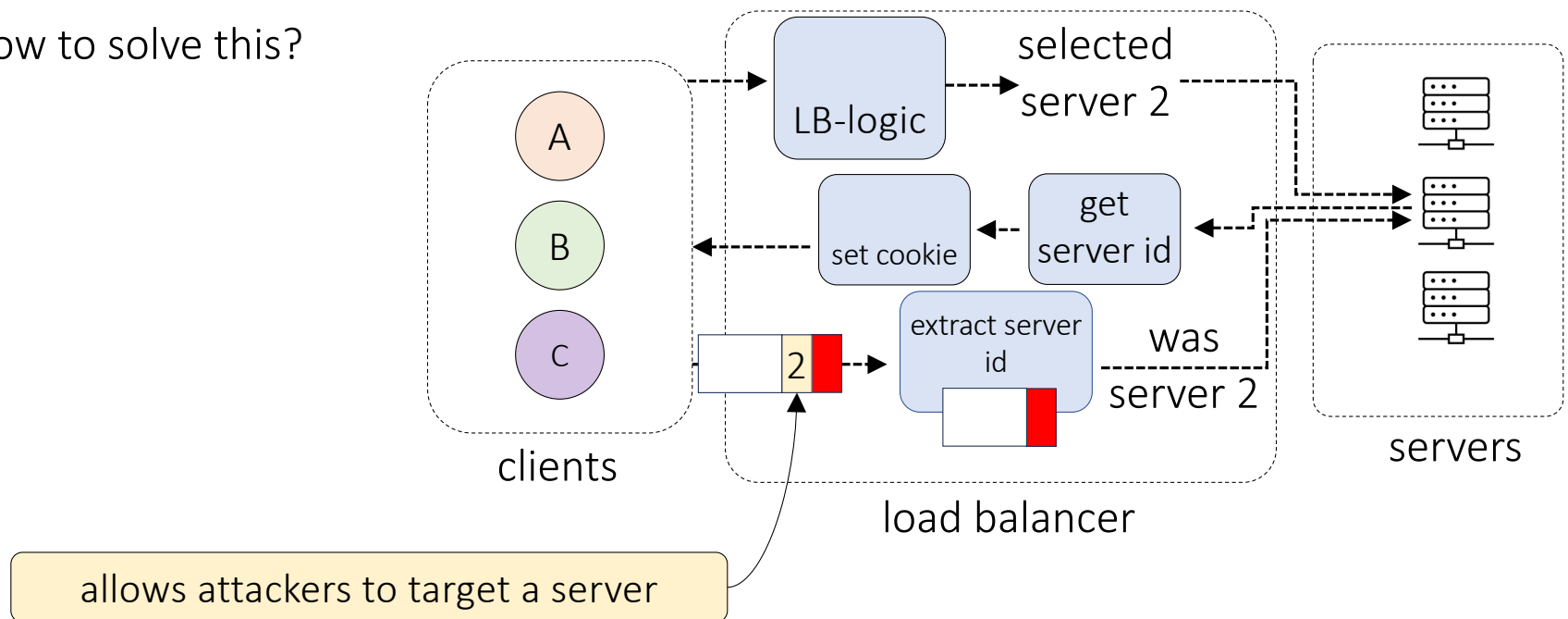
Cheetah overview

- A new request from a user hit the load balancer and a server is selected
- On the way back a cookie is inserted in the reply
- New packets of the same flow need just to insert the cookie (the user remember)

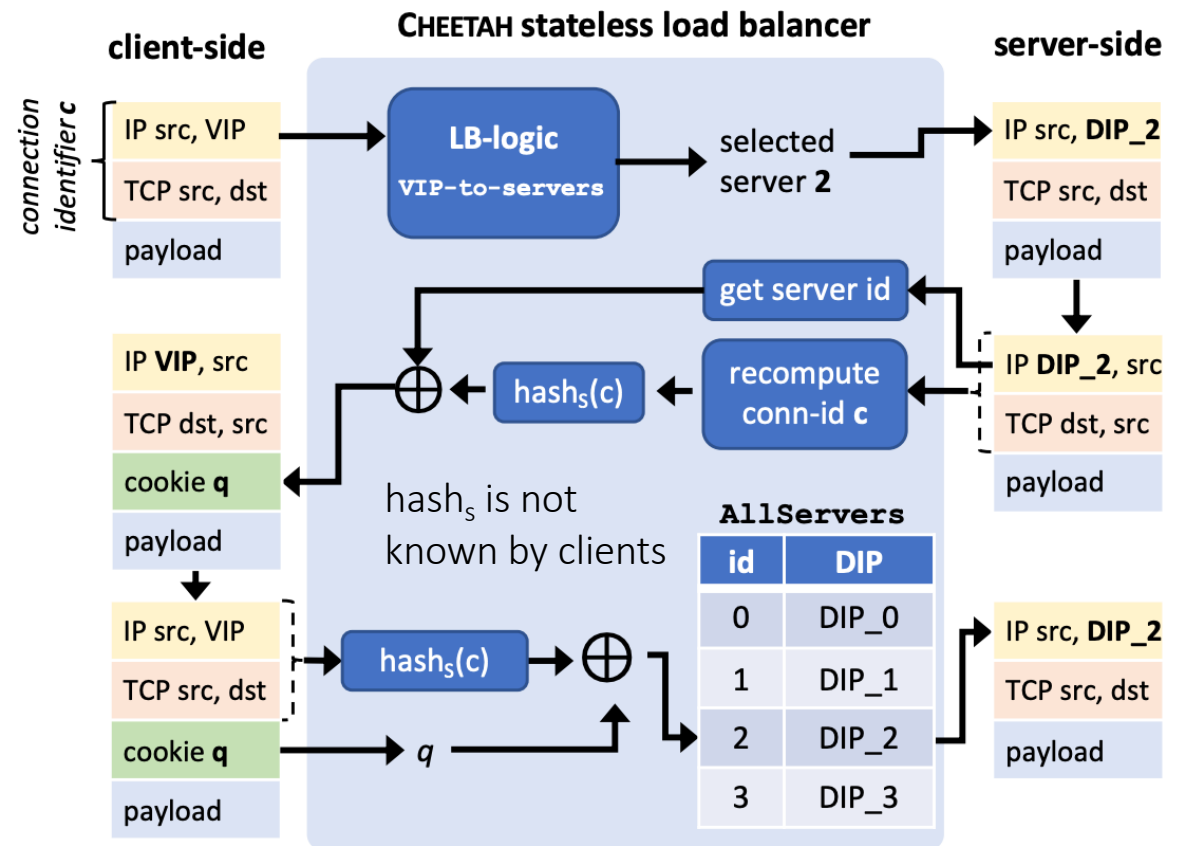


Cheetah overview

- Ideas on how to solve this?

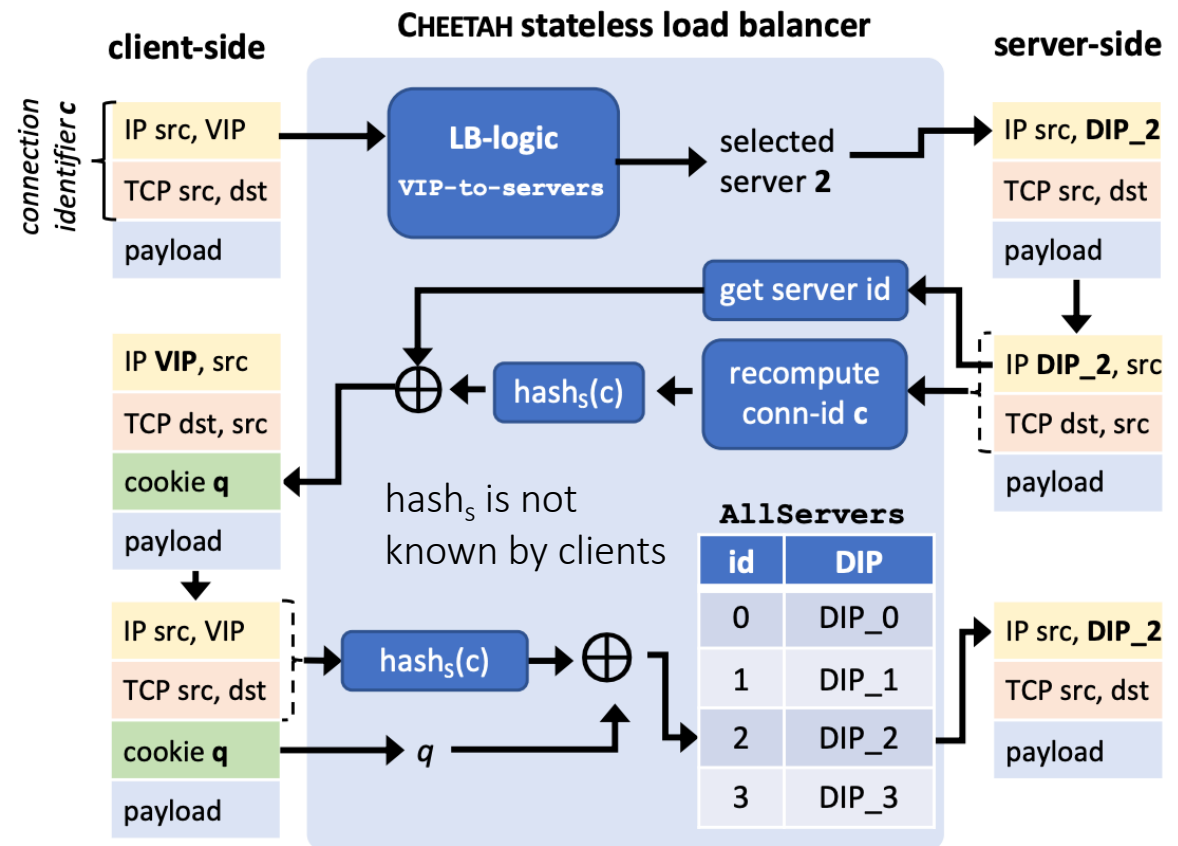


Cheetah overview



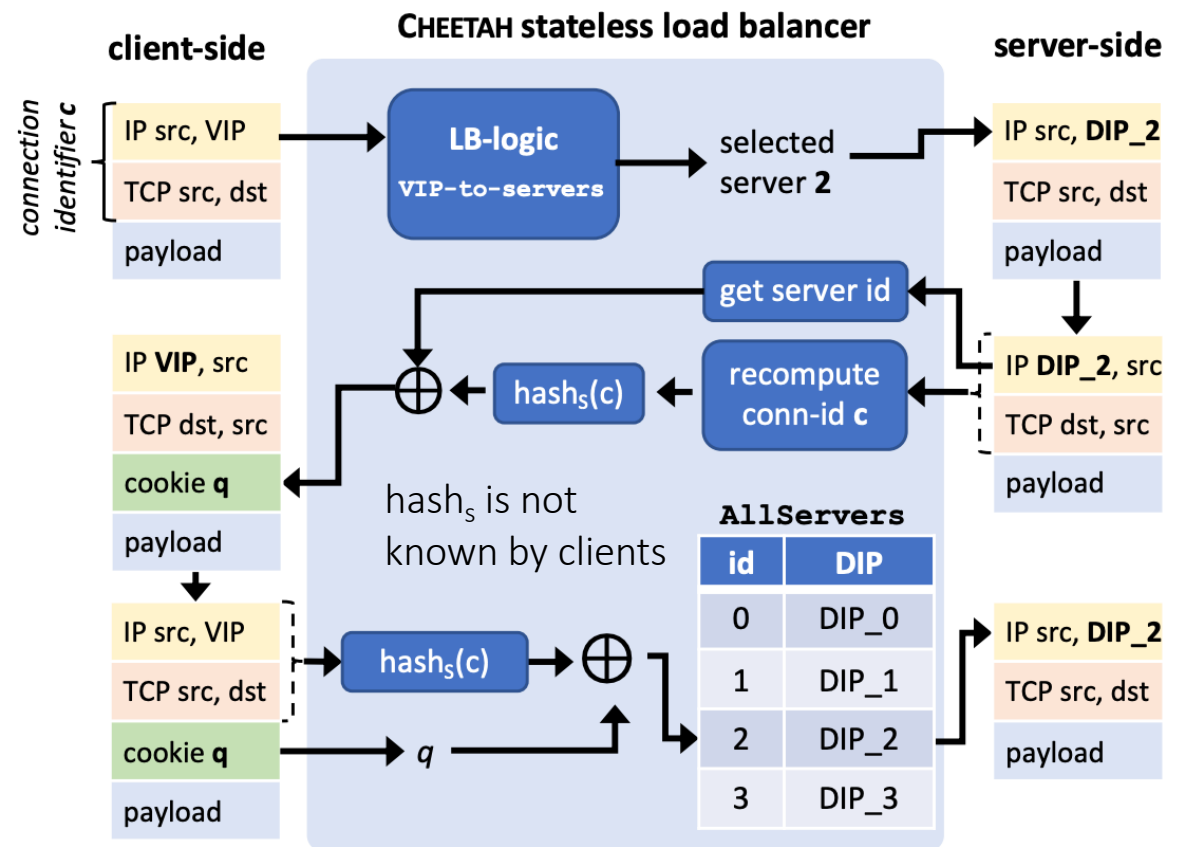
Cheetah overview

- Are you bounded to a specific load balancing algorithm?



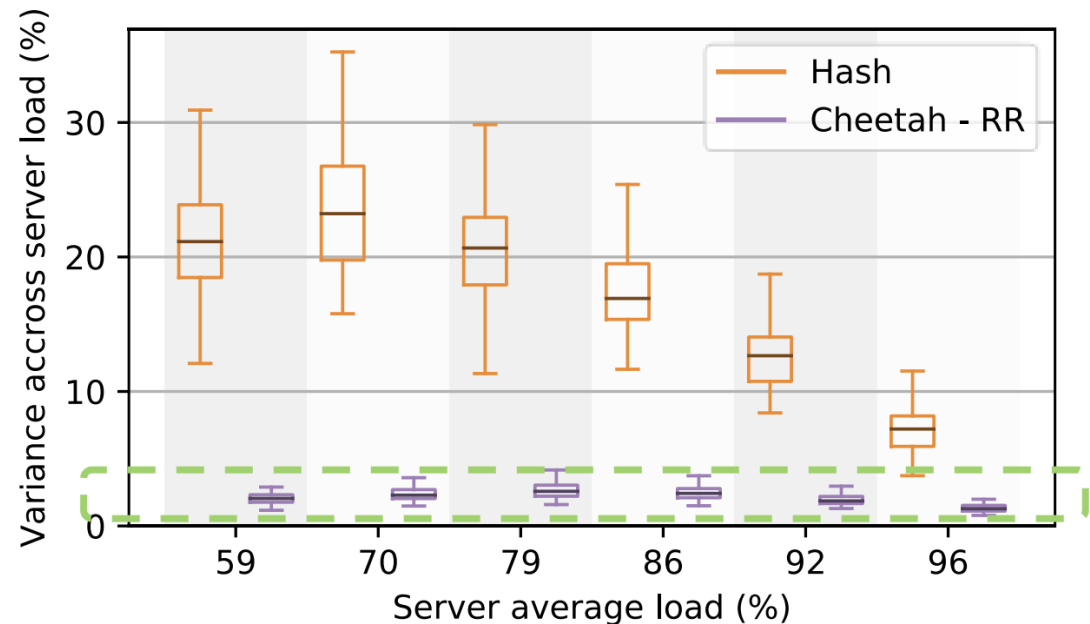
Cheetah overview

- Are you bounded to a specific load balancing algorithm?
 - You can implement your preferred load balancing policy and get PCC for free!
 - You can even implement round robin!



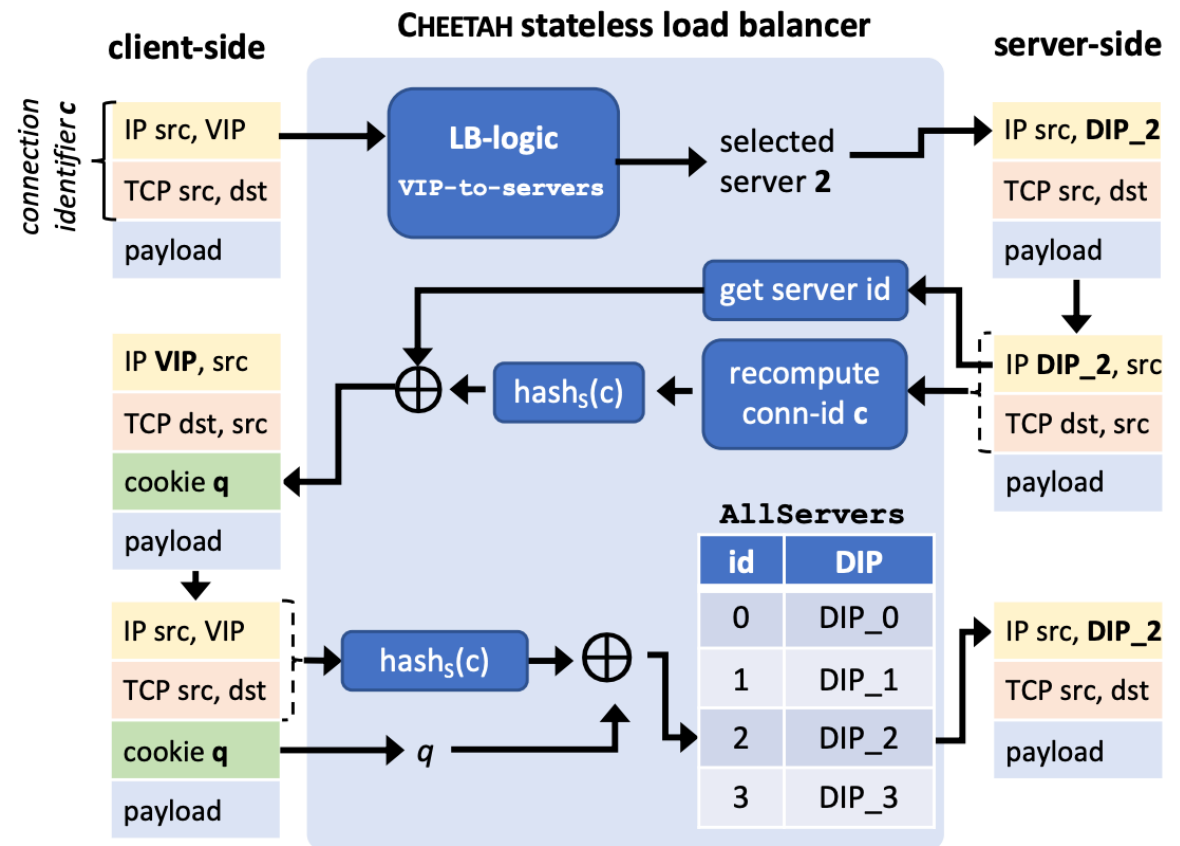
Cheetah in action

- Requests for a 8K file using HTTP to 64 servers
- Hash: 20-30% variance between medium to high load (this is consistent with Google Maglev LB which overprovision up to 30%)
- Cheetah LB can bring down variance



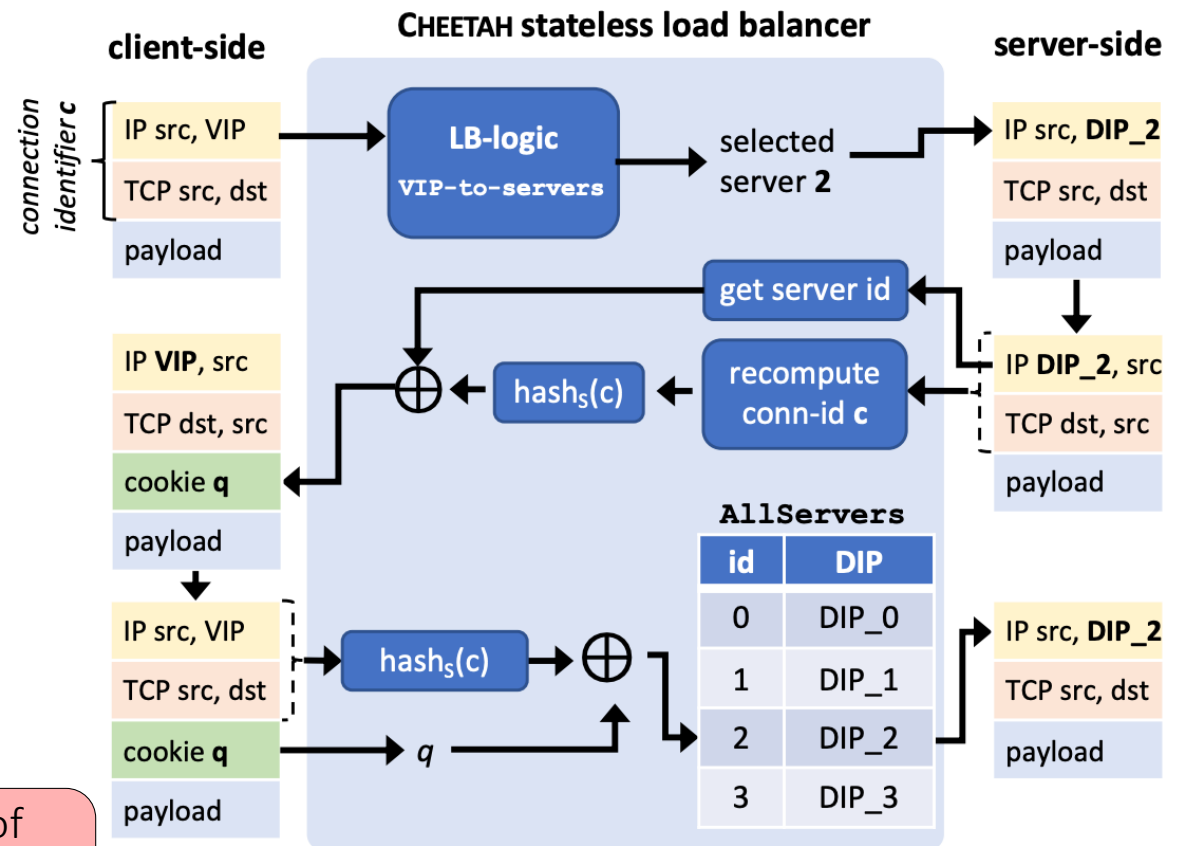
Discussion

- Is Round Robin always the best choice?



Discussion

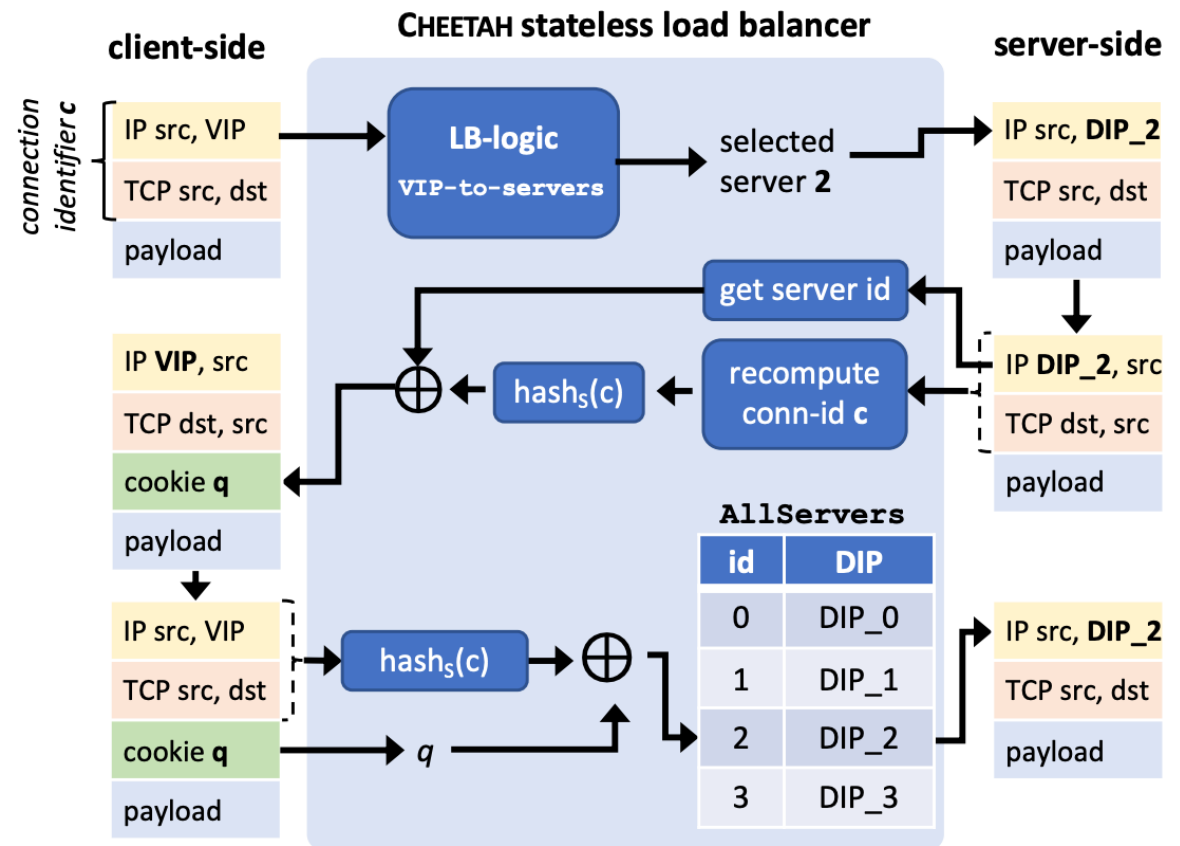
- Is Round Robin always the best choice?



No. Different requirements in terms of computation between different services/requests

Discussion

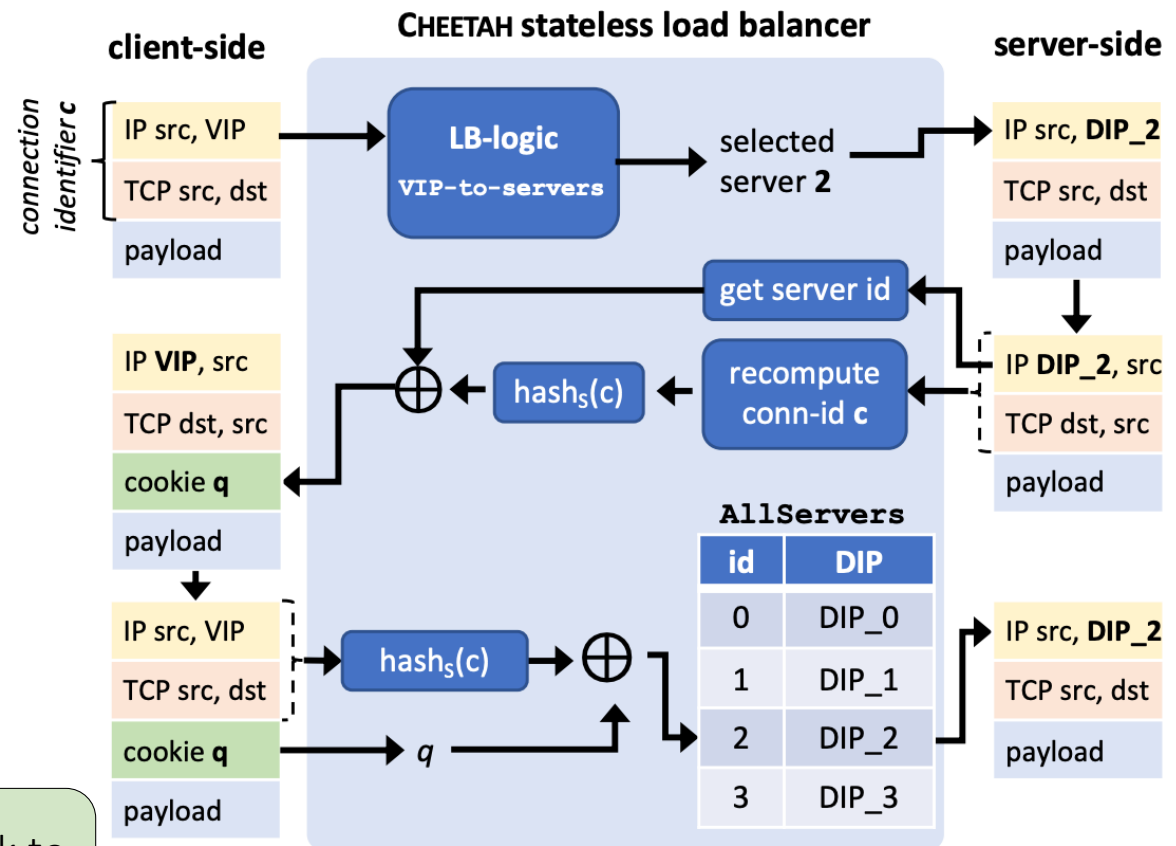
- Is Round Robin always the best choice?
- Can you implement DSR with Cheetah?



Discussion

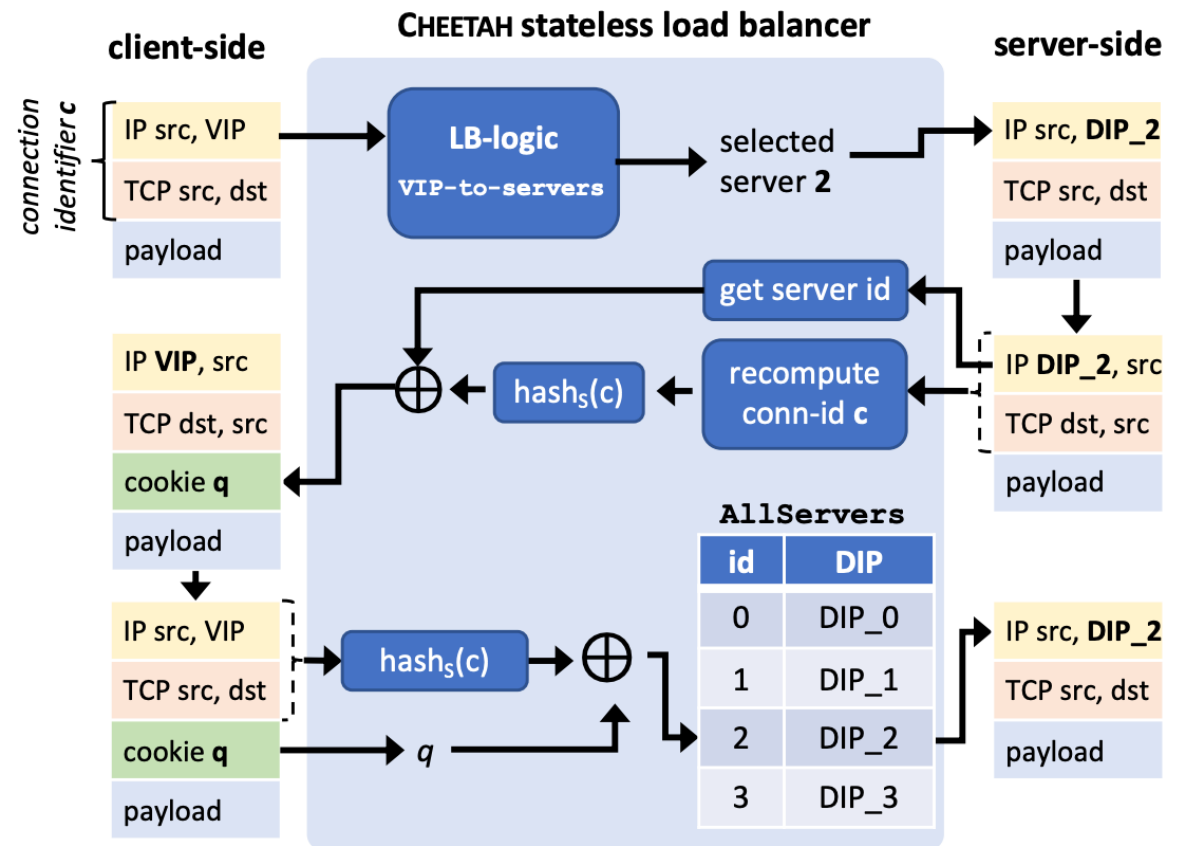
- Is Round Robin always the best choice?
- Can you implement DSR with Cheetah?

Yes. You need to change the server stack to compute the cookie though



Discussion

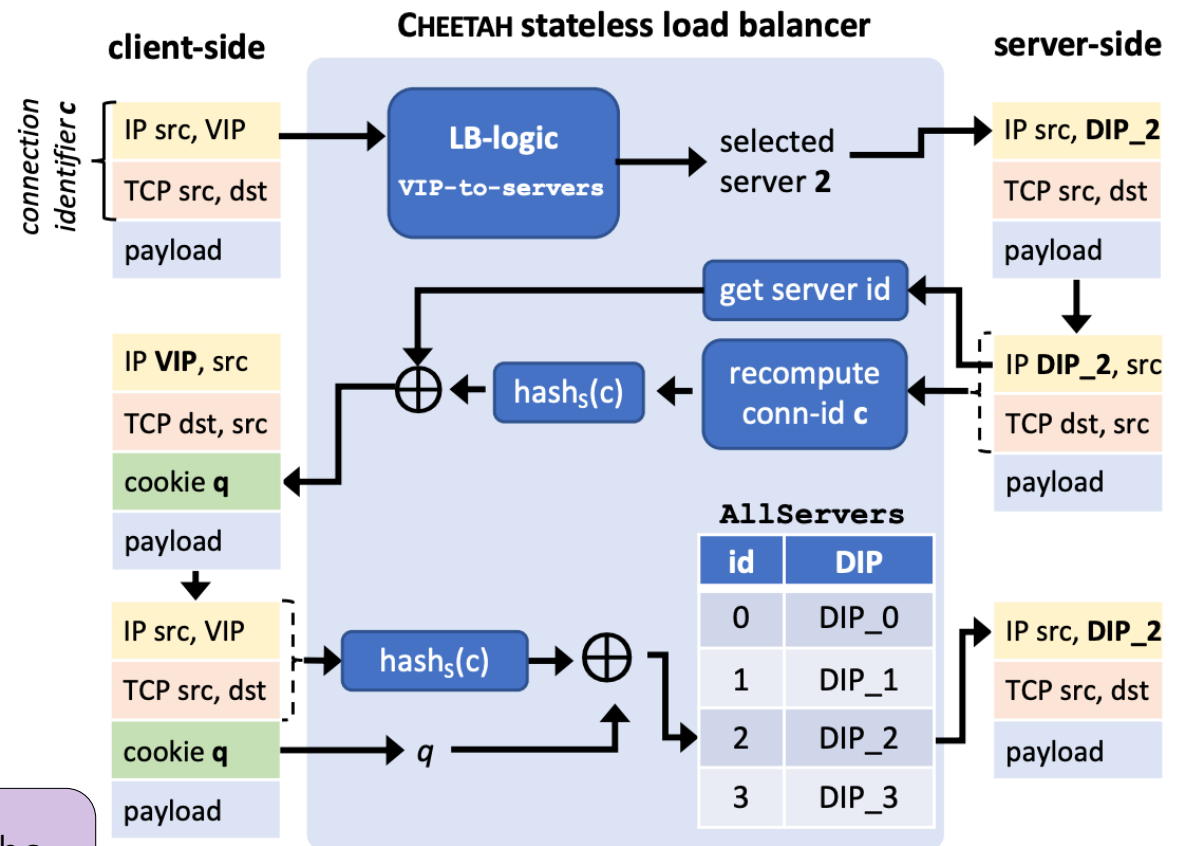
- Is Round Robin always the best choice?
- Can you implement DSR with Cheetah?
- Does Cheetah require changes at the client side?



Discussion

- Is Round Robin always the best choice?
- Can you implement DSR with Cheetah?
- Does Cheetah require changes at the client side?

No. You can use the TCP timestamp echo option



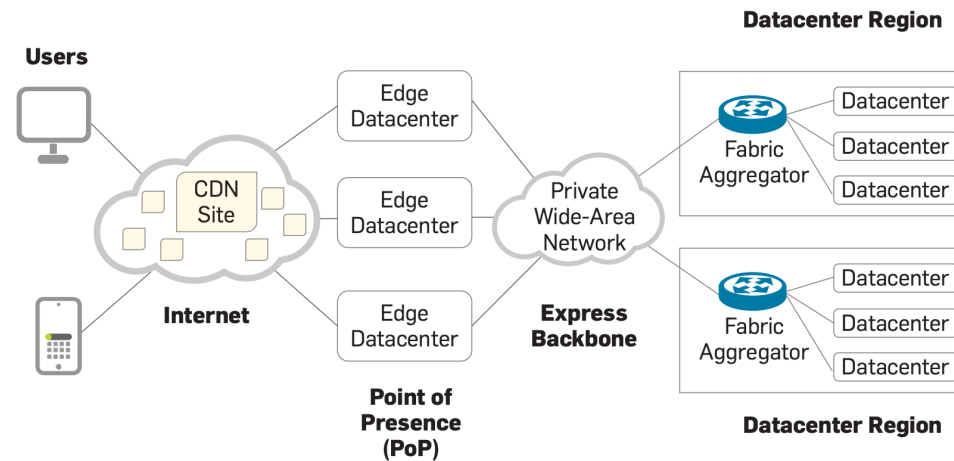
Balancing on the Edge: transport Affinity without Network State

Balancing on the edge: transport affinity without network state

João Taveira Araújo, Lorenzo Saino, Lennert Buytenhek and Raul Landa
Fastly

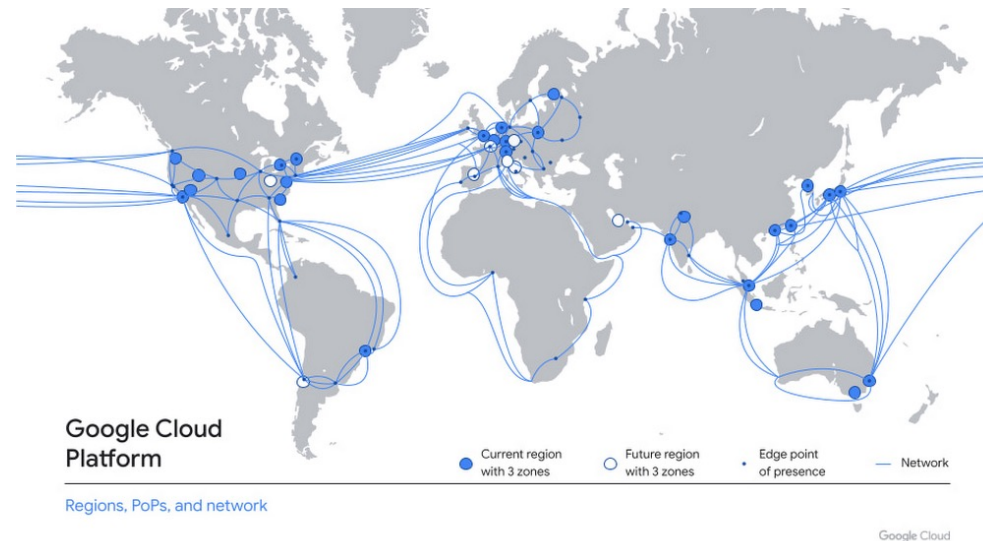
- A distributed load balancer in production at Fastly since 2013
- A solution for the edge
- Work presented at USENIX NSDI 2018 (Renton)

Where the edge fits here?



- Requests enter a specific network through Point of Presence (PoPs)
- Requests that cannot be fulfilled by a PoP is routed through an owned backbone network to a datacenter

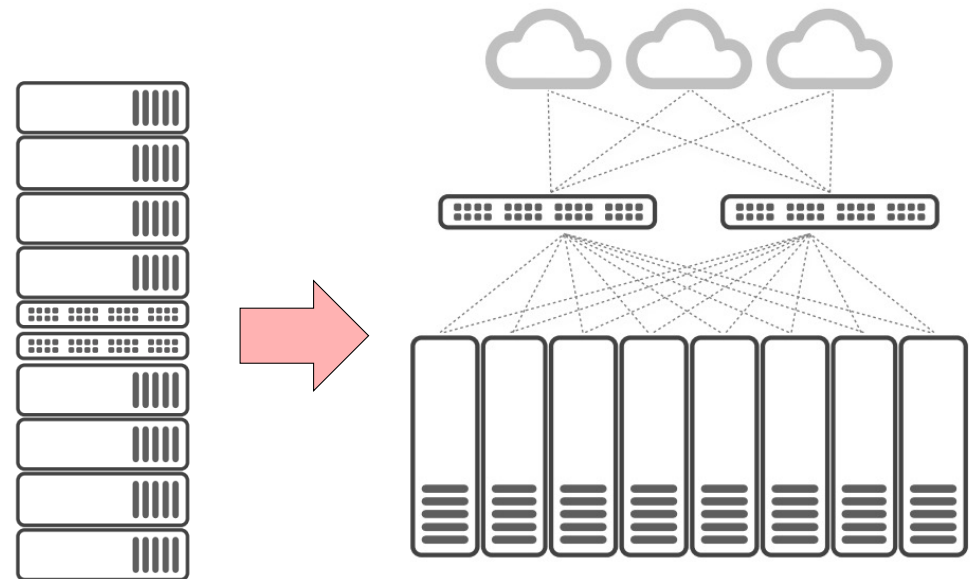
Where the edge fits here?



- Requests enter a specific network through Point of Presence (PoPs)
- Requests that cannot be fulfilled by a PoP is routed through an owned backbone network to a datacenter

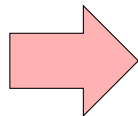
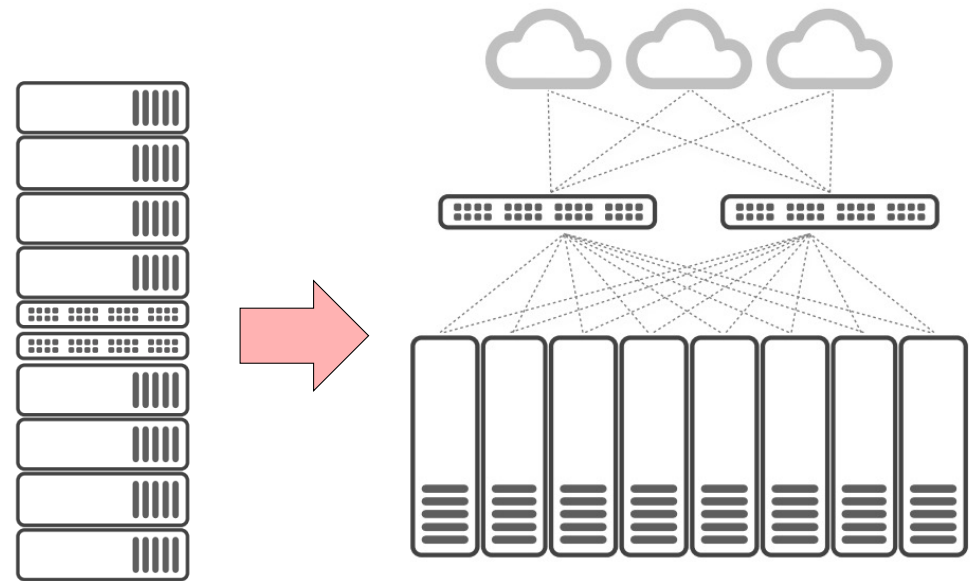
PoP: a small datacenter

- Requirements
 - High performance
 - No single point of failure
 - Absorb DDoS attacks
 - No service disruption on maintenance
- A new load balancing architecture which is *efficient*, *resilient* and *graceful*



PoP: a small datacenter

- Requirements
 - High performance
 - No single point of failure
 - Absorb DDoS attacks
 - No service disruption on maintenance
- A new load balancing architecture which is *efficient*, *resilient* and *graceful*

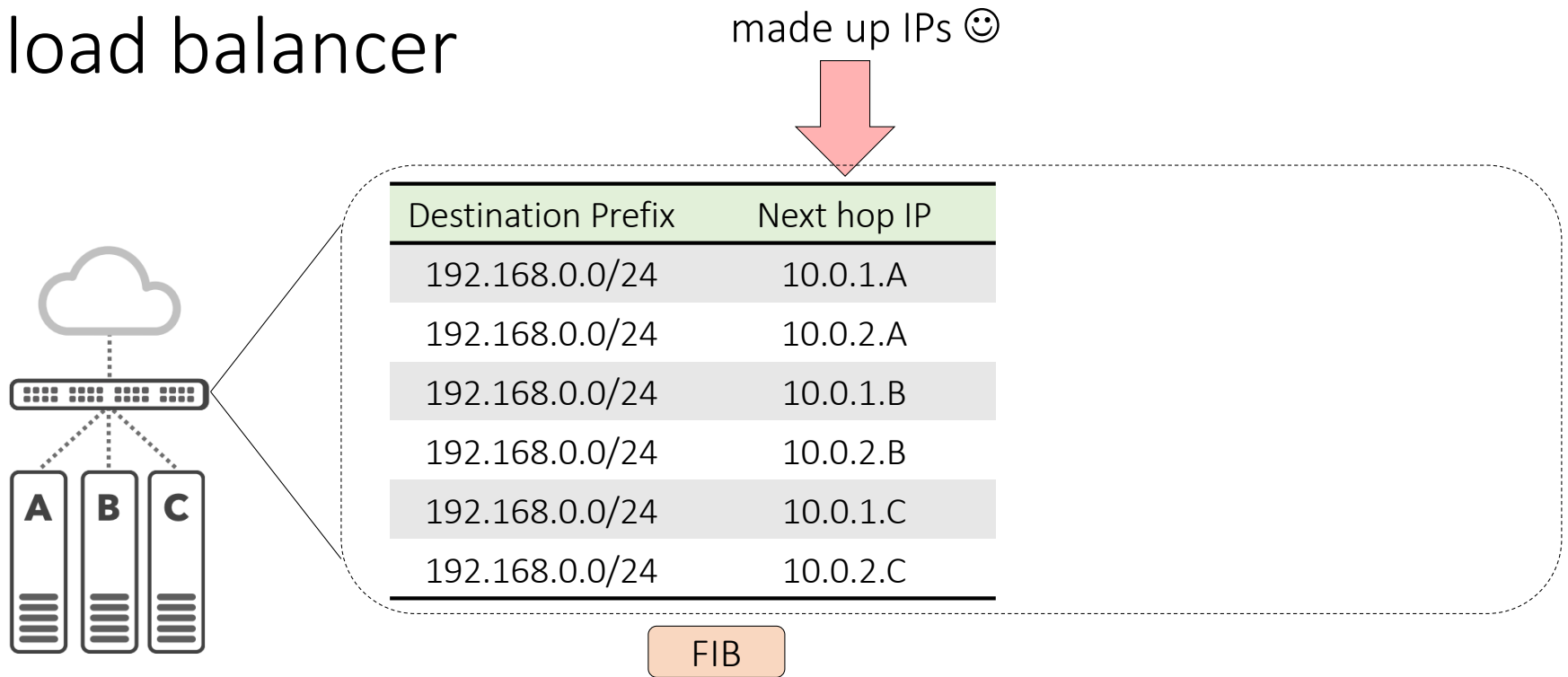


Efficient: no dedicated hardware. No much space available

Resilient: anything that maintains state is easy to DDoS

Graceful: ability to change services without disrupting active flows

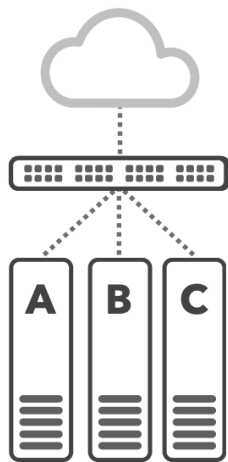
Faild load balancer



Idea:

- Map VIP to static set of virtual next hops

Faild load balancer



Destination Prefix	Next hop IP	IP address	MAC address
192.168.0.0/24	10.0.1.A	10.0.1.A	xx:xx:xx:xx:xx:a
192.168.0.0/24	10.0.2.A	10.0.2.A	xx:xx:xx:xx:xx:a
192.168.0.0/24	10.0.1.B	10.0.1.B	xx:xx:xx:xx:xx:b
192.168.0.0/24	10.0.2.B	10.0.2.B	xx:xx:xx:xx:xx:b
192.168.0.0/24	10.0.1.C	10.0.1.C	xx:xx:xx:xx:xx:c
192.168.0.0/24	10.0.2.C	10.0.2.C	xx:xx:xx:xx:xx:c

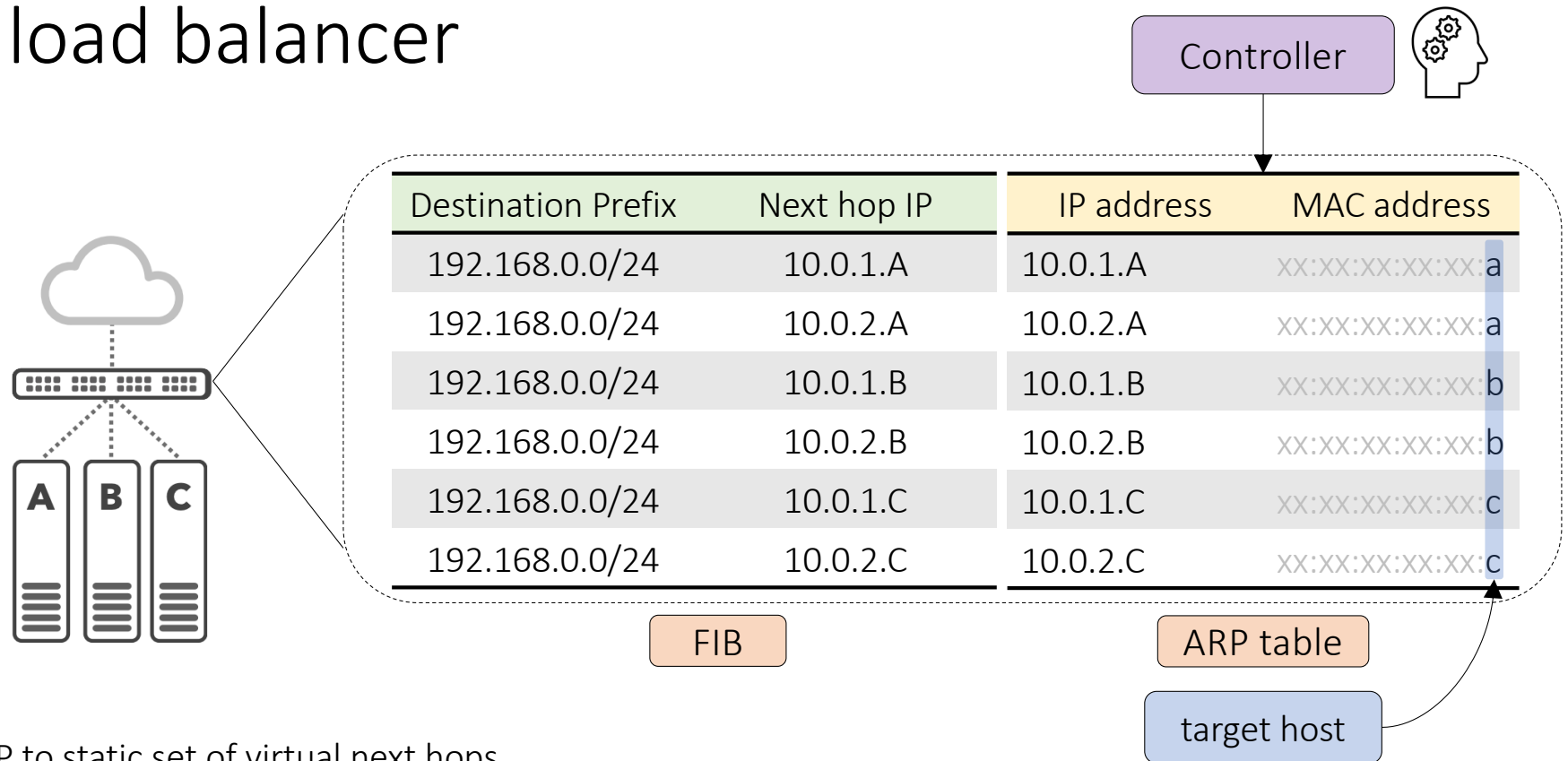
FIB

ARP table

Idea:

- Map VIP to static set of virtual next hops

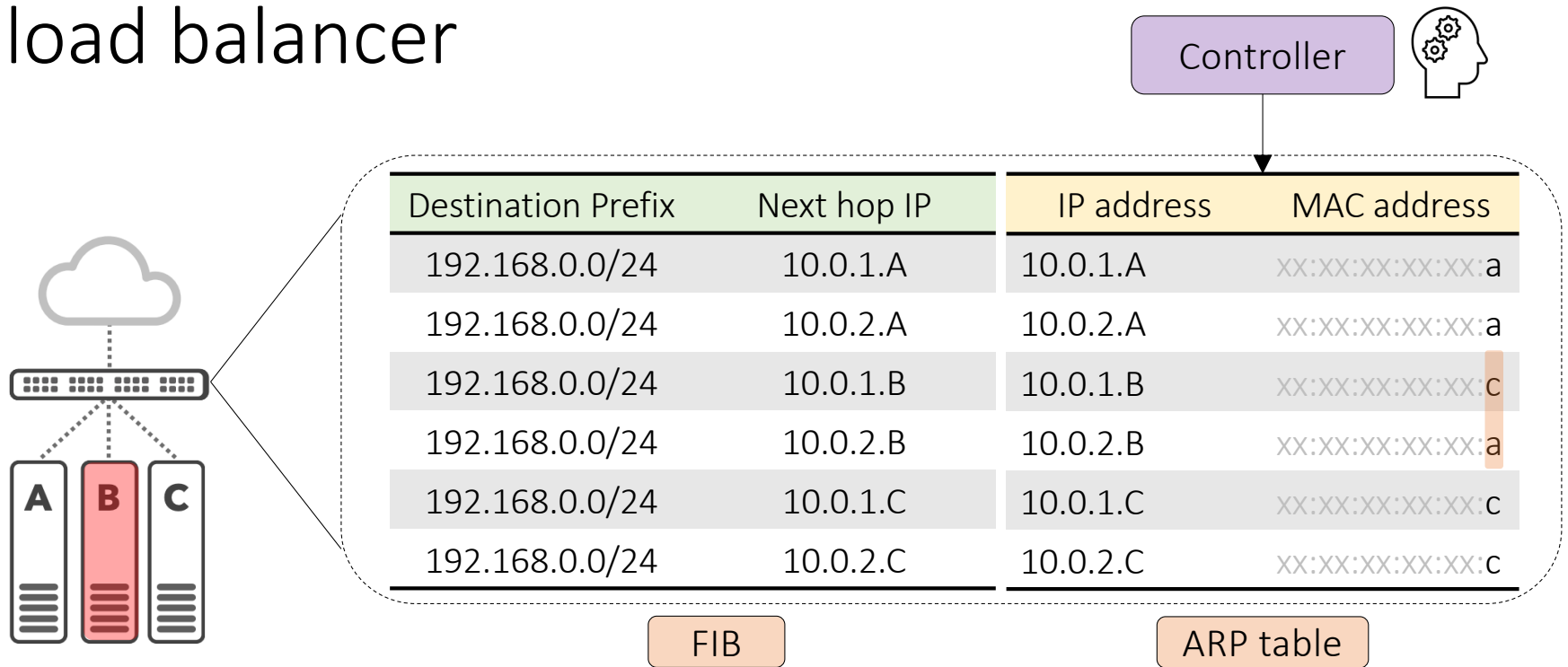
Faild load balancer



Idea:

- Map VIP to static set of virtual next hops
- Control forwarding manipulating ARP entries

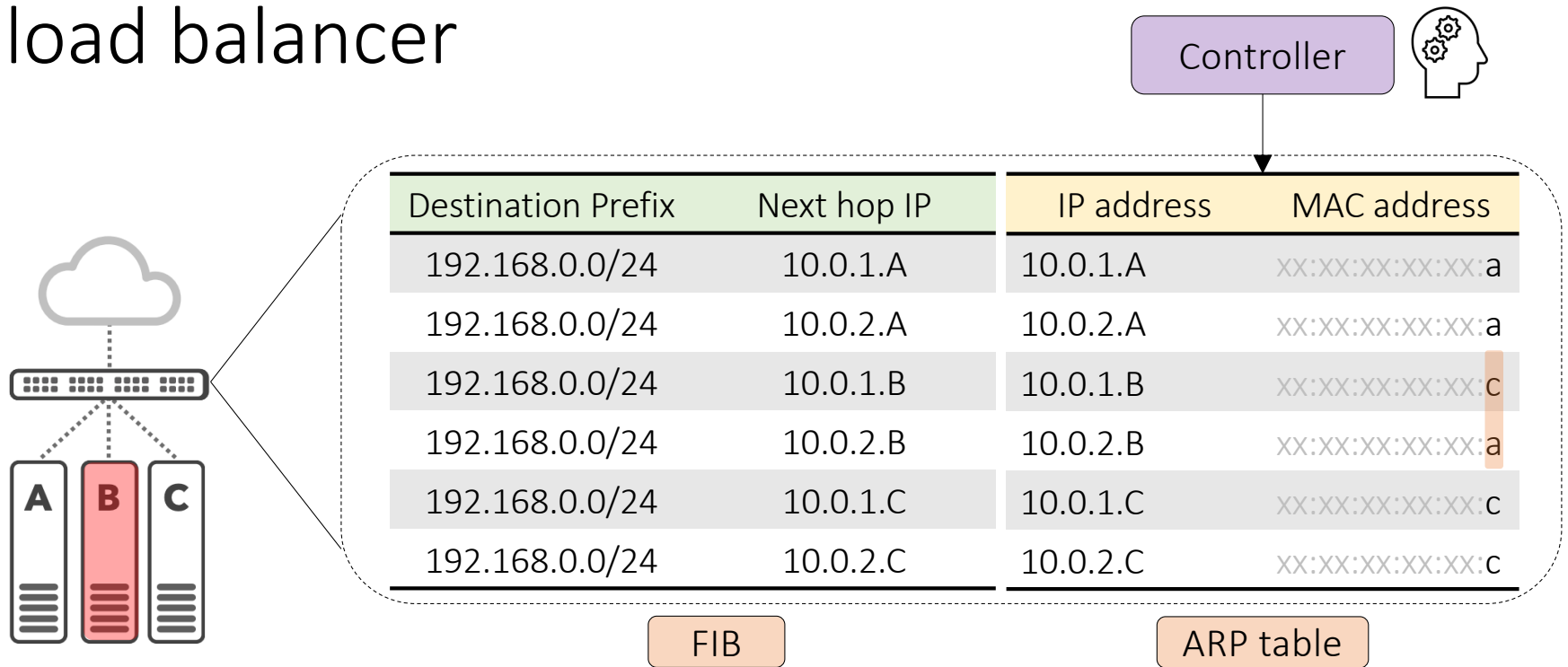
Faild load balancer



Hosts send status to controller

- On drain, update ARP entry
- Balance virtual next hops across available servers

Faild load balancer

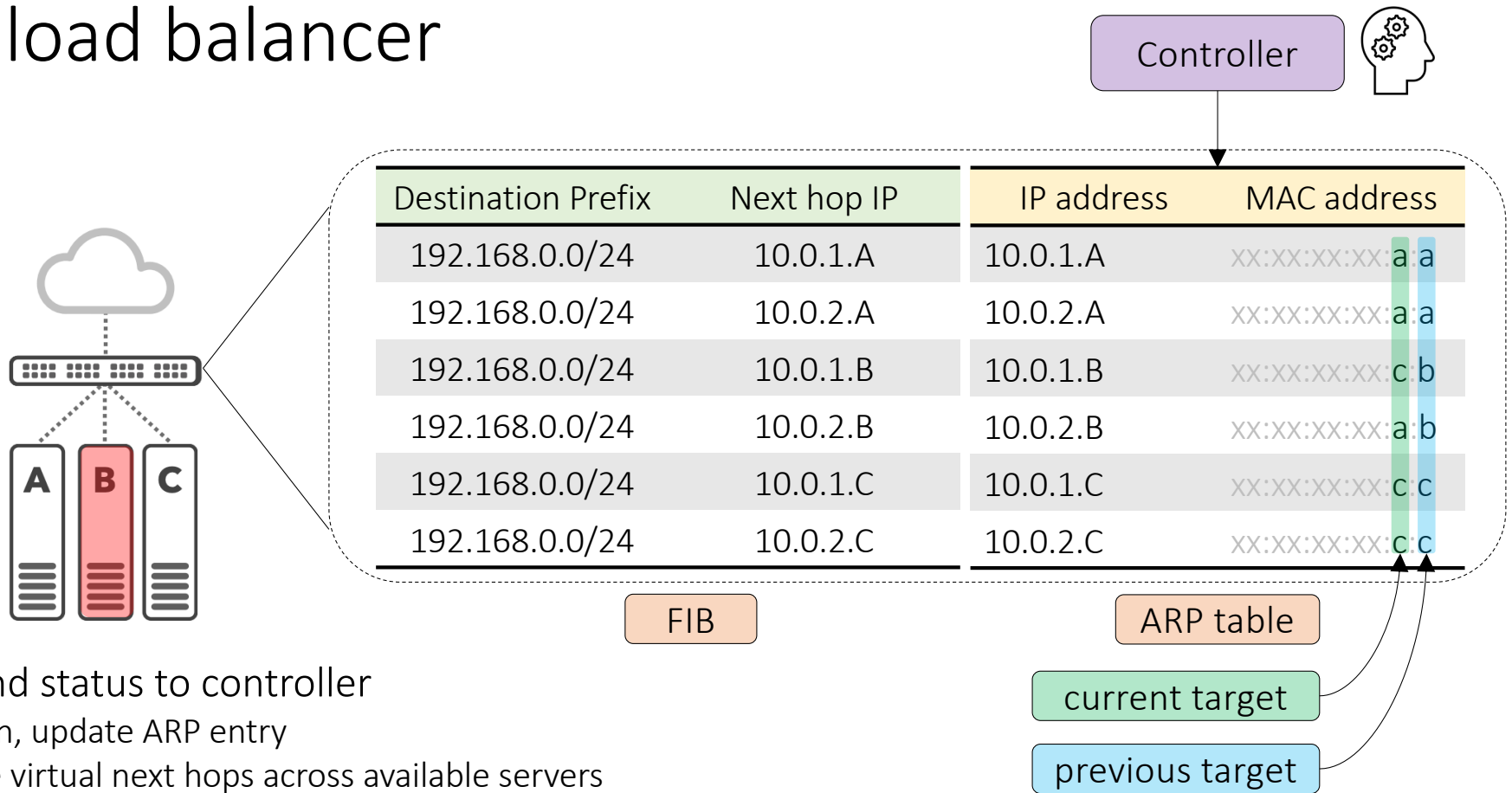


Hosts send status to controller

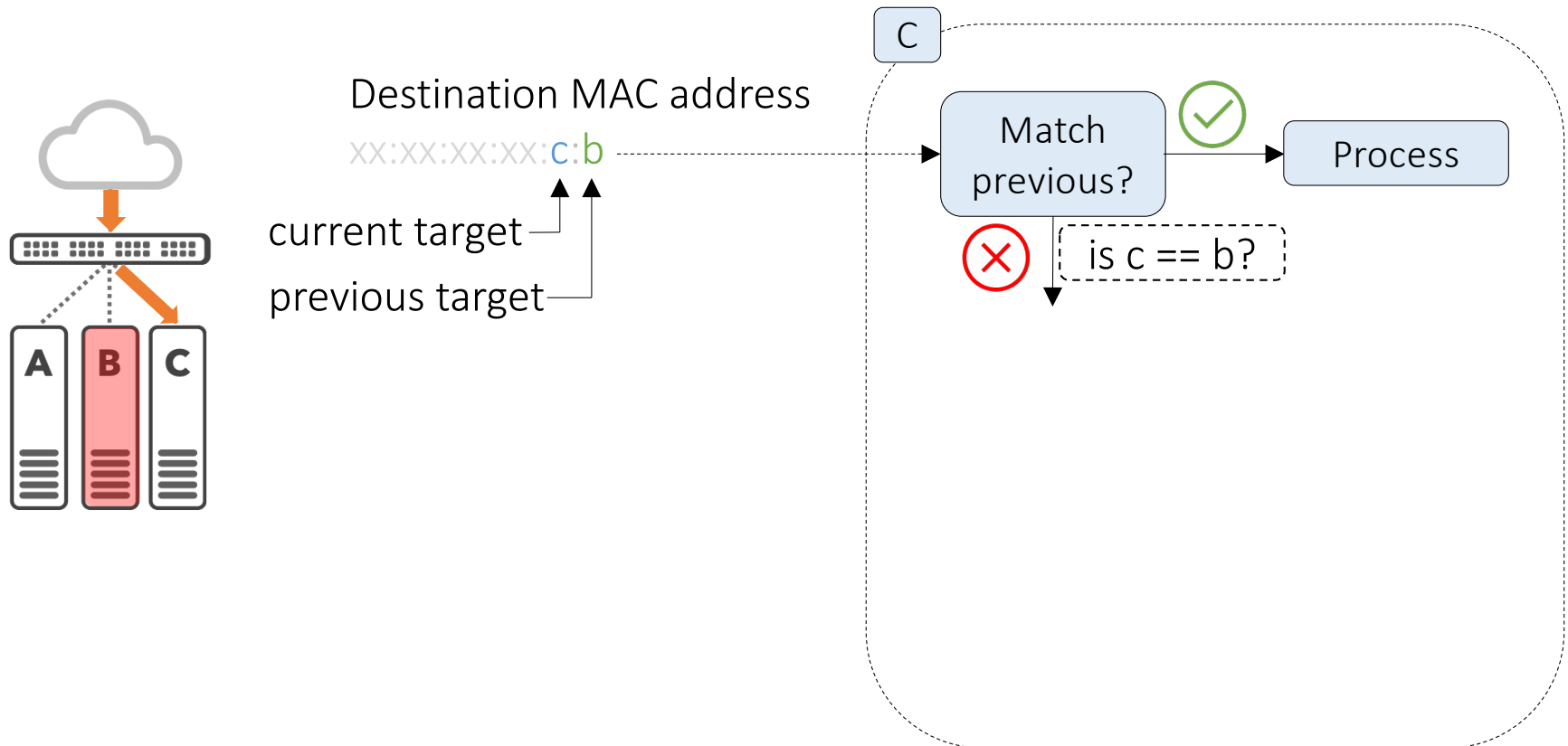
- On drain, update ARP entry
- Balance virtual next hops across available servers

Why this is better than just consistent hashing?

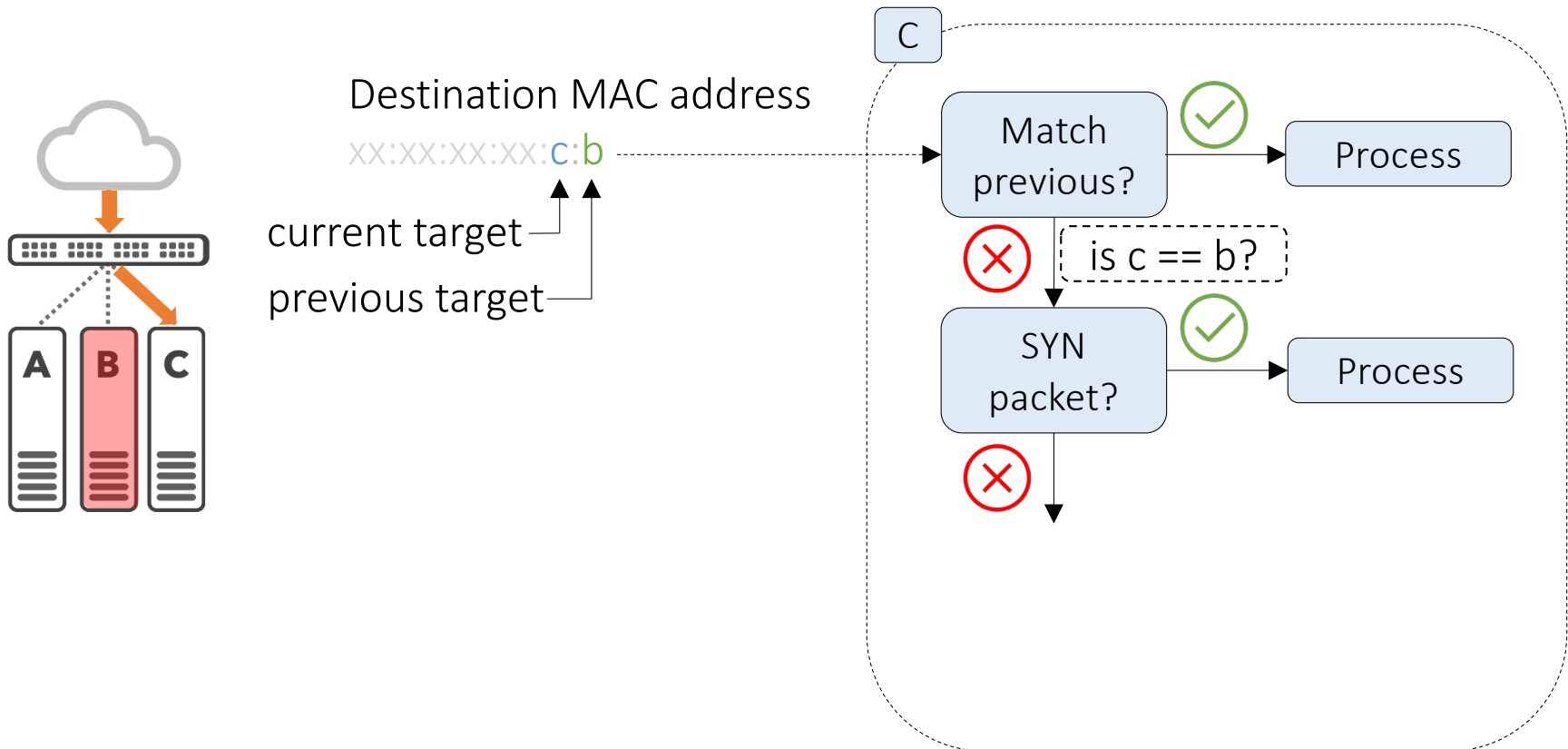
Faild load balancer



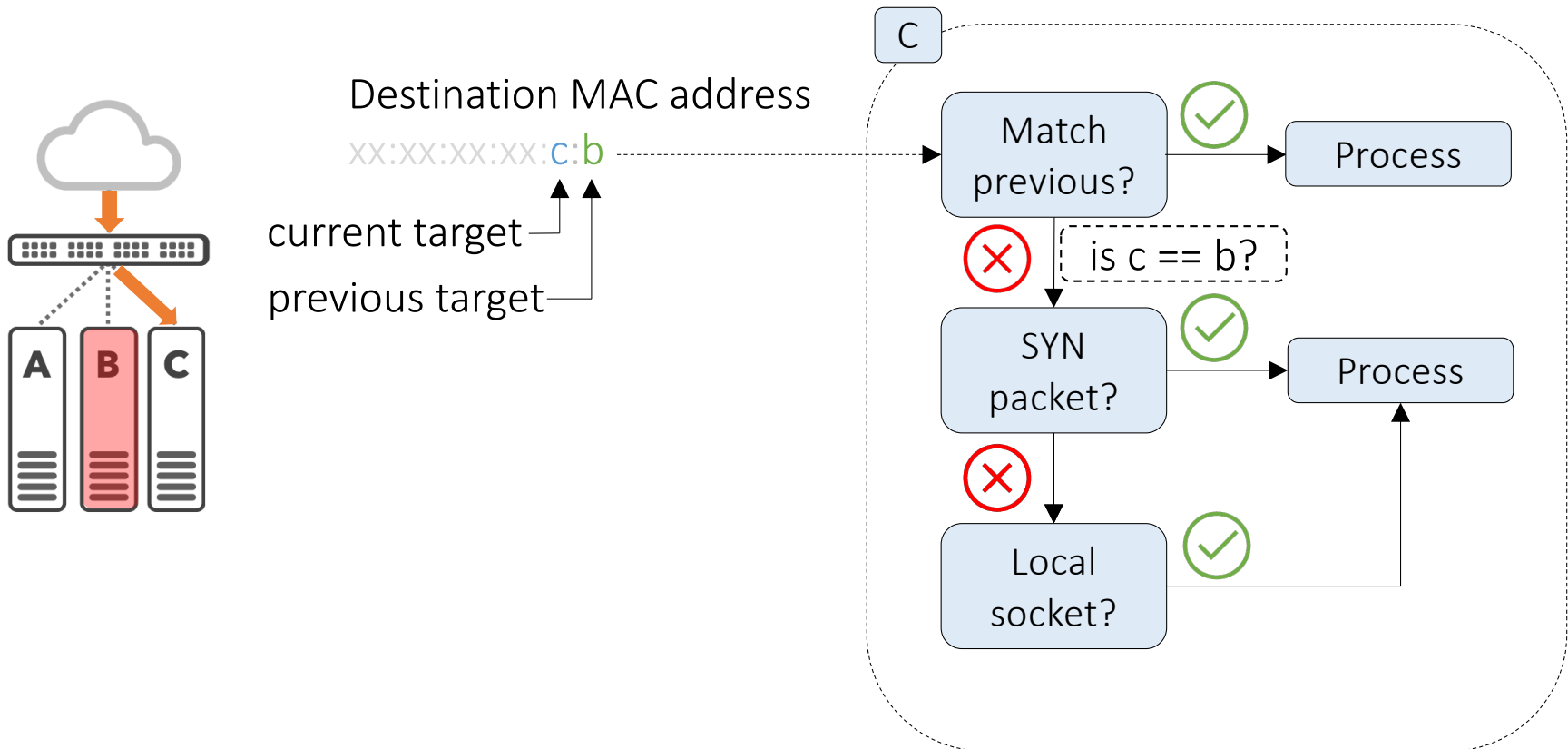
Host processing



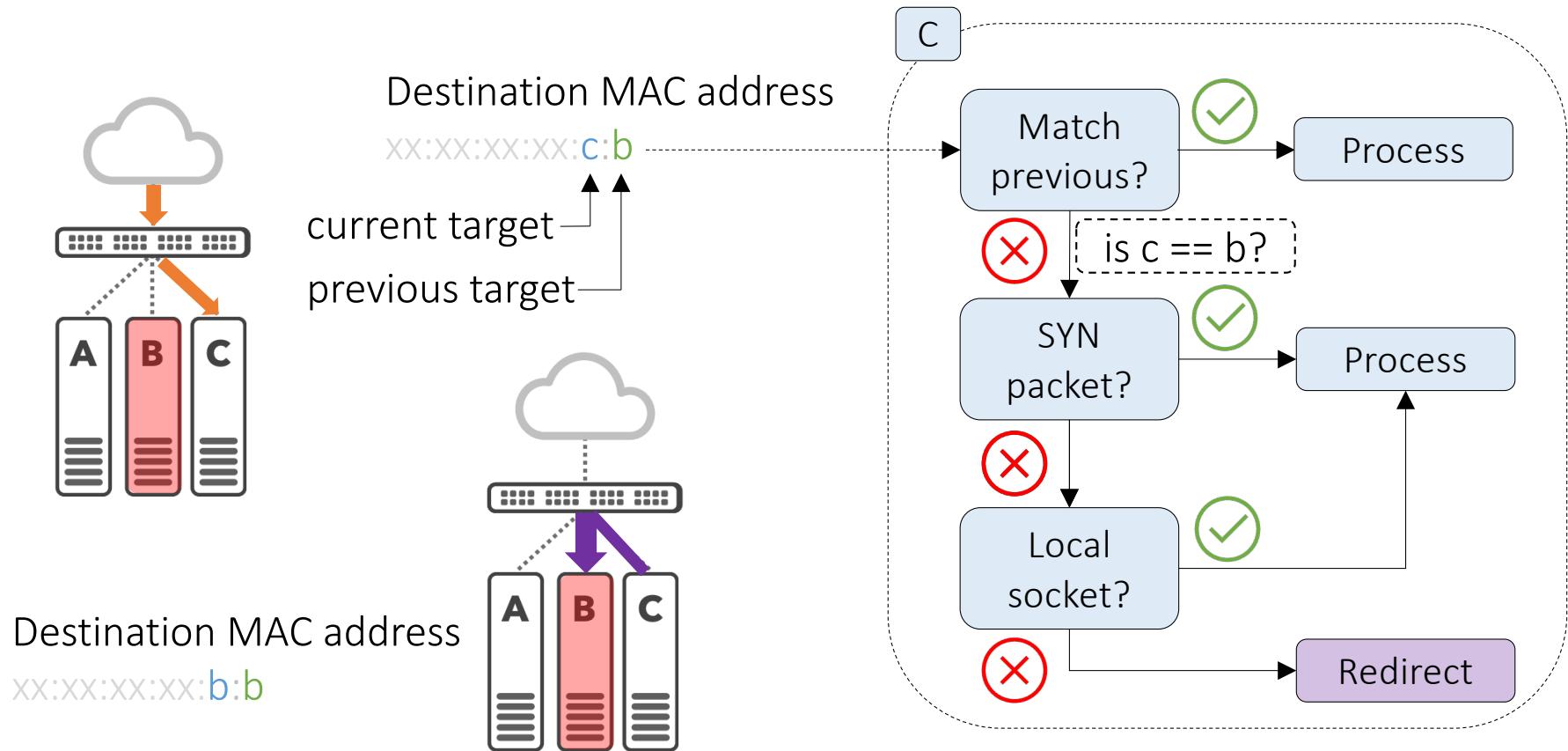
Host processing



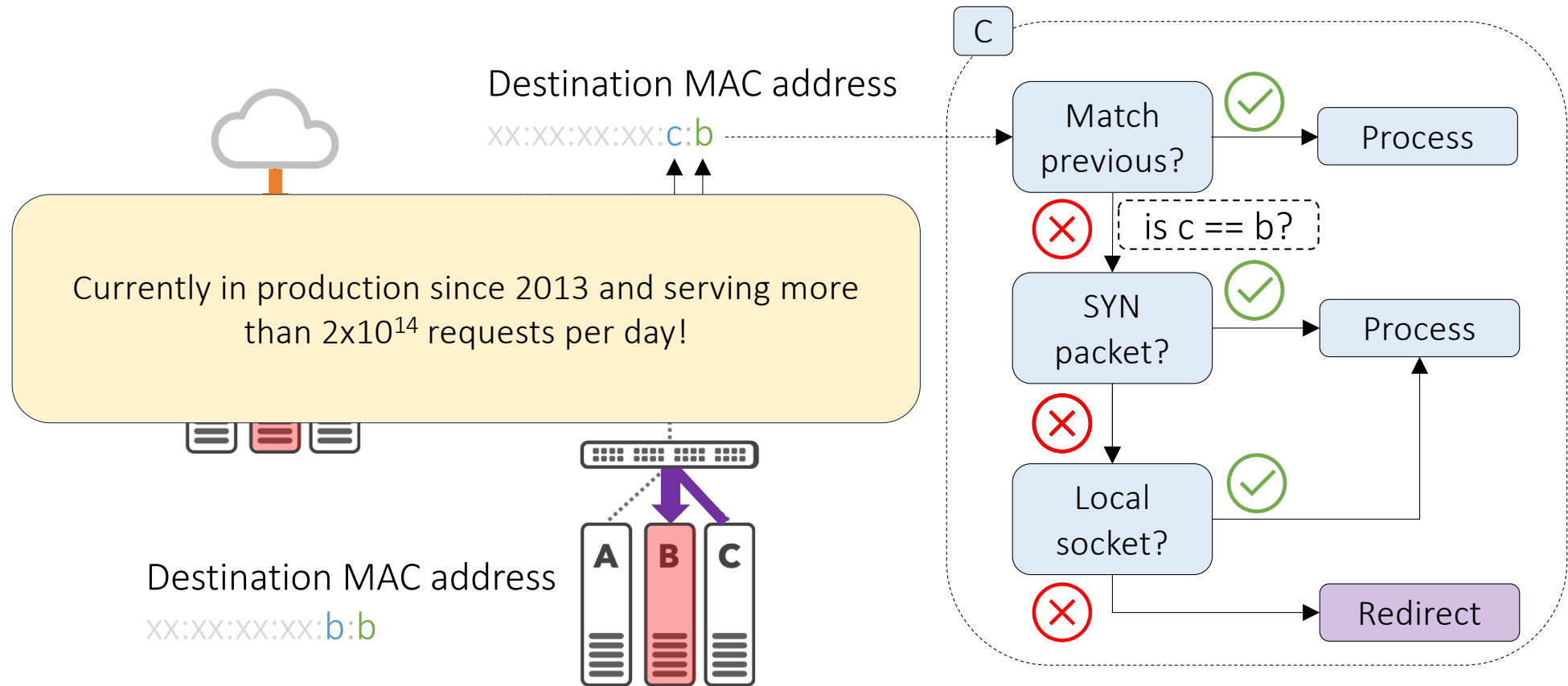
Host processing



Host processing



Host processing



Host processing

